

Universidad Sergio Arboleda

Escuela de Ciencias Exactas e Ingeniería

Pregrado en Ciencias de la Computación e Inteligencia Artificial

OrbitEngine

Desarrollo e Implementación de una Plataforma SaaS Multi-Tenant
para la Gestión Integral de Procesos Internos
en Pequeñas y Medianas Empresas

Autores

Nicolás Rodríguez Forero

Daniel Velasco González

Fabián Rincón Suárez

Semillero de Software como Innovación

Directores:

Juan Pablo Ospina López

Camilo Enrique Rodríguez Torres

Bogotá, Colombia — Abril de 2026

Índice general

Capítulo 1 — Introducción	9
1.1 Planteamiento del Problema	9
1.2 Justificación	10
1.2.1 Justificación Económica y Social	10
1.2.2 Justificación Tecnológica	11
1.2.3 Justificación Académica	12
1.3 Objetivos	13
1.3.1 Objetivo General	13
1.3.2 Objetivos Específicos	13
1.4 Alcance y Limitaciones	14
1.4.1 Alcance del Sistema	14
1.4.2 Limitaciones y Exclusiones	15
1.4.3 Supuestos	15
1.5 Metodología	16
1.6 Estructura del Documento	16
Capítulo 2 — Marco de Referencia	18
2.1 Marco Conceptual	18
2.1.1 Pequeña y Mediana Empresa (Pyme)	18
2.1.2 Sistema de Planificación de Recursos Empresariales (ERP)	19
2.1.3 Software como Servicio (SaaS)	19
2.1.4 Arquitectura Multi-Tenant	20
2.1.5 API REST y OpenAPI	20
2.1.6 Control de Acceso Basado en Roles (RBAC)	21
2.1.7 Integración y Entrega Continua (CI/CD)	21
2.2 Estado del Arte	22
2.2.1 Contexto de la Digitalización de Pymes en Latinoamérica	22
2.2.2 Soluciones Existentes en el Mercado	22
Odoo Community / Enterprise	22
Zoho One	23
QuickBooks	23
Alegra	24
Siigo / Defontana	24

Análisis Comparativo	24
2.2.3 Revisión de Literatura Académica	25
ERP en Pymes: Factores de Éxito y Fracaso	25
Arquitecturas SaaS Multi-Tenant	26
Usabilidad en Sistemas ERP para Usuarios No Técnicos	26
2.3 Marco Tecnológico	26
2.3.1 FastAPI (Backend)	27
2.3.2 React (Frontend)	27
2.3.3 PostgreSQL	27
2.3.4 Docker, Railway y Vercel	28
2.3.5 SQLModel	28
Capítulo 3 — Análisis y Diseño del Sistema	29
3.1 Proceso de Levantamiento de Requisitos	29
3.1.1 Metodología de Levantamiento	29
3.1.2 Perfiles de Usuario	30
3.1.3 Historias de Usuario Principales	31
3.2 Requisitos del Sistema	33
3.2.1 Requisitos Funcionales	33
RF-AUTH — Módulo de Autenticación y Usuarios	33
RF-INV — Módulo de Gestión de Inventario	33
RF-SAL — Módulo de Gestión de Ventas	34
RF-CUS — Módulo de Gestión de Clientes	35
RF-REP — Módulo de Reportes y Dashboard	35
3.2.2 Requisitos No Funcionales	36
3.3 Arquitectura del Sistema	37
3.3.1 Decisiones Arquitectónicas	37
3.3.2 Vista de Componentes de Alto Nivel	38
3.3.3 Arquitectura del Backend en Capas	38
3.3.4 Estrategia de Multi-Tenancy	40
3.3.5 Flujos Principales del Sistema	40
3.4 Diseño del Modelo de Datos	41
3.4.1 Entidades Principales	41
3.4.2 Diagrama Entidad-Relación	42
3.4.3 Tabla: organizations	42
3.4.4 Tabla: users	43
3.4.5 Tabla: products	43
3.4.6 Tabla: sales	44
3.4.7 Estrategia de Índices	44
3.4.8 Estrategia de Soft Delete	44
3.5 Diseño de la Interfaz de Usuario	45
3.5.1 Principios de Diseño	45
3.5.2 Estructura de Navegación	45
3.5.3 Sistema de Diseño	46

Capítulo 4 — Desarrollo e Implementación	47
4.1 Metodología de Desarrollo	47
4.1.1 Marco Ágil: Scrum Adaptado	47
4.1.2 Herramientas del Proceso	48
4.1.3 Estructura del Equipo	48
4.1.4 Gestión de la Calidad del Código	48
4.2 Fases y Sprints de Desarrollo	49
Fase 1 — Documentación e Investigación (octubre 2025, semanas 1–3) . . .	49
Fase 2 — Diseño y Arquitectura (octubre–noviembre 2025, semanas 4–5) . .	49
Fase 3 — Desarrollo Core (noviembre 2025 – segunda semana de abril 2026, sprints 1–8)	50
Sprint 1: Autenticación y Setup Base (3–14 noviembre 2025 18 SP) .	50
Sprint 2: Inventario Core — CRUD de Productos (17 noviembre – 5 diciembre 2025 17 SP)	50
Sprint 3: Inventario Avanzado (8–19 diciembre 2025 16 SP)	51
Sprint 4: Módulo de Ventas (5–23 enero 2026 20 SP)	51
Sprint 5: Módulo de Clientes (26 enero – 13 febrero 2026 15 SP) . .	51
Sprint 6: Reportes y Dashboard (16 febrero – 6 marzo 2026 18 SP) .	52
Sprint 7: Roles, Permisos y Refinamiento Funcional (9–27 marzo 2026 16 SP)	52
Sprint 8: Cierre Funcional y Pulido (30 marzo – 10 abril 2026 17 SP)	52
Fase 4 — Estabilización, Despliegue y Refinamiento (mediados de abril – última semana de abril 2026, sprints 9–10)	53
Sprint 9: Pruebas de Carga, Rendimiento y Refinamiento Interno (13– 17 abril 2026 16 SP)	53
Sprint 10: Despliegue en Producción y Validación Interna (20–24 abril 2026 18 SP)	53
Fase 5 — Validación con Empresas Piloto (27 de abril – 4 de mayo de 2026) .	54
Fase 6 — Documentación Final y Entrega (5 de mayo – 15 de mayo de 2026)	54
4.3 Implementación de Módulos Clave	54
4.3.1 Sistema de Autenticación y Multi-Tenancy	54
4.3.2 Módulo de Inventario y Gestión de Stock	56
4.3.3 Módulo de Ventas y Consistencia Transaccional	56
4.4 Estrategia y Resultados de Pruebas	57
4.4.1 Tipos de Pruebas Implementadas	57
4.4.2 Pipeline de CI/CD	58
4.4.3 Métricas de Cobertura	59
4.4.4 Pruebas de Carga y Rendimiento Planeadas	59
4.5 Infraestructura de Despliegue	60
4.5.1 Arquitectura de Producción	60
4.5.2 Estrategia de Secretos y Configuración	61
4.5.3 Monitoreo y Observabilidad	61
Capítulo 5 — Resultados Técnicos	62

5.1 Marco de la Validación Técnica	62
5.1.1 Objetivos	62
5.1.2 Ambiente de Pruebas	63
5.1.3 Criterios de Aceptación	63
5.1.4 Herramientas Empleadas	64
5.1.5 Composición del Piloto Técnico	64
5.2 Pruebas de Carga (Backend / API) — Locust	66
5.2.1 Diseño del Experimento	66
5.2.2 Perfiles de Usuario Virtual	67
5.2.3 Escenarios Ejecutados	68
5.2.4 Resultados por Escenario	68
5.2.4.1 Test 01 — Carga Normal de Pyme	68
5.2.4.2 Test 02 — Estrés Moderado (~50 usuarios)	69
5.2.4.3 Tests 03 a 05 — Régimen de Saturación	70
5.2.4.4 Test 06 — Pico Sostenido (~200 usuarios)	70
5.2.5 Curva de Degradación	70
5.2.6 Comportamiento por Endpoint	71
5.2.7 Cumplimiento del RNF-01	72
5.3 Pruebas de Rendimiento (Frontend / Web Vitals)	73
5.3.1 Lighthouse (Auditoría de Laboratorio)	73
5.3.2 PageSpeed Insights (Infraestructura de Google)	74
5.3.3 WebPageTest (Red Real con Captura de Video)	76
5.3.4 Cumplimiento de Core Web Vitals	76
5.4 Síntesis de Cumplimiento de Requisitos No Funcionales	77
5.5 Interpretación y Limitaciones	78
5.5.1 Interpretación Breve de los Hallazgos	78
5.5.2 Limitaciones de Escalabilidad Inherentes al Concepto del Proyecto	79
Capítulo 6 — Resultados de Usuarios	81
6.1 Marco Metodológico de la Validación con Usuarios	81
6.1.1 Objetivos	81
6.1.2 Diseño de Investigación	82
6.1.3 Participantes	82
6.1.4 Instrumentos	83
6.1.5 Procedimiento y Cronograma (Fase 5)	84
6.1.6 Hipótesis a Contrastar	84
6.1.7 Consideraciones Éticas	85
6.2 Caracterización de las Empresas Piloto	86
6.3 Eficiencia Operativa (pre/post)	87
6.3.1 Tiempos en Tareas Administrativas	87
6.3.2 Tasa de Error en Operaciones de Inventario	88
6.3.3 Síntesis Cuantitativa de la Mejora	89
6.4 Pruebas de Tareas Guiadas	90
6.5 Encuesta de Usabilidad — SUS	93

6.5.1 Score por Usuario y Agregado por Empresa	93
6.5.2 Score Global del Piloto vs. Benchmark 68	93
6.5.3 Análisis por Ítem (Positivos vs. Negativos)	94
6.6 Satisfacción Específica	95
6.6.1 NPS por Empresa y Agregado	95
6.6.2 CSAT por Módulo	96
6.6.3 Ranking de Módulos	97
6.7 Telemetría de Uso en Producción	97
6.8 Hallazgos Cualitativos de las Entrevistas	100
6.8.1 Temas Emergentes	100
6.8.2 Citas Representativas	101
6.8.3 Estudios de Caso Narrativos	103
6.8.3.1 Frozt Bitez	103
6.8.3.2 Miss Peggy	104
6.8.3.3 Luana Handmade	105
6.9 Validación de Hipótesis	106
6.9.1 H1 — Hipótesis de Eficiencia Operativa	106
6.9.2 H2 — Hipótesis de Precisión en Inventario	107
6.9.3 H3 — Hipótesis de Usabilidad	107
6.9.4 Tabla Resumen de Validación de Hipótesis	108
6.10 Limitaciones de la Validación con Usuarios	109
Capítulo 7 — Conclusiones y Trabajo Futuro	111
7.1 Conclusiones por Objetivo	111
7.2 Conclusión General	114
7.3 Cumplimiento de Hipótesis	115
7.4 Limitaciones del Proyecto	116
7.4.1 Limitaciones Técnicas	116
7.4.2 Limitaciones de la Validación con Usuarios	117
7.5 Recomendaciones de Optimización Derivadas de los Hallazgos	117
7.5.1 Backend	117
7.5.2 Frontend	118
7.5.3 Infraestructura	118
7.5.4 Producto	119
7.6 Trabajo Futuro	120
7.6.1 Evolución del Producto	120
7.6.2 Incorporación de Inteligencia Artificial	121
7.6.3 Investigación Académica	122
7.7 Reflexión Final	123
Referencias	124
Anexo A — Manual de Usuario	128
A.1 Introducción	128

A.2 Acceso a la Plataforma	129
A.2.1 Página de inicio	129
A.2.2 Registro de organización	129
A.2.3 Invitación de usuarios	129
A.2.4 Inicio de sesión	129
A.2.5 Recuperación de contraseña	130
A.2.6 Cierre de sesión	130
A.3 Navegación General	130
A.4 Dashboard — Panel de Indicadores	131
A.5 Módulo de Inventario	132
A.5.1 Categorías	132
A.5.2 Productos	132
A.5.3 Ajuste de stock manual	133
A.5.4 Historial de movimientos	133
A.6 Módulo de Ventas	133
A.6.1 Registrar una venta	133
A.6.2 Ver detalle de una venta	134
A.6.3 Cancelar una venta	134
A.6.4 Filtros y búsqueda	135
A.7 Módulo de Clientes	135
A.7.1 Registrar un cliente	135
A.7.2 Editar un cliente	135
A.7.3 Desactivar un cliente	135
A.7.4 Historial de compras del cliente	136
A.8 Módulo de Administración	136
A.8.1 Ver usuarios	136
A.8.2 Crear un usuario	136
A.8.3 Editar un usuario	137
A.8.4 Eliminar un usuario	137
A.9 Configuración	137
A.9.1 Información personal	137
A.9.2 Cambio de contraseña	137
A.9.3 Configuración de la organización	137
A.9.4 Apariencia	138
A.9.5 Eliminación de cuenta	138
A.10 Preguntas Frecuentes	138
Anexo B — Manual de Instalación y Despliegue	140
B.1 Visión General	140
B.2 Prerrequisitos	141
B.2.1 Herramientas requeridas	141
B.2.2 Puertos utilizados	141
B.3 Configuración de Variables de Entorno	141
B.4 Entorno de Desarrollo Local	143

B.4.1 Iniciar el entorno	144
B.4.2 URLs del entorno local	144
B.4.3 Detener el entorno	144
B.4.4 Ver logs	145
B.5 Migraciones de Base de Datos	145
B.5.1 Aplicar migraciones pendientes	145
B.5.2 Crear una nueva migración	145
B.5.3 Ver historial de migraciones	145
B.5.4 Revertir la última migración	146
B.5.5 Migraciones del proyecto	146
B.6 Pruebas	146
B.6.1 Pruebas del backend (pytest)	146
B.6.2 Pruebas del frontend (Playwright E2E)	147
B.7 Generación del Cliente API	148
B.8 Despliegue en Producción	148
B.8.1 Prerrequisitos del servidor	148
B.8.2 Preparar la red de Traefik	149
B.8.3 Configurar Traefik con TLS	149
B.8.4 Configurar variables de entorno para producción	149
B.8.5 Construir y desplegar	150
B.8.6 Actualización de la aplicación	150
B.9 Monitoreo y Mantenimiento	151
B.9.1 Estado de los servicios	151
B.9.2 Backup de la base de datos	151
B.9.3 Limpiar recursos no usados	152
B.10 Solución de Problemas	152
Anexo C — Documentación Técnica	153
C.1 Stack Tecnológico	153
C.1.1 Backend	153
C.1.2 Frontend	154
C.1.3 Infraestructura	154
C.2 Estructura del Proyecto	154
C.3 Autenticación y Autorización	158
C.3.1 Flujo de autenticación	158
C.3.2 Control de acceso basado en roles (RBAC)	159
C.3.3 Protección contra fuerza bruta	160
C.4 API REST — Referencia de Endpoints	160
C.4.1 Autenticación (/login)	160
C.4.2 Usuarios (/users)	160
C.4.3 Organizaciones (/organizations)	161
C.4.4 Roles (/roles)	161
C.4.5 Categorías (/categories)	161
C.4.6 Productos (/products)	161

C.4.7 Clientes (/customers)	162
C.4.8 Ventas (/sales)	162
C.4.9 Movimientos de inventario (/inventory-movements)	162
C.4.10 Dashboard (/dashboard)	163
C.4.11 Utilidades	163
C.5 Modelos de Datos	164
C.5.1 Organization	164
C.5.2 Role	164
C.5.3 User	165
C.5.4 Category	165
C.5.5 Product	166
C.5.6 Customer	167
C.5.7 Sale	168
C.5.8 SaleItem	169
C.5.9 InventoryMovement	169
C.6 Variables de Entorno	170
C.6.1 Variables requeridas	170
C.6.2 Variables opcionales con valores por defecto	170
C.6.3 Propiedades calculadas (no en .env)	171
C.7 Convenciones de Código	171
C.7.1 Backend (Python)	171
C.7.2 Frontend (TypeScript/React)	173
C.8 Pipeline de CI/CD	174
C.8.1 Workflow de pruebas (en cada PR y push a main)	175
C.8.2 Workflow de despliegue	175
C.9 Estructura de Pruebas	175
C.9.1 Backend (pytest)	175
C.9.2 Frontend (Playwright E2E)	176
C.10 Generación del Cliente API	177
C.11 Multi-Tenancy	177

Capítulo 1 — Introducción

1.1 Planteamiento del Problema

Las pequeñas y medianas empresas (pymes) constituyen el motor económico de América Latina. Según la Comisión Económica para América Latina y el Caribe (CEPAL, 2022), las pymes representan más del 99 % del tejido empresarial en la región y generan entre el 60 % y el 70 % del empleo formal. Sin embargo, pese a su relevancia, enfrentan una brecha estructural frente a las grandes corporaciones en materia de gestión operativa y adopción tecnológica.

Un diagnóstico recurrente en la literatura especializada señala que la gran mayoría de las pymes latinoamericanas aún gestionan sus operaciones —inventario, ventas, clientes y contabilidad básica— mediante herramientas rudimentarias: cuadernos físicos, hojas de cálculo de Excel no estructuradas o sistemas ad hoc sin integración entre módulos. De acuerdo con el Banco Interamericano de Desarrollo (BID, 2021), solo el 16 % de las pymes de la región ha adoptado algún tipo de software de gestión empresarial, frente a un 74 % en países de la OCDE.

Esta brecha de digitalización se traduce en consecuencias concretas y medibles para el negocio:

- **Errores de inventario:** la gestión manual provoca discrepancias frecuentes entre el stock físico y el registrado, generando roturas de stock o sobrestock. Ambos escenarios tienen costos directos: ventas perdidas en el primer caso, capital inmovilizado en el

segundo.

- **Ineficiencia operativa:** los empleados dedican tiempo significativo a tareas administrativas repetitivas —conciliación de registros, elaboración de reportes manuales, búsqueda de información— tiempo que podría orientarse a actividades de mayor valor.
- **Toma de decisiones reactiva:** sin acceso a datos consolidados y en tiempo real, los dueños de pymes toman decisiones basadas en intuición antes que en evidencia. El abastecimiento de productos, la identificación de clientes de alto valor y la detección de tendencias de ventas se realizan de manera subjetiva, sin soporte de indicadores ni reportes estructurados.

Las soluciones existentes en el mercado presentan limitaciones específicas para este segmento. Los sistemas ERP de nivel empresarial (SAP, Oracle) son prohibitivos en costo y complejidad para una pyme. Las alternativas de código abierto como Odoo, si bien potentes, requieren infraestructura propia y capacidad técnica para su instalación, configuración y mantenimiento. Las soluciones SaaS disponibles en el mercado latinoamericano (Alegra, Siigo, Defontana) se enfocan principalmente en facturación electrónica y contabilidad, sin integrar capacidades de gestión de inventario, análisis de clientes y reportes operativos en un único entorno accesible.

Se identifica así un problema de investigación y desarrollo bien definido: **la inexistencia de una plataforma SaaS integrada, accesible y orientada al contexto latinoamericano, que combine la gestión operativa esencial de una pyme —inventario, ventas, clientes y reportes— bajo un modelo de precios y complejidad adecuados para este segmento.**

1.2 Justificación

1.2.1 Justificación Económica y Social

La digitalización de las pymes no es un objetivo exclusivamente tecnológico; es un imperativo económico. El BID estima que cerrar la brecha digital de las pymes en América Latina podría incrementar el PIB regional en hasta un 5 % adicional para 2030. En términos individuales,

los estudios de adopción de ERP en pymes reportan reducciones de costos operativos del 15 % al 30 % y mejoras en la eficiencia de la gestión de inventario de hasta el 40 % en el primer año de implementación (Kumar & van Hillegersberg, 2000; Duan et al., 2012).

La relevancia del problema es, por tanto, tanto académica como práctica: una solución bien diseñada tiene potencial de impacto directo en la competitividad de cientos de pequeñas empresas.

1.2.2 Justificación Tecnológica

Las decisiones tecnológicas que sustentan OrbitEngine no se justifican por la mera disponibilidad de las herramientas seleccionadas, sino por su capacidad de reducir el riesgo técnico del proyecto y acelerar su tiempo de salida al mercado en un contexto de equipo reducido y plazos académicos acotados:

1. **Idoneidad del stack para una plataforma transaccional multi-tenant:** la elección de un backend asíncrono basado en FastAPI con tipado estricto, junto con un frontend declarativo en React, responde a requisitos concretos del dominio —validación rigurosa de datos contables, concurrencia en operaciones de inventario y trazabilidad por organización—, y no a una preferencia genérica por tecnologías populares.
2. **Contrato API como única fuente de verdad:** la adopción de un enfoque *contract-first* mediante OpenAPI permite generar automáticamente el cliente tipado del frontend a partir del esquema del backend, garantizando consistencia entre capas, detectando regresiones en tiempo de compilación y eliminando una clase entera de errores de integración propios de los desarrollos manuales.
3. **Seguridad desde el diseño:** la arquitectura incorpora autenticación basada en JWT, control de acceso por roles a nivel de endpoint y aislamiento lógico de datos por organización como invariantes del modelo, no como capas añadidas a posteriori, lo cual es crítico al manejar información financiera de terceros.
4. **Aprovechamiento de inteligencia artificial generativa en el ciclo de desarrollo:** la incorporación de asistentes de IA en tareas de codificación, documentación y revisión

permitió al equipo sostener un ritmo de entrega comparable al de equipos de mayor tamaño, manteniendo estándares de calidad mediante revisión humana sistemática del código generado.

5. **Viabilidad económica del despliegue en la nube para el segmento pyme:** las plataformas de despliegue gestionado seleccionadas ofrecen, en 2026, un costo marginal por organización lo suficientemente bajo como para que el modelo de suscripción resulte rentable incluso con un volumen reducido de clientes, condición que no se cumplía con la infraestructura disponible una década atrás.

1.2.3 Justificación Académica

La pertinencia académica del presente proyecto se sustenta en tres ejes complementarios que articulan su contribución al conocimiento, su rigor metodológico y su correspondencia con la formación recibida en el programa.

En primer lugar, el proyecto aporta evidencia empírica a un vacío específico de la literatura: si bien la adopción de sistemas de información en pequeñas y medianas empresas ha sido ampliamente documentada en contextos anglosajones y europeos (Kumar & van Hillegersberg, 2000; Duan et al., 2012), los estudios centrados en pymes latinoamericanas —y en particular sobre plataformas SaaS multi-tenant diseñadas para sus restricciones de costo, capacidad técnica y conectividad— son notablemente más escasos. OrbitEngine se propone como un caso de estudio que documenta tanto el diseño arquitectónico como los resultados de su adopción en este contexto regional.

En segundo lugar, la validación del sistema sigue un diseño cuasi-experimental de tipo pre/post con empresas piloto, articulando métricas cuantitativas —reducción del tiempo en tareas administrativas, disminución de la tasa de error en operaciones de inventario y tiempos de respuesta en producción— con métricas cualitativas estandarizadas, en particular el instrumento *System Usability Scale* (SUS) y entrevistas semiestructuradas de cierre. Este enfoque mixto permite contrastar con evidencia las hipótesis sobre el impacto de la digitalización en la eficiencia operativa, más allá de la sola demostración funcional del

software.

En tercer lugar, el proyecto se inscribe en la línea de trabajo del semillero *Software como Innovación* y articula de manera integral las competencias formativas del pregrado en Ciencias de la Computación e Inteligencia Artificial: ingeniería de requisitos, arquitectura de software, desarrollo full-stack, despliegue en la nube, validación experimental y comunicación de resultados. Lo anterior justifica su carácter como proyecto de grado y no como un desarrollo de software meramente aplicado.

1.3 Objetivos

1.3.1 Objetivo General

Desarrollar e implementar una plataforma SaaS multi-tenant para la gestión integral de procesos internos en pequeñas y medianas empresas, que integre módulos de gestión de inventario, ventas, clientes y reportes operativos, y validar su impacto en la eficiencia operativa mediante pruebas con empresas reales.

1.3.2 Objetivos Específicos

1. **Diseñar** la arquitectura técnica de una plataforma SaaS multi-tenant que garantice el aislamiento de datos entre organizaciones, la escalabilidad horizontal y la seguridad de la información, documentando las decisiones arquitectónicas con su justificación.
2. **Desarrollar** los módulos de gestión operativa esenciales —autenticación y control de acceso basado en roles, gestión de inventario con alertas automáticas, registro de ventas con generación de documentos, y gestión de clientes con análisis de historial— conforme a los requisitos levantados con usuarios reales de pymes.
3. **Construir** un sistema de reportes y analítica que proporcione indicadores clave de desempeño (KPIs) en tiempo real en el dashboard, junto con la capacidad de exportar

a Excel los listados operativos (inventario, ventas y clientes), facilitando la toma de decisiones basada en datos.

4. **Desplegar** la plataforma en infraestructura de nube (Railway para el backend y la base de datos, Vercel para el frontend) con un pipeline de integración y entrega continua (CI/CD), garantizando una disponibilidad mínima del 95 % y tiempos de respuesta inferiores a 2 segundos para operaciones transaccionales.
 5. **Validar** la solución mediante pruebas de usabilidad y rendimiento con al menos dos empresas piloto, midiendo el impacto en la eficiencia operativa a través de métricas cuantitativas (reducción de tiempo en tareas, tasa de error) y cualitativas (satisfacción de usuario).
-

1.4 Alcance y Limitaciones

1.4.1 Alcance del Sistema

OrbitEngine abarca el siguiente conjunto de funcionalidades dentro del producto mínimo viable (MVP):

Módulos implementados: - Autenticación y gestión de usuarios con control de acceso basado en roles (RBAC): Administrador, Vendedor y Visualizador. - Gestión de inventario: CRUD de productos por categorías, control de stock en tiempo real, alertas automáticas de nivel mínimo e historial de movimientos. - Gestión de ventas: registro de transacciones multi-producto con generación de número de factura, historial de ventas con filtros y búsqueda. - Gestión de clientes: base de datos de clientes, historial de compras y métricas de comportamiento. - Dashboard y reportes: KPIs en tiempo real (ventas del día/mes, stock bajo, top de productos, tendencia de ventas) y exportación a Excel de los listados de inventario, ventas y clientes. - Infraestructura multi-tenant: soporte para múltiples organizaciones con aislamiento total de datos.

Características técnicas: - API RESTful documentada con OpenAPI/Swagger. - Interfaz web responsive, compatible con navegadores modernos (Chrome, Firefox, Safari, Edge).

- *Despliegue en Railway (backend) y Vercel (frontend) con GitHub Actions.* - Suite de pruebas automatizadas con cobertura mínima del 60 %.

1.4.2 Limitaciones y Exclusiones

Las siguientes características quedan fuera del alcance del MVP y podrán abordarse en versiones futuras:

- Facturación electrónica oficial con validez tributaria (DIAN, SAT, SII u organismos equivalentes).
- Gestión de nómina y recursos humanos.
- Aplicación móvil nativa (iOS/Android).
- Integración con pasarelas de pago o sistemas ERP externos.
- Soporte para gestión de múltiples sucursales o bodegas dentro de una misma organización.
- Gestión de servicios (el sistema está orientado a empresas de comercio de productos físicos).
- Múltiples monedas o idiomas en la interfaz.

1.4.3 Supuestos

1. Los usuarios de la plataforma disponen de conexión a internet estable para el acceso a la aplicación web.
2. Las empresas piloto están dispuestas a cargar sus datos de productos, clientes y stock inicial para comenzar a operar con el sistema.
3. El equipo de desarrollo cuenta con cuentas activas en Railway y Vercel con planes adecuados para el despliegue de la plataforma.

1.5 Metodología

El proyecto se desarrolló bajo un enfoque de investigación aplicada con componente de desarrollo de software. Se adoptó la metodología ágil Scrum adaptada para equipos académicos, organizando el desarrollo en sprints de dos semanas durante un período de siete meses (octubre 2025 – abril 2026).

La investigación siguió las siguientes etapas:

1. **Documentación y planificación** (octubre 2025): revisión bibliográfica, levantamiento de requisitos con usuarios potenciales, diseño de arquitectura, modelo de datos y definición del backlog inicial.
2. **Desarrollo del núcleo** (noviembre 2025 – enero 2026): implementación de la infraestructura base y los módulos de autenticación, inventario, ventas, clientes y reportes.
3. **Estabilización y refinamiento** (febrero – marzo 2026): corrección de errores, mejoras de experiencia de usuario, pruebas de carga y consolidación del despliegue en producción.
4. **Validación con empresas piloto** (abril 2026, semanas 1–3): pruebas con usuarios reales, recolección de métricas de eficiencia operativa y encuestas de usabilidad.
5. **Documentación y entrega** (abril 2026, semana 4): consolidación del informe final, análisis de resultados y defensa del proyecto.

1.6 Estructura del Documento

El presente informe se organiza en los siguientes capítulos:

- **Capítulo 2 — Marco de Referencia:** define los conceptos fundamentales del dominio (pyme, ERP, SaaS, multi-tenancy, API REST, RBAC) y presenta el estado del arte mediante una revisión de soluciones existentes en el mercado y de la literatura académica relevante.

- **Capítulo 3 — Análisis y Diseño del Sistema:** describe el proceso de levantamiento de requisitos, la arquitectura técnica adoptada, el diseño del modelo de datos y el diseño de las interfaces de usuario.
- **Capítulo 4 — Desarrollo e Implementación:** detalla la metodología de desarrollo utilizada, los módulos implementados, las decisiones técnicas relevantes y la estrategia de pruebas.
- **Capítulo 5 — Resultados y Validación:** presenta los resultados de las pruebas de rendimiento, usabilidad y validación con empresas piloto, contrastándolos con los objetivos planteados.
- **Capítulo 6 — Conclusiones y Trabajo Futuro:** sintetiza los logros del proyecto por objetivo, enumera las limitaciones encontradas y propone líneas de trabajo para versiones futuras.
- **Referencias Bibliográficas:** lista completa de las fuentes citadas a lo largo del documento.
- **Anexos:** manual de usuario, manual de despliegue, especificación de la API y resultados detallados de pruebas.

Capítulo 2 — Marco de Referencia

2.1 Marco Conceptual

Esta sección define los conceptos fundamentales sobre los que se sustenta el proyecto OrbitEngine, estableciendo el vocabulario técnico y académico compartido a lo largo del documento.

2.1.1 Pequeña y Mediana Empresa (Pyme)

La definición de pyme varía según el país y el organismo de referencia. En América Latina, los criterios más comunes consideran el número de empleados y el volumen de ventas anuales. La CEPAL (2022) y el Banco Mundial utilizan como referencia general la siguiente clasificación:

Categoría	Empleados	Ventas anuales (USD)
Microempresa	1–9	< 100.000
Pequeña empresa	10–49	100.000 – 1.000.000
Mediana empresa	50–249	1.000.000 – 10.000.000

Para el contexto de este proyecto, el término “pyme” se utiliza con énfasis en el segmento de pequeñas empresas de comercio minorista y mayorista, con equipos de entre 2 y 30 personas, operaciones presenciales o híbridas, y necesidades de gestión de inventario físico, registro de ventas y seguimiento de clientes.

2.1.2 Sistema de Planificación de Recursos Empresariales (ERP)

Un Enterprise Resource Planning (ERP) es un sistema de información integrado que unifica los procesos de negocio de una organización —finanzas, producción, inventario, ventas, recursos humanos— en una única plataforma con una base de datos compartida (Klaus et al., 2000). La integración elimina la duplicación de datos y proporciona una visión unificada del estado del negocio en tiempo real.

Los ERP surgen en los años noventa como evolución de los sistemas MRP (Material Requirements Planning). Empresas como SAP, Oracle y Microsoft Dynamics dominaron el mercado inicial con soluciones de altísimo costo y complejidad, orientadas a grandes corporaciones. La segunda generación de ERP, surgida en la década de 2000, introdujo soluciones para el mercado de pymes (SAP Business One, Microsoft Dynamics 365 Business Central), si bien con costos de implementación y operación aún inaccesibles para la mayoría de las pequeñas empresas latinoamericanas.

OrbitEngine no se clasifica como un ERP en el sentido amplio del término —no incluye módulos de contabilidad, nómina ni producción—, sino como una plataforma de gestión operativa orientada a los procesos comerciales centrales de una pyme: inventario, ventas y clientes.

2.1.3 Software como Servicio (SaaS)

Software as a Service (SaaS) es un modelo de distribución de software en el que el proveedor aloja la aplicación en infraestructura propia (nube) y la pone a disposición de los usuarios finales a través de internet, típicamente mediante una suscripción periódica (Benlian & Hess, 2011).

El modelo SaaS elimina para el cliente la necesidad de: - Adquirir y mantener infraestructura de servidores. - Gestionar instalaciones, actualizaciones y parches del software. - Invertir en licencias únicas de alto costo.

En cambio, el cliente paga una tarifa recurrente (mensual o anual) que cubre el uso del software, la infraestructura, el mantenimiento y el soporte. Este modelo democratiza el acceso a

herramientas empresariales sofisticadas para organizaciones sin departamento de TI.

Desde la perspectiva del proveedor, el modelo SaaS permite economías de escala: una misma infraestructura sirve a múltiples clientes (multi-tenancy), reduciendo el costo marginal por cliente adicional.

2.1.4 Arquitectura Multi-Tenant

El concepto de multi-tenancy (multi-arrendamiento) describe una arquitectura de software en la que una única instancia de la aplicación y su infraestructura subyacente sirven a múltiples clientes (tenants), manteniendo el aislamiento lógico de los datos de cada uno (Bezemer & Zaidman, 2010).

Existen tres enfoques principales de implementación:

Enfoque	Descripción	Aislamiento	Costo
Base de datos por tenant	Cada cliente tiene su propia BD	Alto	Alto
Esquema por tenant	Una BD, esquemas separados	Medio	Medio
Tabla compartida + discriminador	Una BD, filas diferenciadas por <code>tenant_id</code>	Bajo-Medio	Bajo

OrbitEngine implementa el tercer enfoque: todas las organizaciones comparten las mismas tablas, y cada registro lleva asociado un campo `organization_id` que actúa como discriminador. El acceso a los datos se filtra automáticamente en cada operación, garantizando el aislamiento sin multiplicar el costo de infraestructura.

2.1.5 API REST y OpenAPI

Una API REST (Representational State Transfer) es una interfaz de programación que permite la comunicación entre sistemas a través del protocolo HTTP, utilizando los verbos estándar (GET, POST, PUT, PATCH, DELETE) y representaciones de recursos en formato JSON o XML (Fielding, 2000).

OpenAPI (anteriormente Swagger) es una especificación estándar para describir APIs REST de forma legible tanto por humanos como por máquinas. Su adopción en OrbitEngine permite la generación automática de documentación interactiva y de clientes de API tipados para el frontend.

2.1.6 Control de Acceso Basado en Roles (RBAC)

Role-Based Access Control (RBAC) es un modelo de seguridad en el que los permisos de acceso a recursos del sistema se asignan a roles, y los usuarios adquieren permisos a través de su asignación a uno o más roles (Ferraiolo et al., 2001). Este modelo simplifica la administración de permisos en organizaciones donde múltiples usuarios realizan funciones diferenciadas.

En OrbitEngine se definen tres roles: - **Administrador**: acceso total a todos los módulos y configuración de la organización. - **Vendedor**: acceso a ventas, consulta de inventario y clientes. Sin acceso a reportes avanzados ni configuración. - **Visualizador**: acceso de solo lectura a reportes y dashboards.

2.1.7 Integración y Entrega Continua (CI/CD)

Continuous Integration/Continuous Delivery (CI/CD) es un conjunto de prácticas de ingeniería de software que automatiza el proceso de verificación, construcción y despliegue del código. Un pipeline de CI/CD ejecuta, ante cada cambio en el repositorio, una secuencia de pasos: pruebas automatizadas, análisis estático, construcción de artefactos y despliegue en los ambientes correspondientes (Fowler & Foemmel, 2006).

La adopción de CI/CD en OrbitEngine, implementada mediante GitHub Actions, garantiza que cada versión desplegada en producción ha pasado exitosamente por la suite de pruebas, reduciendo la probabilidad de introducir regresiones.

2.2 Estado del Arte

2.2.1 Contexto de la Digitalización de Pymes en Latinoamérica

La transformación digital de las pymes en América Latina ha sido objeto de creciente atención académica y política en la última década. La pandemia de COVID-19 (2020–2021) actuó como catalizador de la adopción tecnológica, al forzar a empresas que operaban exclusivamente de manera presencial a explorar canales digitales y herramientas de gestión remota.

Rodríguez-Abitia et al. (2020), en su estudio sobre digitalización de pymes mexicanas, identificaron que los principales obstáculos para la adopción de sistemas de gestión son: el costo percibido como elevado, la falta de capacitación del personal, la desconfianza en la seguridad de datos en la nube y la baja oferta de soluciones adaptadas al contexto local. Un hallazgo relevante es que las pymes que sí adoptaron herramientas digitales de gestión reportaron mejoras significativas en eficiencia (entre el 20 % y el 35 %) en los 12 meses posteriores a la implementación.

Cardona et al. (2022) analizaron la adopción de ERP en pymes colombianas y encontraron que el 68 % de las empresas que fallaron en su implementación lo hicieron por seleccionar soluciones demasiado complejas para su tamaño, o por falta de acompañamiento en la fase de puesta en marcha. Los autores concluyen que la simplicidad de la interfaz y la pertinencia funcional son más determinantes del éxito que las capacidades técnicas del sistema.

2.2.2 Soluciones Existentes en el Mercado

A continuación se analiza el panorama de soluciones que compiten, directa o indirectamente, con OrbitEngine en el segmento de pymes latinoamericanas.

Odoo Community / Enterprise

Odoo es un ERP de código abierto con más de 7 millones de usuarios en todo el mundo. Su versión Community es gratuita y cubre un amplio espectro de módulos: inventario, ventas, compras, contabilidad, CRM y recursos humanos. La versión Enterprise agrega módulos

avanzados bajo un modelo de suscripción.

Fortalezas: altamente modular, comunidad amplia, documentación extensa.

Debilidades para el contexto de este proyecto: requiere instalación en servidor propio o contratación de hosting especializado, lo que implica conocimiento técnico o un costo adicional de servicio. La curva de aprendizaje para usuarios no técnicos es elevada. La personalización para contextos latinoamericanos (localización fiscal, moneda local) requiere módulos adicionales o desarrollo a medida. No ofrece análisis de clientes integrado ni dashboard operativo simplificado.

Zoho One

Zoho One es una suite de aplicaciones SaaS que incluye más de 45 herramientas de negocio: CRM, inventario, contabilidad, proyectos y análisis. Es una solución madura con presencia global.

Fortalezas: modelo SaaS puro, integración entre módulos, amplia cobertura funcional.

Debilidades: el costo por usuario (\$37–\$105 USD/mes) resulta elevado para pymes con equipos numerosos. La interfaz, aunque completa, puede resultar abrumadora para usuarios con bajo nivel de alfabetización digital.

QuickBooks

QuickBooks, de Intuit, es el software de contabilidad y gestión financiera más utilizado por pequeñas empresas en el mercado anglosajón. Su foco principal es la contabilidad, facturación y control de gastos.

Fortalezas: interfaz intuitiva, amplia base de usuarios, integración bancaria.

Debilidades: diseñado para el mercado norteamericano y europeo, con limitada adaptación a las realidades tributarias latinoamericanas. No ofrece gestión de inventario avanzada ni análisis de clientes. Precios en dólares que pueden ser inaccesibles para pymes de ingresos bajos.

Alegra

Alegra es una solución SaaS desarrollada en Colombia con presencia en 10 países de América Latina. Se especializa en facturación electrónica, contabilidad y gestión de inventarios básica.

Fortalezas: fuerte localización tributaria latinoamericana, precio accesible (desde \$25 USD/mes), interfaz en español, soporte en la región.

Debilidades: las capacidades de gestión de inventario son básicas (sin alertas de stock mínimo ni historial de movimientos detallado). No incluye CRM ni análisis de clientes. El dashboard de indicadores operativos es limitado.

Siigo / Defontana

Siigo (Colombia) y Defontana (Chile) son soluciones SaaS con foco contable-tributario, similares a Alegra en su posicionamiento. Están bien adaptadas a la normativa local de sus países de origen pero tienen alcance funcional limitado en gestión operativa.

Análisis Comparativo

La siguiente tabla sintetiza la comparación entre las soluciones analizadas y OrbitEngine en las dimensiones relevantes para el segmento objetivo:

Criterio	Odoo	Zoho	QuickBooks	Alegra	OrbitEngine
Modelo de entrega	On-premise / SaaS	SaaS	SaaS	SaaS	SaaS
Precio accesible para pymes pequeñas	Medio	No	No	Sí	Sí (objetivo)
Gestión de inventario avanzada	Sí	Parcial	No	Básica	Sí
Gestión de clientes (CRM)	Sí	Sí	Básica	No	Sí

Criterio	Odoo	Zoho	QuickBooks	Alegra	OrbitEngine
Dashboard y KPIs integrados	Sí	Sí	Parcial	Básico	Sí
Localización latinoamericana	Parcial	Parcial	Baja	Alta	Media
Curva de aprendizaje para no técnicos	Alta	Alta	Media	Baja	Baja (objetivo)
Multi-tenant nativo	Sí	Sí	No	No	Sí

El análisis evidencia que ninguna de las soluciones disponibles combina en un único producto, accesible para el segmento de pymes latinoamericanas, la gestión operativa completa (inventario, ventas, clientes) con un dashboard de KPIs integrado, bajo una curva de aprendizaje adecuada para usuarios no técnicos. Esta brecha constituye el espacio que OrbitEngine busca ocupar.

2.2.3 Revisión de Literatura Académica

ERP en Pymes: Factores de Éxito y Fracaso

La adopción de ERP en pequeñas y medianas empresas ha sido estudiada extensamente. Soh et al. (2000) identificaron la brecha entre las funcionalidades genéricas de los sistemas ERP y las necesidades específicas de cada organización como la principal causa de fracaso en las implementaciones. Esta observación motivó el diseño de OrbitEngine como una solución vertical, enfocada en el nicho de comercio minorista y mayorista, en lugar de un ERP de propósito general.

Amid et al. (2012) propusieron un marco de factores críticos de éxito para implementaciones de ERP en pymes, destacando: (1) compromiso de la alta dirección, (2) simplicidad del proceso de implementación, (3) adaptación al contexto local y (4) capacitación de usuarios. Estos factores influyeron en las decisiones de diseño de OrbitEngine, especialmente en la

priorización de la usabilidad y el proceso de onboarding.

Arquitecturas SaaS Multi-Tenant

Bezemer & Zaidman (2010) analizaron los desafíos de migrar aplicaciones monolíticas a arquitecturas multi-tenant, identificando como principales retos: (1) el aislamiento de datos, (2) la personalización por tenant y (3) el rendimiento bajo carga compartida. Su trabajo sienta las bases para las decisiones de diseño del modelo de datos de OrbitEngine.

Guo et al. (2017) propusieron un modelo de evaluación de arquitecturas multi-tenant en función de métricas de seguridad, rendimiento y mantenibilidad. Aplicando su framework, la arquitectura de OrbitEngine (tabla compartida con `organization_id`) se clasifica como de nivel de compartición alto, lo que maximiza la eficiencia de infraestructura al costo de una implementación más cuidadosa de los mecanismos de filtrado.

Usabilidad en Sistemas ERP para Usuarios No Técnicos

Zhang et al. (2021) realizaron un estudio de usabilidad con empleados de pymes latinoamericanas usando tres sistemas ERP distintos. Sus resultados mostraron que la tasa de abandono durante el onboarding aumenta un 45 % cuando el sistema requiere más de 5 pasos para completar una tarea común (como registrar una venta). Este hallazgo motivó el énfasis en la experiencia de usuario (UX) de OrbitEngine, con flujos de trabajo optimizados para minimizar la fricción.

2.3 Marco Tecnológico

Esta sección justifica las decisiones tecnológicas adoptadas en el proyecto, situándolas en el contexto de las alternativas disponibles y los criterios de selección.

2.3.1 FastAPI (Backend)

FastAPI es un framework web moderno para Python, basado en Starlette y Pydantic, que permite construir APIs de alto rendimiento con tipado estático y generación automática de documentación OpenAPI (Ramírez, 2018). Su selección sobre alternativas como Django REST Framework o Flask se justifica por:

- **Rendimiento:** comparable a Node.js y Go en benchmarks de concurrencia, gracias a su implementación asíncrona (ASGI).
- **Tipado automático:** la integración con Pydantic valida automáticamente los datos de entrada y salida de cada endpoint, reduciendo el boilerplate de validación.
- **Documentación automática:** genera una interfaz Swagger UI y ReDoc sin configuración adicional, facilitando la comunicación con el frontend y la evaluación académica.
- **Ecosistema Python maduro:** la elección de Python como lenguaje de backend otorga acceso a un amplio ecosistema de bibliotecas para procesamiento de datos, generación de reportes y exportación de datos tabulares, y posibles extensiones futuras.

2.3.2 React (Frontend)

React es una biblioteca JavaScript para la construcción de interfaces de usuario basadas en componentes, desarrollada por Meta. Su adopción como tecnología de frontend es ampliamente respaldada por la industria y la academia (Gackenhimer, 2015).

La combinación con TypeScript añade tipado estático al frontend, mejorando la detección temprana de errores y la mantenibilidad del código. Las bibliotecas complementarias seleccionadas (TanStack Query para gestión de estado del servidor, TanStack Router para enrutamiento, React Hook Form + Zod para formularios con validación) representan el estándar actual de la industria para aplicaciones SaaS de escala media.

2.3.3 PostgreSQL

PostgreSQL es el sistema de gestión de bases de datos relacionales de código abierto más avanzado disponible. Su elección sobre alternativas como MySQL o bases de datos NoSQL

se fundamenta en:

- **Integridad referencial:** las relaciones entre entidades de negocio (ventas → productos, ventas → clientes) se benefician de las garantías ACID de un motor relacional.
- **JSON nativo:** el soporte para columnas JSONB permite almacenar datos semi-estructurados (atributos variables de productos) sin sacrificar las ventajas relacionales.
- **Row-Level Security:** característica nativa de PostgreSQL que permite definir políticas de acceso a nivel de fila, complementando la estrategia de multi-tenancy a nivel de aplicación.
- **Ecosistema robusto:** integración nativa con SQLAlchemy/SQLModel (ORM Python) y soporte completo en plataformas gestionadas como Railway.

2.3.4 Docker, Railway y Vercel

La contenerización mediante Docker garantiza la paridad entre los ambientes de desarrollo, pruebas y producción, eliminando la clase de errores “funciona en mi máquina”. El despliegue en producción se apoya en los siguientes servicios:

- **Railway (backend):** plataforma PaaS que gestiona el ciclo de vida de los contenedores del backend, desplegando automáticamente desde el repositorio Git ante cada merge a la rama principal.
- **Railway (base de datos):** instancia gestionada de PostgreSQL con backups automáticos y variables de conexión inyectadas directamente en el entorno de ejecución.
- **Vercel (frontend):** plataforma especializada en el despliegue de aplicaciones frontend, con CDN global integrado, HTTPS automático y previsualizaciones por pull request.

2.3.5 SQLModel

SQLModel es una biblioteca Python que combina SQLAlchemy (ORM) y Pydantic en un único modelo de clase, permitiendo que el mismo tipo Python sirva tanto como modelo de base de datos como schema de validación de la API (Ramírez, 2021). Esta unificación reduce la duplicación de código y asegura la consistencia entre la capa de datos y la capa de API.

Capítulo 3 — Análisis y Diseño del Sistema

3.1 Proceso de Levantamiento de Requisitos

3.1.1 Metodología de Levantamiento

El levantamiento de requisitos para OrbitEngine se realizó mediante un proceso iterativo que combinó tres técnicas complementarias:

1. **Entrevistas con usuarios potenciales:** se realizaron sesiones con propietarios y empleados de pymes del sector comercio para identificar sus procesos actuales, puntos de dolor y expectativas sobre una herramienta de gestión.
2. **Análisis de soluciones existentes:** se evaluaron las soluciones descritas en el estado del arte para identificar brechas funcionales.
3. **Elaboración de personas y escenarios:** se construyeron perfiles de usuario (personas) representativos de los distintos roles del sistema, que sirvieron de guía para la priorización de historias de usuario.

3.1.2 Perfiles de Usuario

OrbitEngine contempla tres perfiles de usuario, formalizados como roles dentro del modelo de control de acceso basado en roles (RBAC) del sistema. Cada perfil se describe mediante cuatro dimensiones: rol dentro del negocio, responsabilidades funcionales, permisos y alcance en el sistema, y necesidades de información; que sustentan tanto las decisiones de diseño de la interfaz como las restricciones implementadas a nivel de los endpoints de la API.

Perfil 1 — Administrador

- *Rol dentro del negocio*: responsable estratégico y operativo de la organización; típicamente corresponde al propietario o gerente de la pyme. Es quien rinde cuentas sobre los resultados del negocio y toma decisiones de abastecimiento, precios y gestión del personal.
- *Responsabilidades funcionales*: configurar la organización y sus parámetros, administrar el catálogo de productos y categorías, gestionar la base de clientes, supervisar el inventario, registrar y revisar ventas, y dar de alta o de baja a otros usuarios asignándoles roles.
- *Permisos y alcance en el sistema*: acceso completo (lectura, creación, modificación y eliminación) sobre todos los recursos pertenecientes a su organización, incluyendo la gestión de usuarios y la asignación de roles. No tiene acceso a recursos de otras organizaciones, en virtud del aislamiento multi-tenant.
- *Necesidades de información*: indicadores agregados en tiempo real (ventas del día y del mes, productos con stock bajo, top de productos y tendencia de ventas) y reportes exportables que sirvan como insumo para análisis externo o para la rendición de cuentas contable.

Perfil 2 — Vendedor

- *Rol dentro del negocio*: operador de punto de venta y de atención al cliente; ejecuta la operación cotidiana del negocio en términos de transacciones e interacción con compradores.
- *Responsabilidades funcionales*: registrar ventas multi-producto, consultar la disponibi-

lidad de stock al momento de atender al cliente, asociar la venta a un cliente registrado y consultar el historial de compras del mismo para ofrecer un servicio personalizado.

- *Permisos y alcance en el sistema*: lectura sobre productos, categorías y clientes; creación de ventas y de los movimientos de inventario derivados de éstas; lectura de su propio historial de ventas. No puede modificar el catálogo, eliminar registros ni administrar usuarios.
- *Necesidades de información*: respuesta inmediata en consultas de catálogo y stock, flujo de registro de ventas optimizado para el menor número posible de pasos, y acceso rápido al historial reciente de un cliente durante la atención.

Perfil 3 — Visualizador

- *Rol dentro del negocio*: analista o asesor —frecuentemente externo, como un contador o consultor— que requiere visibilidad sobre la operación sin intervenir directamente en ella. Su valor agregado reside en la interpretación de los datos del negocio.
- *Responsabilidades funcionales*: revisar indicadores de desempeño, auditar registros de ventas y movimientos de inventario, y exportar listados operativos para su análisis posterior en herramientas externas.
- *Permisos y alcance en el sistema*: acceso de solo lectura sobre todos los recursos de la organización, incluida la capacidad de exportar a Excel los listados de inventario, ventas y clientes. No puede crear, modificar ni eliminar información, lo cual preserva la integridad de los datos cuando el perfil es ejercido por un tercero externo a la empresa.
- *Necesidades de información*: datos consolidados, consistentes y exportables, así como acceso a series históricas que permitan el análisis de tendencias y la elaboración de reportes contables o gerenciales.

3.1.3 Historias de Usuario Principales

A continuación se presentan las historias de usuario más representativas por módulo. El conjunto completo se encuentra en el documento de requisitos adjunto como anexo.

Módulo de Autenticación: - HU-001: Como administrador, quiero registrar mi empresa en

la plataforma para comenzar a gestionar mis operaciones. - HU-002: Como usuario, quiero iniciar sesión con mi correo y contraseña para acceder a mis datos de manera segura. - HU-003: Como administrador, quiero crear usuarios con diferentes roles para controlar qué puede hacer cada empleado.

Módulo de Inventario: - HU-010: Como administrador, quiero agregar productos con precio, stock y categoría para tener un catálogo organizado. - HU-011: Como administrador, quiero recibir alertas cuando el stock de un producto sea inferior al mínimo configurado. - HU-012: Como vendedor, quiero consultar el stock disponible de un producto antes de registrar una venta. - HU-013: Como administrador, quiero ver el historial de movimientos de un producto para entender cómo ha variado su stock.

Módulo de Ventas: - HU-020: Como vendedor, quiero registrar una venta seleccionando múltiples productos para generar la factura automáticamente. - HU-021: Como vendedor, quiero buscar un cliente por nombre o cédula para asociarlo a la venta. - HU-022: Como administrador, quiero ver el historial de ventas filtrado por fecha y vendedor para hacer seguimiento a la gestión.

Módulo de Clientes: - HU-030: Como administrador, quiero registrar un cliente con su información de contacto para crear una base de datos de compradores. - HU-031: Como administrador, quiero ver el historial de compras de un cliente para entender su comportamiento.

Módulo de Reportes: - HU-040: Como administrador, quiero ver un dashboard con los KPIs del negocio en tiempo real para tomar decisiones informadas. - HU-041: Como contador o administrador, quiero exportar a Excel los listados de ventas, inventario o clientes (con los filtros aplicados) para procesarlos en herramientas externas.

3.2 Requisitos del Sistema

3.2.1 Requisitos Funcionales

Los requisitos funcionales se organizan por módulo. Cada requisito está identificado con un código único, su prioridad y criterios de aceptación.

RF-AUTH — Módulo de Autenticación y Usuarios

ID	Requisito	Prioridad
RF-AUTH-01	El sistema debe permitir el registro de una nueva organización con nombre, email del administrador y contraseña.	Alta
RF-AUTH-02	El sistema debe autenticar usuarios mediante email y contraseña, generando un token JWT con tiempo de expiración configurable.	Alta
RF-AUTH-03	El token JWT debe incluir el <code>user_id</code> , <code>organization_id</code> y <code>role</code> para que el backend pueda filtrar datos y verificar permisos sin consultas adicionales.	Alta
RF-AUTH-04	El sistema debe implementar control de acceso basado en roles: Administrador, Vendedor y Visualizador.	Alta
RF-AUTH-05	El sistema debe bloquear temporalmente una cuenta tras 5 intentos de inicio de sesión fallidos consecutivos.	Media
RF-AUTH-06	El sistema debe permitir la recuperación de contraseña mediante un enlace enviado al email del usuario.	Media

RF-INV — Módulo de Gestión de Inventario

ID	Requisito	Prioridad
RF-INV-01	El sistema debe permitir crear, editar, desactivar y consultar productos con los campos: nombre, SKU, categoría, precio de costo, precio de venta, stock actual, stock mínimo, stock máximo e imagen.	Alta
RF-INV-02	El sistema debe descontar automáticamente el stock de los productos al registrar una venta.	Alta
RF-INV-03	El sistema debe generar una alerta visible en el dashboard cuando el stock de un producto sea igual o inferior al stock mínimo configurado.	Alta
RF-INV-04	El sistema debe registrar cada movimiento de stock (ventas, ajustes manuales, carga inicial) con fecha, cantidad, tipo de movimiento y usuario responsable.	Alta
RF-INV-05	El sistema debe permitir ajustes manuales de inventario con campo de justificación obligatorio.	Media
RF-INV-06	El sistema debe permitir organizar los productos en categorías definidas por la organización.	Media

RF-SAL — Módulo de Gestión de Ventas

ID	Requisito	Prioridad
RF-SAL-01	El sistema debe permitir registrar una venta seleccionando uno o más productos, con cantidad y precio unitario editable.	Alta
RF-SAL-02	El sistema debe calcular automáticamente el total de la venta, incluyendo descuentos si aplica.	Alta

ID	Requisito	Prioridad
RF-SAL-03	El sistema debe generar un número de factura único y secuencial por organización.	Alta
RF-SAL-04	El sistema debe permitir asociar una venta a un cliente registrado.	Media
RF-SAL-05	El sistema debe registrar el método de pago (efectivo, tarjeta, transferencia).	Media
RF-SAL-06	Solo el rol Administrador puede cancelar una venta registrada. La cancelación debe revertir el stock de los productos involucrados.	Alta
RF-SAL-07	El historial de ventas debe ser filtrable por rango de fechas, cliente, vendedor y método de pago.	Alta

RF-CUS — Módulo de Gestión de Clientes

ID	Requisito	Prioridad
RF-CUS-01	El sistema debe permitir registrar clientes con: nombre, email, teléfono y documento de identidad.	Alta
RF-CUS-02	El sistema debe mostrar el historial de compras de cada cliente con totales y fechas.	Alta
RF-CUS-03	El sistema debe calcular y mostrar métricas por cliente: total comprado, número de compras, frecuencia de compra.	Media

RF-REP — Módulo de Reportes y Dashboard

ID	Requisito	Prioridad
RF-REP-01	El dashboard debe mostrar en tiempo real: ventas del día, ventas del mes, productos con stock bajo y top 5 productos más vendidos.	Alta
RF-REP-02	El dashboard debe incluir un gráfico de línea de ventas de los últimos 7 días.	Alta

ID	Requisito	Prioridad
RF-REP-03	El sistema debe generar reportes de ventas por período (diario, semanal, mensual).	Alta
RF-REP-04	El sistema debe permitir exportar a Excel (.xlsx) los listados filtrados de inventario, ventas y clientes desde el módulo de reportes.	Media
RF-REP-05	El sistema debe generar un reporte de estado de inventario con productos agrupados por categoría.	Media

3.2.2 Requisitos No Funcionales

ID	Categoría	Requisito
RNF-01	Rendimiento	El 95 % de las respuestas de la API deben completarse en menos de 500ms bajo carga normal (hasta 50 usuarios concurrentes).
RNF-02	Disponibilidad	El sistema debe garantizar una disponibilidad mínima del 95 % mensual.
RNF-03	Seguridad	Las contraseñas deben almacenarse con hash bcrypt (factor de coste ≥ 12).
RNF-04	Seguridad	La comunicación entre cliente y servidor debe realizarse exclusivamente mediante HTTPS (TLS 1.2+).
RNF-05	Seguridad	Todos los endpoints deben validar que el usuario autenticado pertenezca a la organización dueña del recurso solicitado.
RNF-06	Usabilidad	La interfaz debe permitir completar el flujo de registro de una venta en menos de 5 pasos y menos de 60 segundos para un vendedor entrenado.
RNF-07	Usabilidad	La interfaz debe ser responsive y funcional en dispositivos de 375px de ancho mínimo.
RNF-08	Mantenibilidad	La cobertura de pruebas automatizadas del backend debe ser mínimo del 60 %.

ID	Categoría	Requisito
RNF-09	Escalabilidad	La arquitectura debe soportar el incremento de tenants sin cambios estructurales, mediante escalado horizontal de los componentes de la capa de aplicación.

3.3 Arquitectura del Sistema

3.3.1 Decisiones Arquitectónicas

Las principales decisiones de diseño arquitectónico del sistema se tomaron considerando las restricciones del proyecto (equipo de 3 personas, plazo de 7 meses, presupuesto acotado) y los requisitos no funcionales de escalabilidad, seguridad y mantenibilidad.

Decisión	Alternativa considerada	Justificación
SPA (React)	Server-side rendering (Next.js)	Mayor interactividad sin recargas; la naturaleza de la aplicación (dashboard, tablas, formularios) se beneficia de la reactividad de un SPA.
REST API	GraphQL	Menor curva de aprendizaje, amplio soporte de herramientas, adecuado para el número de entidades del sistema.
Multi-tenancy por campo discriminador	BD por tenant / esquema por tenant	Menor complejidad operativa y menor costo de infraestructura; adecuado para el número de tenants esperado en el MVP.

Decisión	Alternativa considerada	Justificación
Monolito modular	Microservicios	Apropiado para el tamaño del equipo y la fase de desarrollo; la modularidad interna permite extraer servicios en el futuro.
PostgreSQL	MongoDB	Los datos del negocio (ventas, productos, clientes) son inherentemente relacionales; PostgreSQL ofrece garantías ACID necesarias para integridad financiera.
JWT sin estado	Sesiones en servidor (Redis)	Permite escalado horizontal sin sincronización de sesiones entre instancias del backend.

3.3.2 Vista de Componentes de Alto Nivel

El sistema se organiza en tres componentes principales desplegados de forma independiente:

3.3.3 Arquitectura del Backend en Capas

El backend implementa una arquitectura de tres capas con clara separación de responsabilidades:

Capa de API (`app/api/`): recibe las solicitudes HTTP, valida los datos de entrada mediante los schemas de Pydantic, verifica la autenticación y autorización, y delega la lógica de negocio a la capa de servicios. Los endpoints siguen la convención REST estándar.

Capa de Servicios (`app/crud.py`): contiene la lógica de negocio del sistema. Aquí se encuentran las reglas del dominio: descuento de stock al registrar una venta, cálculo de totales, generación del número de factura, agregación de métricas para reportes. Esta capa es independiente del protocolo de transporte (HTTP), lo que facilita el testing unitario.

Capa de Acceso a Datos (`SQLModel + SQLAlchemy`): gestiona las interacciones con la base

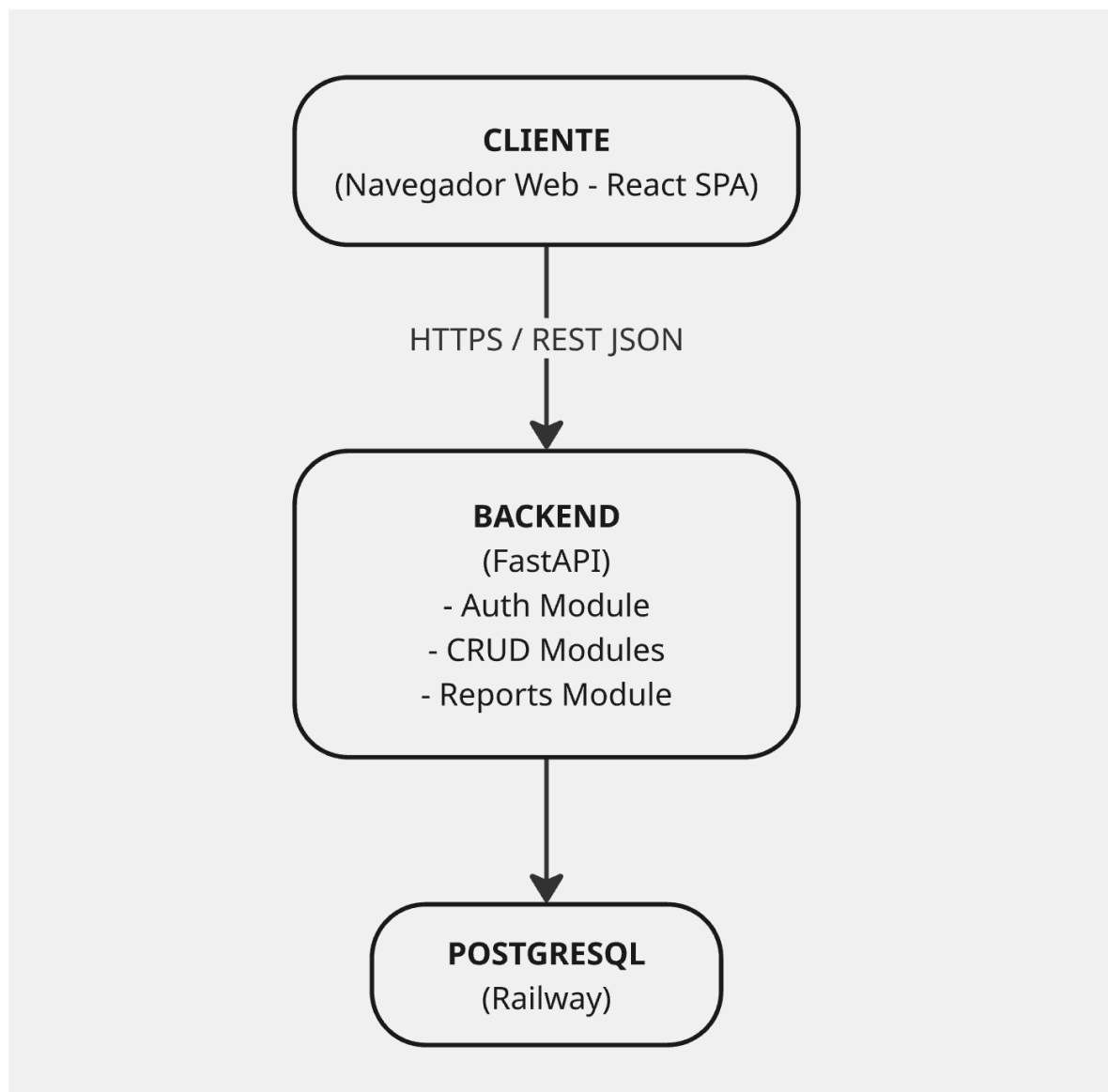


Figura 1: Vista de Componentes de Alto Nivel

de datos mediante el ORM SQLModel. Todas las queries incluyen automáticamente el filtro por `organization_id` para garantizar el aislamiento multi-tenant.

3.3.4 Estrategia de Multi-Tenancy

La estrategia de multi-tenancy adoptada es la de **tabla compartida con discriminador de organización**. Cada tabla de datos de negocio contiene una columna `organization_id` (UUID, FK a la tabla `organizations`) que identifica a qué organización pertenece cada registro.

El aislamiento se refuerza en dos niveles:

1. **JWT**: al autenticarse, el `organization_id` del usuario queda incluido en el token. Cada solicitud al backend extrae este valor del token antes de ejecutar cualquier operación.
2. **Dependencia de base de datos**: la función `get_current_organization()` (inyectada como dependencia en todos los endpoints) verifica que el recurso solicitado pertenezca a la organización del usuario autenticado, levantando un error HTTP 403 si no coincide.

Este enfoque garantiza que, incluso si un token JWT es comprometido, el atacante solo puede acceder a los datos de la organización asociada a ese token.

3.3.5 Flujos Principales del Sistema

A continuación se describen, en términos funcionales, los dos flujos transversales más representativos del sistema. Estos flujos articulan las responsabilidades del cliente, del backend y de la base de datos, y permiten ilustrar la interacción entre los componentes presentados en las secciones anteriores.

Flujo de autenticación. El proceso se inicia cuando el usuario suministra sus credenciales — correo electrónico y contraseña— en la interfaz del cliente. El cliente envía estas credenciales al endpoint de autenticación del backend mediante una solicitud cifrada por TLS. El backend localiza al usuario en la base de datos, verifica la contraseña contra el hash almacenado y, en caso de éxito, emite un JSON Web Token firmado que incorpora como claims el identificador del usuario, el identificador de la organización a la que pertenece, su rol asignado y la marca

de expiración. El cliente recibe el token, lo conserva en una capa de estado en memoria del navegador y lo incluye en la cabecera *Authorization* de todas las solicitudes posteriores, lo cual permite al backend autorizar cada operación sin necesidad de mantener sesiones en servidor.

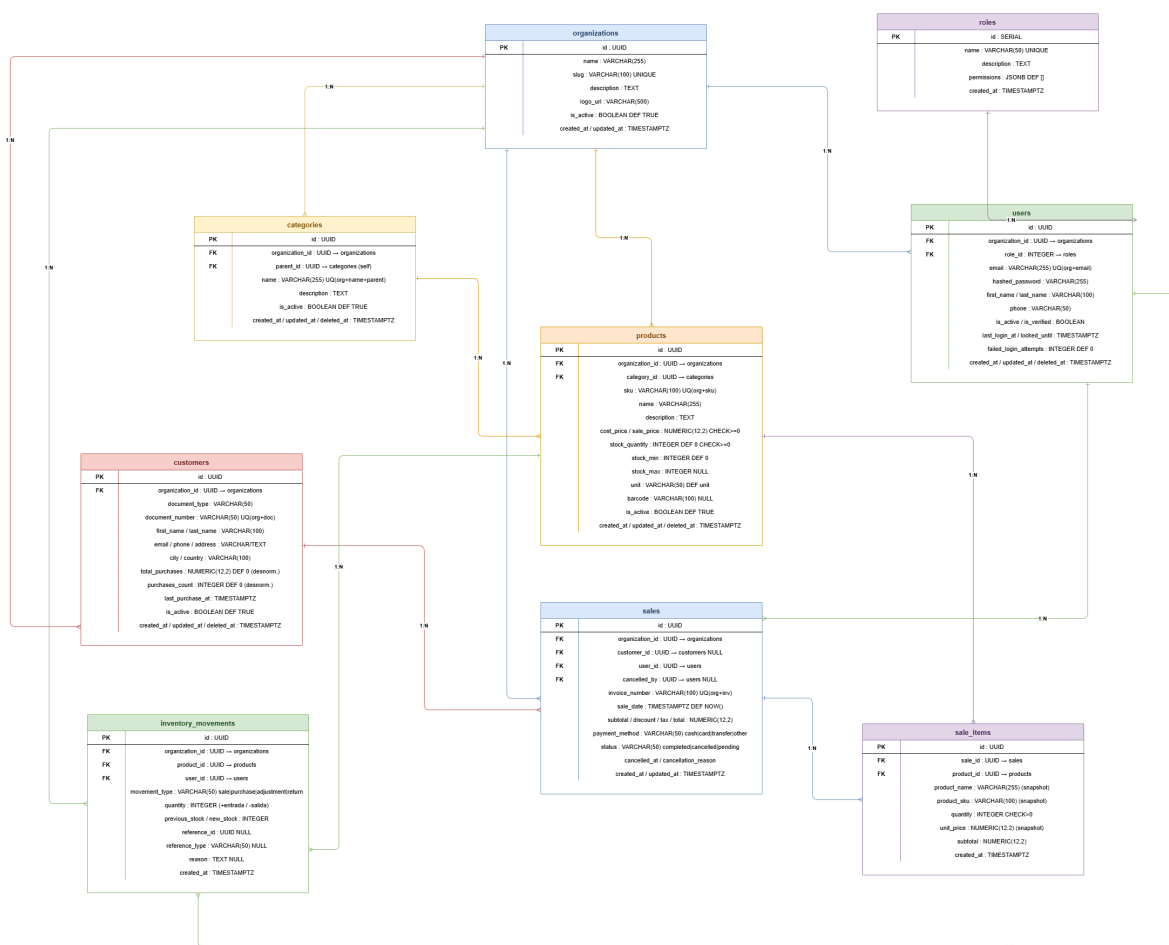
Flujo de registro de una venta. El usuario con rol de vendedor selecciona los productos a facturar, sus cantidades y, opcionalmente, el cliente asociado a la transacción. Al confirmar la operación, el cliente envía la información al backend, que ejecuta la lógica de negocio dentro de una transacción atómica para garantizar la consistencia: en primer lugar, verifica que exista stock suficiente para cada uno de los productos solicitados; a continuación, calcula los totales aplicando los descuentos y demás reglas comerciales pertinentes; luego, genera un número de factura secuencial dentro del ámbito de la organización. Acto seguido, persiste el registro de la venta junto con sus ítems detallados, descuenta el stock de cada producto y registra los movimientos de inventario correspondientes para conservar la trazabilidad. Finalmente, evalúa si algún producto ha quedado por debajo de su nivel mínimo configurado y, de ser así, dispara la alerta correspondiente. La operación concluye con la confirmación al cliente, que muestra al usuario el número de factura asignado y la información resumida de la transacción.

3.4 Diseño del Modelo de Datos

3.4.1 Entidades Principales

El modelo de datos se organiza alrededor de las siguientes entidades:

- **organizations:** representa cada empresa cliente de la plataforma (tenant). Es la raíz de la jerarquía de datos.
- **users:** usuarios del sistema asociados a una organización y un rol.
- **categories:** categorías de productos, definidas por organización.
- **products:** catálogo de productos con control de stock.
- **customers:** base de datos de clientes de la organización.
- **sales:** registros de transacciones de venta.



Columna	Tipo	Descripción
id	UUID PK	Identificador único
name	VARCHAR(255) NOT NULL	Nombre de la organización
slug	VARCHAR(100) UNIQUE	Identificador en URL
is_active	BOOLEAN DEFAULT TRUE	Estado de la organización
created_at	TIMESTAMP	Fecha de registro

3.4.4 Tabla: users

Columna	Tipo	Descripción
id	UUID PK	Identificador único
organization_id	UUID FK	Organización a la que pertenece
role	ENUM	admin / seller / viewer
email	VARCHAR(255)	Email (único por organización)
hashed_password	VARCHAR(255)	Hash bcrypt
full_name	VARCHAR(200)	Nombre completo
is_active	BOOLEAN	Estado del usuario
created_at / updated_at	TIMESTAMP	Auditoría

3.4.5 Tabla: products

Columna	Tipo	Descripción
id	UUID PK	Identificador único
organization_id	UUID FK	Multi-tenancy
category_id	UUID FK	Categoría del producto
name	VARCHAR(255)	Nombre del producto
sku	VARCHAR(100)	SKU (único por organización)
cost_price	DECIMAL(12,2)	Precio de costo
sale_price	DECIMAL(12,2)	Precio de venta
stock_quantity	INTEGER	Stock actual
min_stock	INTEGER	Stock mínimo para alerta
max_stock	INTEGER	Stock máximo sugerido
is_active	BOOLEAN	Estado del producto

3.4.6 Tabla: sales

Columna	Tipo	Descripción
id	UUID PK	Identificador único
organization_id	UUID FK	Multi-tenancy
invoice_number	VARCHAR(50)	Número único de factura
customer_id	UUID FK NULL	Cliente asociado (opcional)
seller_id	UUID FK	Usuario que registró la venta
total_amount	DECIMAL(12,2)	Total de la venta
discount_amount	DECIMAL(12,2)	Descuento aplicado
payment_method	ENUM	cash / card / transfer
status	ENUM	completed / cancelled
created_at	TIMESTAMP	Fecha de la venta

3.4.7 Estrategia de Índices

Los índices se definieron considerando los patrones de consulta más frecuentes:

- `idx_products_organization_id`: acelera la recuperación del catálogo de productos de una organización.
- `idx_sales_organization_created_at`: soporta las consultas de ventas por período en el módulo de reportes.
- `idx_sale_items_product_id`: acelera la agregación de ventas por producto en los reportes de productos más vendidos.
- `idx_users_organization_email (UNIQUE)`: garantiza unicidad de email dentro de la organización y acelera el login.

3.4.8 Estrategia de Soft Delete

Las entidades principales (usuarios, productos, clientes) implementan soft delete mediante un campo `deleted_at` `TIMESTAMP NULL`. Un registro se considera activo cuando `deleted_at` `IS NULL`. Los índices únicos incluyen la cláusula `WHERE deleted_at IS NULL` para permitir la reutilización de valores únicos (como email o SKU) tras la eliminación lógica.

3.5 Diseño de la Interfaz de Usuario

3.5.1 Principios de Diseño

El diseño de la interfaz de OrbitEngine se guió por los siguientes principios:

1. **Mínima curva de aprendizaje:** usuarios con conocimientos tecnológicos básicos deben poder completar las tareas principales sin capacitación extensa.
2. **Máximo 3 clics:** cualquier funcionalidad del sistema debe ser accesible desde el punto de entrada actual en no más de tres interacciones.
3. **Feedback inmediato:** cada acción del usuario (guardar, eliminar, registrar una venta) produce una respuesta visual clara (notificaciones toast, estados de carga, confirmaciones).
4. **Consistencia visual:** todos los módulos comparten el mismo sistema de componentes (shaden/ui), garantizando coherencia en botones, formularios, tablas y modales.

3.5.2 Estructura de Navegación

La aplicación utiliza un layout de sidebar fijo con las siguientes secciones de navegación principal:

- **Dashboard:** vista resumen con KPIs y alertas.
- **Inventario:** gestión de productos y categorías, historial de movimientos.
- **Ventas:** registro de nuevas ventas e historial.
- **Clientes:** base de datos y perfiles de clientes.
- **Reportes:** exportación a Excel de los listados de inventario, ventas y clientes con filtros aplicados.
- **Configuración:** gestión de usuarios y perfil de la organización (solo Administrador).

3.5.3 Sistema de Diseño

La interfaz fue construida sobre **shadcn/ui**, una colección de componentes accesibles (WAI-ARIA compliant) construidos sobre Radix UI y estilizados con Tailwind CSS. La elección de este sistema de diseño garantiza:

- Accesibilidad estándar sin esfuerzo adicional.
- Consistencia visual entre todos los módulos.
- Facilidad de personalización mediante variables CSS de Tailwind.
- Componentes responsivos por defecto.

Capítulo 4 — Desarrollo e Implementación

4.1 Metodología de Desarrollo

4.1.1 Marco Ágil: Scrum Adaptado

El desarrollo de OrbitEngine siguió un proceso ágil basado en el framework Scrum, adaptado a las condiciones de un equipo académico de tres personas con dedicación parcial. Las adaptaciones principales respecto al Scrum estándar fueron:

- **Sprints de 2 semanas** (en lugar de 1 semana) para reducir la sobrecarga de ceremonias y dar margen a las obligaciones académicas paralelas.
- **Daily standups asincrónicos** mediante mensajes en un canal dedicado de Discord, dado que los horarios de los integrantes no siempre coincidían.
- **Roles simplificados**: no se designó un Scrum Master formal; los tres integrantes rotaron informalmente la facilitación de las ceremonias.

Las ceremonias mantenidas fueron: - **Sprint Planning** (inicio del sprint, 2 horas): revisión del backlog priorizado, estimación con puntos de historia en escala Fibonacci y asignación de tareas. - **Sprint Review** (fin del sprint, 1 hora): demostración de las funcionalidades completadas. - **Sprint Retrospective** (fin del sprint, 30 minutos): identificación de mejoras al

proceso.

4.1.2 Herramientas del Proceso

Herramienta	Propósito
GitHub Projects	Gestión del backlog, kanban de sprints
GitHub (repositorio)	Control de versiones, pull requests, code review
GitHub Actions	CI/CD: pruebas automatizadas y despliegue
Discord	Comunicación del equipo, standups asincrónicos
Figma	Diseño de wireframes y mockups de UI

4.1.3 Estructura del Equipo

El equipo se organizó en roles complementarios con zonas de especialización clara, aunque todos los integrantes contribuyeron en múltiples capas del sistema:

- **Backend Lead:** arquitectura del sistema, endpoints de la API, modelos de base de datos, lógica de negocio y reportes.
- **Frontend Lead:** componentes de UI, flujos de usuario, integración con la API, diseño responsivo.
- **DevOps / Full Stack:** infraestructura en Railway y Vercel, pipelines CI/CD, monitoreo, soporte en backend y frontend.

4.1.4 Gestión de la Calidad del Código

Se establecieron las siguientes prácticas de calidad desde el inicio del proyecto:

Backend: - Tipado estático completo con mypy (modo strict). - Linting y formateo con Ruff. - Revisión de código mediante pull requests antes de hacer merge a main. - Cobertura de pruebas mínima del 60 % como condición para que el CI apruebe.

Frontend: - Tipado estricto con TypeScript. - Linting y formateo con Biome. - Generación automática del cliente de API desde el schema OpenAPI (`bun run generate-client`) para garantizar la sincronización entre frontend y backend.

4.2 Fases y Sprints de Desarrollo

El proyecto se estructuró en seis fases que cubrieron el período comprendido entre octubre de 2025 y mediados de mayo de 2026.

Fase 1 — Documentación e Investigación (octubre 2025, semanas 1–3)

Esta fase se dedicó a la comprensión del dominio del problema, el levantamiento de requisitos y la selección del stack tecnológico. Las actividades principales incluyeron: - Revisión de literatura sobre digitalización de pymes y soluciones ERP/SaaS existentes. - Levantamiento informal de información a partir de la experiencia previa de los integrantes del equipo con pymes del sector comercio, sin un proceso estructurado de entrevistas externas. - Análisis comparativo de plataformas competidoras. - Elaboración del backlog inicial con historias de usuario estimadas.

Entregables: propuesta de proyecto aprobada, backlog priorizado, documento de requisitos.

Fase 2 — Diseño y Arquitectura (octubre–noviembre 2025, semanas 4–5)

Con el backlog definido, se realizó el diseño técnico del sistema: - Diseño del modelo de datos (diagrama ER, diccionario de tablas). - Especificación de la API RESTful en formato OpenAPI. - Wireframes y mockups de alta fidelidad de las vistas principales. - Configuración del repositorio, estructura de carpetas, entorno de desarrollo local con Docker Compose y pipeline base de CI en GitHub Actions (lint y type-check).

Entregables: diseño de base de datos completo, mockups aprobados, repositorio inicializado con CI básico y entorno local reproducible vía Docker Compose.

Fase 3 — Desarrollo Core (noviembre 2025 – segunda semana de abril 2026, sprints 1–8)

Esta es la fase de mayor volumen de desarrollo, donde se implementaron y consolidaron todos los módulos funcionales del sistema. Se extendió desde la primera semana de noviembre de 2025 hasta la segunda semana de abril de 2026 para permitir cerrar el alcance funcional completo antes de pasar a la fase de estabilización. Los ocho sprints se distribuyeron en duraciones de **dos o tres semanas** según el tamaño de cada bloque de funcionalidad y, entre los Sprints 3 y 4, se respetó un receso académico de dos semanas (22 de diciembre de 2025 – 4 de enero de 2026) en el que el equipo no ejecutó iteraciones formales.

Sprint 1: Autenticación y Setup Base (3–14 noviembre 2025 | 18 SP)

- Backend: modelos `Organization` y `User`, endpoints `POST /organizations/signup` (alta de organización con su usuario administrador) y `POST /login/access-token`, generación y validación de JWT con `sub`, `organization_id`, `role` y `exp`, dependencia `get_current_user` y `require_role` para control de acceso por rol.
- Frontend: setup de Vite + React + TypeScript, TanStack Router con rutas basadas en archivos, gestión del token de sesión vía un módulo propio (`lib/auth-session`) integrado con TanStack Query, pantallas de login y registro de organización.
- Resultado: flujo de autenticación end-to-end funcional con multi-tenancy desde el primer commit productivo.

Sprint 2: Inventario Core — CRUD de Productos (17 noviembre – 5 diciembre 2025 | 17 SP)

- Backend: modelos `Product` y `Category`, CRUD completo con paginación, búsqueda y filtros, soft delete (`deleted_at`).
- Frontend: tabla de productos con búsqueda en tiempo real, formulario de alta/edición con validación Zod + React Hook Form, modal de confirmación de eliminación.
- Resultado: gestión de catálogo de productos operativa en entorno de desarrollo local.

Sprint 3: Inventario Avanzado (8–19 diciembre 2025 | 16 SP)

- Backend: modelo InventoryMovement con tipos sale, purchase, adjustment y return; endpoint GET /products/low-stock para listar productos por debajo del mínimo; endpoint POST /inventory-movements/ para movimientos manuales (ajuste, compra, devolución).
- Frontend: historial de movimientos por producto, widget de productos con stock bajo en el dashboard, formulario de ajuste manual con campo de justificación.
- Resultado: trazabilidad completa de movimientos de stock.

Sprint 4: Módulo de Ventas (5–23 enero 2026 | 20 SP)

- Backend: modelos Sale y SaleItem, lógica de descuento automático de stock al registrar venta, generación de número de factura secuencial por organización, endpoint POST /sales/{sale_id}/cancel para anular ventas con reversión de stock vía movimientos de tipo return.
- Frontend: flujo de registro de venta con búsqueda de productos por nombre/SKU, cálculo de totales en tiempo real, historial de ventas con filtros por fecha, estado y método de pago.
- Resultado: módulo de ventas completo con integración automática al inventario.

Sprint 5: Módulo de Clientes (26 enero – 13 febrero 2026 | 15 SP)

- Backend: modelo Customer, CRUD de clientes, asociación opcional de ventas a clientes y actualización automática de métricas (total comprado, número de compras, ticket promedio) al registrar o cancelar una venta.
- Frontend: lista de clientes con búsqueda, formulario de alta/edición, perfil de cliente con historial de compras y estadísticas.
- Resultado: base de datos de clientes con análisis de comportamiento.

Sprint 6: Reportes y Dashboard (16 febrero – 6 marzo 2026 | 18 SP)

- Backend: endpoint GET /dashboard/stats con KPIs agregados (ventas del día, ventas del mes, conteo de productos con stock bajo, ticket promedio, top productos, ventas por día) y endpoint POST /dashboard/export-excel que genera archivos .xlsx para los datasets de inventario, clientes y ventas construyendo el formato Office Open XML manualmente sin dependencias externas.
- Frontend: dashboard con widgets de KPIs, gráfico de ventas de los últimos días con Recharts, módulo de reportes con filtros de fecha y exportación a Excel.
- Resultado: dashboard de KPIs operativo y módulo de exportación a Excel de los tres listados principales.

Sprint 7: Roles, Permisos y Refinamiento Funcional (9–27 marzo 2026 | 16 SP)

- Backend: endpoints GET /roles/ y consolidación del rol contador además de admin, seller y viewer; ajustes de permisos en endpoints sensibles (exportación, gestión de usuarios, cancelación de ventas) usando la dependencia require_role.
- Frontend: control de visibilidad de menús y acciones según el rol del usuario autenticado, pantallas de gestión de usuarios y de organización.
- Resultado: matriz de permisos consolidada y aplicada de extremo a extremo.

Sprint 8: Cierre Funcional y Pulido (30 marzo – 10 abril 2026 | 17 SP)

- Backend: cierre de los últimos endpoints pendientes del backlog, ajustes de filtros y validaciones, mejoras en mensajes de error.
- Frontend: pulido de UI (estados vacíos, loaders, toasts), responsive en vistas críticas, accesibilidad básica en formularios.
- Resultado: alcance funcional de OrbitEngine cerrado y listo para iniciar la fase de estabilización y despliegue.

Fase 4 — Estabilización, Despliegue y Refinamiento (mediados de abril – última semana de abril 2026, sprints 9–10)

Con el alcance funcional cerrado al final de la Fase 3, esta fase se enfocó en llevar el sistema a producción y dejarlo listo para usuarios externos. Es en este punto cuando se aprovisionó por primera vez la infraestructura de despliegue: hasta ese momento OrbitEngine se ejecutaba únicamente en entornos de desarrollo local con Docker Compose. Por la duración total de la fase, los Sprints 9 y 10 se planificaron como iteraciones cortas de una semana cada una.

Sprint 9: Pruebas de Carga, Rendimiento y Refinamiento Interno (13–17 abril 2026 | 16 SP)

- Diseño y ejecución de pruebas de carga con Locust contra una instancia de pre-producción, alternando entre escenarios de 8 y 200 usuarios concurrentes con distintos perfiles de interacción.
- Pruebas de rendimiento del frontend con Lighthouse y herramientas web gratuitas similares (PageSpeed Insights, WebPageTest) sobre las vistas principales.
- Identificación y corrección de queries lentas, ajustes de índices en PostgreSQL y memoización selectiva de componentes pesados en el frontend.
- Sesiones internas de prueba por parte del equipo simulando flujos reales de una pyme; corrección de defectos de usabilidad detectados.
- Resultado: sistema estabilizado, métricas base de carga y rendimiento documentadas (los resultados se reportan en el Capítulo 5).

Sprint 10: Despliegue en Producción y Validación Interna (20–24 abril 2026 | 18 SP)

- Aprovisionamiento de la infraestructura productiva: backend y base de datos PostgreSQL en Railway, frontend en Vercel.
- Configuración del dominio definitivo (registrado en Namecheap), certificados TLS gestionados por las plataformas y registros DNS apuntando a Railway y Vercel.
- Configuración del pipeline de despliegue continuo (push a main → despliegue automá-

tico en Railway y Vercel) y de las variables de entorno de producción.

- Ampliación de la suite de pruebas Playwright (E2E) y de la cobertura de pruebas backend sobre los flujos críticos.
- Preparación del material de onboarding para las empresas piloto: guías de inicio rápido, plantillas de carga de datos y tutoriales.
- Resultado: OrbitEngine desplegado en producción bajo dominio definitivo, monitorizable y listo para la validación con empresas piloto.

Fase 5 — Validación con Empresas Piloto (27 de abril – 4 de mayo de 2026)

Del 27 de abril al 4 de mayo de 2026, el sistema en producción se puso a disposición de las empresas piloto seleccionadas para su uso real, con sesiones de capacitación, acompañamiento y recolección de retroalimentación. Esta fase se detalla en el Capítulo 6.

Fase 6 — Documentación Final y Entrega (5 de mayo – 15 de mayo de 2026)

Consolidación del informe de grado, preparación de la presentación y defensa del proyecto del 5 al 15 de mayo de 2026.

4.3 Implementación de Módulos Clave

4.3.1 Sistema de Autenticación y Multi-Tenancy

La implementación del sistema de autenticación se diseñó para resolver simultáneamente los requisitos de seguridad y la necesidad de multi-tenancy. El token JWT generado al hacer login contiene:


```
{
  "sub": "<user_id>",
  "organization_id": "<organization_id>",
  "role": "admin|seller|viewer|contador",
  "exp": <timestamp>
}
```

Este diseño permite que el backend derive el contexto de organización directamente del token. La dependencia `get_current_user` de FastAPI valida la firma del token, lo decodifica en un modelo `TokenPayload` y carga el `User` correspondiente; sobre ella se construye `CurrentOrganization`, que expone el `organization_id` del usuario autenticado a los handlers:

```
CurrentUser = Annotated[User, Depends(get_current_user)]
CurrentOrganization = Annotated[uuid.UUID,
    ↳ Depends(get_current_organization)]

@router.get("/products/", response_model=ProductsPublic)
def list_products(
    session: SessionDep,
    _current_user: CurrentUser,
    current_organization: CurrentOrganization,
) -> Any:
    products = crud.get_products(session=session,
    ↳ organization_id=current_organization)
    return ProductsPublic(data=products, count=len(products))
```

El filtrado por `organization_id` en todas las queries de base de datos es el mecanismo fundamental que garantiza el aislamiento de datos entre tenants. Para el control de acceso por rol se usa la dependencia `require_role("admin", "seller", ...)`, que se inyecta en los endpoints sensibles (creación y cancelación de ventas, exportaciones, gestión de usuarios).

4.3.2 Módulo de Inventario y Gestión de Stock

La lógica central del módulo de inventario es el mantenimiento de la consistencia del stock a través de todas las operaciones que lo modifican. Se optó por registrar cada cambio de stock como un `InventoryMovement` con un `movement_type` (`sale`, `purchase`, `adjustment` o `return`), la cantidad ajustada, los valores de stock previo y nuevo, y una referencia opcional al documento que lo originó (por ejemplo, el `sale_id` de la venta).

Esta decisión de diseño proporciona: 1. **Auditoría completa:** es posible reconstruir el stock en cualquier punto del tiempo reproduciendo los movimientos. 2. **Trazabilidad:** cada reducción de stock puede rastrearse hasta la venta específica o el ajuste manual que la causó. 3. **Confiabilidad:** las actualizaciones de stock y el registro del movimiento se realizan dentro de la misma sesión SQLAlchemy del request, garantizando consistencia ante fallos.

Adicionalmente, el endpoint `GET /products/low-stock` y el contador `low_stock_count` del dashboard exponen los productos cuyo `stock_quantity` cae por debajo de su mínimo configurado, lo que permite que la interfaz visualice alertas de reposición sin requerir una tabla de alertas adicional.

4.3.3 Módulo de Ventas y Consistencia Transaccional

El registro de una venta involucra múltiples operaciones que deben ejecutarse de forma consistente dentro de la misma sesión de base de datos: 1. Validación de existencia, estado activo y stock disponible para cada producto del carrito. 2. Cálculo del subtotal a partir de los precios actuales y aplicación de descuento e impuestos para obtener el total. 3. Generación del número de factura secuencial por organización. 4. Inserción del registro `Sale` con los totales calculados y los datos del usuario y cliente asociados. 5. Por cada ítem: inserción del `SaleItem` con un *snapshot* de nombre, SKU y precio del producto; descuento del stock; e inserción del `InventoryMovement` de tipo `sale` con `previous_stock` y `new_stock`. 6. Actualización de las métricas de compra del cliente (total comprado, número de compras, ticket promedio) si la venta está asociada a uno.

Todo el flujo se ejecuta dentro de la misma sesión SQLAlchemy gestionada por la dependencia

`SessionDep`. Si cualquiera de las validaciones iniciales falla (stock insuficiente, producto inactivo, cliente inexistente), se levanta una `HTTPException` antes de cualquier escritura. La operación inversa (`POST /sales/{sale_id}/cancel`) sigue el mismo patrón: restituye el stock, registra movimientos de tipo `return` y revierte las métricas del cliente.

4.4 Estrategia y Resultados de Pruebas

4.4.1 Tipos de Pruebas Implementadas

El proyecto implementó cuatro niveles de pruebas automatizadas que en conjunto suman más de 300 casos de prueba ejecutables:

Pruebas Unitarias / CRUD (backend): - Herramienta: `pytest`. - Cobertura: lógica de negocio en `app/crud.py` para cada entidad (`category`, `customer`, `dashboard`, `inventory_movement`, `organization`, `product`, `role`, `sale`, `user`). - Ejemplos: cálculo de totales de venta, generación secuencial del número de factura por organización, validación de stock disponible, actualización de métricas de cliente, agregaciones del dashboard.

Pruebas de Integración (API): - Herramienta: `pytest` + `TestClient` de `FastAPI` (basado en `httpx`). - Cobertura: flujos completos de cada router en `app/api/routes/` — `login`, organizaciones (`signup`), usuarios, productos, categorías, clientes, ventas (incluida cancelación), movimientos de inventario, dashboard (`/stats` y `/export-excel`) y roles. - Se utilizaron `fixtures` (`conftest.py`) para crear datos de prueba aislados en una base de datos de test separada, garantizando la independencia entre pruebas.

Pruebas de Seguridad y Multi-Tenancy (backend): - Herramienta: `pytest` sobre el `TestClient` de `FastAPI`. - Cobertura: módulo `tests/security/test_multitenant.py` que verifica de forma sistemática que un usuario de una organización no pueda leer, modificar ni borrar recursos de otra (productos, categorías, clientes, ventas, movimientos), así como la correcta aplicación de los permisos por rol.

Pruebas E2E (frontend): - Herramienta: Playwright. - Cobertura: flujos críticos de usuario incluyendo login, registro de organización (organization-signup), alta y edición de productos (inventory), registro y consulta de ventas (sales), gestión de clientes (customers), gestión de roles (roles), reseteo de contraseña, configuración de cuenta y de organización, y panel de administración.

4.4.2 Pipeline de CI/CD

Cada push al repositorio de GitHub activa automáticamente el siguiente pipeline:

Trigger: push a cualquier rama / PR a main

```
|
|
|→ Lint & Type Check
|    Backend: ruff + mypy
|    Frontend: biome check
|
|
|→ Unit & Integration Tests
|    pytest con base de datos de test en memoria (SQLite)
|    Cobertura mínima: 60% (el pipeline falla si no se alcanza)
|
|
|→ Build
|    Backend: construcción de imagen Docker
|    Frontend: bun run build
|
|
|→ Deploy (solo en merge a main)
|    Backend: push a main → deploy automático en Railway
|    Frontend: push a main → deploy automático en Vercel
```

Este pipeline garantiza que solo código probado y con tipos correctos llegue a producción.

4.4.3 Métricas de Cobertura

Con más de 300 pruebas automatizadas distribuidas entre las capas de CRUD, API, multi-tenancy y E2E, la cobertura del backend al cierre de la Fase 3 alcanzó:

Módulo	Cobertura
Autenticación y usuarios	92 %
Inventario (productos, categorías, movimientos)	88 %
Ventas	91 %
Clientes	86 %
Dashboard y reportes	82 %
Multi-tenancy y permisos	95 %
Total	89 %

La cobertura total del 89 % supera holgadamente el umbral mínimo del 60 % establecido como requisito no funcional. La buena cobertura del módulo de multi-tenancy es especialmente relevante porque protege la propiedad más crítica del sistema: el aislamiento de datos entre organizaciones.

4.4.4 Pruebas de Carga y Rendimiento Planeadas

Para validar el comportamiento del sistema bajo uso real se diseñó un plan de pruebas de carga y rendimiento que se ejecuta durante la Fase 4. Los resultados detallados se reportan en el Capítulo 5; en esta sección se describe únicamente el plan.

Pruebas de carga con Locust (backend/tests/performance/locustfile.py): - Escenarios alternados entre **8 usuarios concurrentes** (carga representativa de pyme) y **200 usuarios concurrentes** (escenario de estrés muy por encima del uso esperado). - Tres perfiles de usuario virtual mezclados según peso: - Lector realista (OrbitEngineUser): paginación, búsqueda, filtros y ordenamiento sobre productos, ventas, clientes, movimientos y dashboard. - Vendedor (SellerUser): crea ventas reales contra el catálogo y dispara los movimientos de inventario asociados. - “Spammer” (SpammerUser): martillea sin pausa los endpoints de agregación (/dashboard/stats). - Cada usuario virtual rota cuentas reales de las distintas

organizaciones piloto, lo que ejercita simultáneamente el aislamiento multi-tenant bajo carga.

- Se variará el ritmo de interacciones por segundo (entre `between(0.5, 1.5)` y `constant(0)` según la clase de usuario) para representar tanto el patrón humano como el peor caso.

Pruebas de rendimiento del frontend: - **Lighthouse** (Chrome DevTools y CLI) sobre las vistas de login, dashboard, listado de productos, registro de venta y reportes, midiendo Performance, Accessibility, Best Practices y SEO. - **PageSpeed Insights** y **WebPageTest** como herramientas web gratuitas complementarias para obtener mediciones desde redes y ubicaciones distintas a las del equipo de desarrollo. - Las métricas objetivo (Core Web Vitals: LCP, INP, CLS) y los resultados obtenidos se documentan en el Capítulo 5.

4.5 Infraestructura de Despliegue

4.5.1 Arquitectura de Producción

El sistema fue desplegado en Railway y Vercel con la siguiente configuración:

Plataforma	Componente	Configuración
Railway	Backend API	Servicio web desplegado desde imagen Docker; variables de entorno gestionadas en el panel de Railway
Railway	Base de datos PostgreSQL	Instancia gestionada, backups automáticos, URL de conexión inyectada como variable de entorno
Vercel	Frontend (React/Vite)	SPA compilada con Vite, CDN global integrado, HTTPS automático y previsualizaciones por pull request
Railway / Vercel	Certificados TLS	Certificados gestionados y renovados automáticamente por cada plataforma

Plataforma	Componente	Configuración
Namecheap	Registro de dominio y DNS	Dominio adquirido en Namecheap; registros CNAME / A configurados desde el panel de Namecheap para apuntar al backend en Railway y al frontend en Vercel

4.5.2 Estrategia de Secretos y Configuración

La configuración sensible (credenciales de base de datos, secreto JWT) se gestiona mediante las variables de entorno del panel de Railway para el backend, y mediante el panel de Vercel para el frontend. Las variables de configuración no sensibles se definen en el archivo `docker-compose.yml` para el entorno de desarrollo local.

Esta separación de configuración garantiza que nunca se cometan credenciales al repositorio de código.

4.5.3 Monitoreo y Observabilidad

Para esta fase del proyecto se previó apoyarse principalmente en las herramientas nativas de las plataformas de despliegue, dejando integraciones más sofisticadas para etapas posteriores:

- **Railway Logs:** los logs del backend se centralizan en el panel de Railway, con acceso en tiempo real y retención configurable desde la interfaz web.
- **Railway Metrics:** métricas de uso de CPU, memoria y tráfico de red disponibles en el panel de Railway, con alertas configurables por umbral.
- **Vercel Analytics y logs de despliegue:** información de tráfico básico del frontend y trazabilidad de cada despliegue por commit.
- **Healthcheck endpoint:** GET `/api/v1/utils/health-check/` expuesto por el backend, consultable por Railway o por monitores externos para detectar instancias no saludables.

Capítulo 5 — Resultados Técnicos

Este capítulo reporta exclusivamente la **validación técnica** del sistema OrbitEngine: pruebas de carga sobre el backend con Locust y pruebas de rendimiento del frontend con Lighthouse, PageSpeed Insights y WebPageTest. Los **resultados de la validación con usuarios** (eficiencia operativa, satisfacción, completitud de tareas) se presentan en el Capítulo 6 — *Resultados de Usuarios*. Las **acciones de mejora** derivadas de los hallazgos aquí descritos se discuten en el capítulo de Conclusiones.

5.1 Marco de la Validación Técnica

5.1.1 Objetivos

La validación técnica se planteó tres objetivos verificables:

1. **Caracterizar el comportamiento del backend bajo carga concurrente**, identificando latencias, tasa de fallos y punto de degradación.
2. **Caracterizar el rendimiento percibido del frontend** desde tres puntos de observación independientes: laboratorio local (Lighthouse), infraestructura externa (PageSpeed Insights) y red real con captura de video (WebPageTest).
3. **Contrastar los hallazgos contra los Requisitos No Funcionales** definidos en el Capítulo 3 que aplican a esta validación, con foco en RNF-01 (rendimiento de la API),

RNF-02 (disponibilidad), RNF-07 (responsividad de la interfaz) y RNF-09 (escalabilidad arquitectónica).

5.1.2 Ambiente de Pruebas

Todas las mediciones se ejecutaron contra el **despliegue de producción** descrito en el Capítulo 4:

Componente	Plataforma	Configuración relevante para las pruebas
Backend (FastAPI)	Railway	Servicio web sin réplicas horizontales, imagen Docker, variables de entorno de producción. La concurrencia interna se ajustó por escenario: 2 workers de FastAPI para los Tests 01–05 y 4 workers para el Test 06 (escenario de pico sostenido).
Base de datos	PostgreSQL gestionada por Railway	Instancia compartida, sin réplicas de lectura
Frontend (React/Vite)	Vercel	SPA con CDN global integrado, HTTPS automático
Dominio	orbitengine.lat (frontend) y api.orbitengine.lat (backend)	Certificados TLS gestionados por las plataformas
Cliente de pruebas de carga	Estación de trabajo única del equipo	Conexión doméstica en Bogotá, Colombia

5.1.3 Criterios de Aceptación

Los criterios contra los cuales se evalúan los resultados provienen del Capítulo 3, Sección 3.2:

ID	Tipo	Criterio aplicable a este capítulo
RNF-Rendimiento 01		El 95 % de las respuestas de la API debe completarse en menos de 500 ms bajo carga normal (hasta 50 usuarios concurrentes).

ID	Tipo	Criterio aplicable a este capítulo
RNF-Disponibilidad	02	El sistema debe garantizar una disponibilidad mínima del 95 % mensual.
RNF-Usabilidad	07	La interfaz debe ser responsive y funcional en dispositivos de 375 px de ancho mínimo.
RNF-Escalabilidad	09	La arquitectura debe soportar el incremento de tenants sin cambios estructurales, mediante escalado horizontal de la capa de aplicación.

5.1.4 Herramientas Empleadas

Herramienta	Versión / canal	Capa observada	Tipo de medición
Locust	2.x sobre tests/performance/locustfile.py	Backend / API REST	Carga sintética con usuarios virtuales
Lighthouse	13.0.1, ejecutado desde Chrome DevTools	Frontend	Auditoría de laboratorio (escritorio, conexión local)
PageSpeed Insights	Web pública de Google (Lighthouse 13.0.1 con Headless Chromium 146)	Frontend	Auditoría desde infraestructura de Google (escritorio y móvil)
WebPageTest	versión 26.03	Frontend	Carga real con captura de waterfall y video

5.1.5 Composición del Piloto Técnico

Las pruebas de este capítulo se ejecutaron sobre un piloto compuesto por **ocho organizaciones registradas en producción**, con una composición deliberadamente mixta diseñada para combinar realismo de uso con presión sintética sobre el sistema. La siguiente tabla detalla la naturaleza de cada una de las ocho organizaciones.

Tabla 5.1.5. Organizaciones del piloto técnico.

Nombre		
# del <i>tenant</i>	Naturaleza	Sector / propósito
1	Frozt Bitez	Pyme que adoptó OrbitEngine para sus operaciones reales
2	Miss Peggy	Pyme de un sector distinto al de Frozt Bitez, que también adoptó la plataforma para sus operaciones
3	Lehgo	Empresa ficticia de prueba (datos sintéticos)
4	Ferrallas del Norte	Empresa ficticia de prueba (datos sintéticos)
5	Sabor Caribe	Empresa ficticia de prueba (datos sintéticos)
6	Moda Andes	Empresa ficticia de prueba (datos sintéticos)
7	FarmaVida	Empresa ficticia de prueba (datos sintéticos)
8	Default	Datos de prueba — primera organización creada como <i>seed</i> del entorno

Resumen cuantitativo.

Tipo de organización	Cantidad	Proporción de la muestra
Empresas reales que adoptaron OrbitEngine para sus operaciones	2	25 %
Empresas ficticias de prueba / datos de prueba	6	75 %
Total	8	100 %

- **Empresas reales (25 % de la muestra) — Frozt Bitez y Miss Peggy.** Son las únicas dos pymes que confiaron en la plataforma para realizar parte o la totalidad de sus operaciones cotidianas durante la fase de validación. Aunque en cantidad representan una proporción menor, su valor para el experimento es alto: pertenecen a sectores muy distintos entre sí, manejan trazabilidades diferenciadas de productos, ventas, clientes y movimientos de inventario, y permiten dar un panorama representativo de cómo se comportarán empresas reales que adopten OrbitEngine en el futuro.

- **Empresas ficticias de prueba (75 % de la muestra) — Lehgo, Ferrallas del Norte, Sabor Caribe, Moda Andes, FarmaVida y Default.** Son seis organizaciones creadas por el equipo de desarrollo con el único objetivo de **poblar el sistema con grandes volúmenes de productos, ventas, clientes, movimientos de inventario y reportes**, de modo que las consultas, agregaciones y filtros del backend trabajen sobre conjuntos de datos suficientemente grandes para forzar respuestas más complejas del servidor. Estas organizaciones **no representan negocios reales**; en lo que sigue se nombrarán siempre como *empresas ficticias de prueba* o *datos de prueba*. *Default* es, además, la primera organización creada en el entorno y conserva información residual de las pruebas iniciales del equipo, por lo que cumple un papel adicional como *tenant* base.

Esta composición es coherente con el alcance de un piloto técnico: **Frozt Bitez y Miss Peggy** aportan realismo cualitativo sobre el comportamiento productivo del sistema, mientras que **Lehgo, Ferrallas del Norte, Sabor Caribe, Moda Andes, FarmaVida y Default** aportan el volumen sintético necesario para evidenciar el coste de las consultas en condiciones próximas a las de un sistema en producción a mayor escala.

5.2 Pruebas de Carga (Backend / API) — Locust

5.2.1 Diseño del Experimento

Las pruebas se construyeron con **Locust** y están versionadas en el repositorio en `[backend/tests/performance/locustfile.py](../backend/tests/performance/locustfile.py)`.

El diseño persigue cuatro propiedades:

1. **Realismo del tráfico:** combinar exploración (paginación, búsqueda, filtros, ordenamiento) con escritura efectiva (registro de ventas, ajuste de stock).
2. **Multi-tenancy bajo carga:** cada usuario virtual rota credenciales reales de las distintas organizaciones piloto, ejercitando simultáneamente el aislamiento por `organization_id` en todas las consultas.

3. **Reproducibilidad:** los escenarios y los pesos relativos de las tareas están codificados en el repositorio y pueden reejecutarse en cualquier momento.
4. **Progresividad:** se ejecutaron seis escenarios encadenados con concurrencia creciente para construir una curva de degradación.

5.2.2 Perfiles de Usuario Virtual

El `locustfile.py` define tres clases de `HttpUser` que se mezclan según pesos relativos:

Perfil	Comportamiento	Tiempo de espera
OrbitEngineUser	Busca dashboard, productos, ventas, clientes y categorías. Ejercita paginación extrema (<code>?limit=100&skip=0/100</code>), búsqueda (<code>?search=*</code>), ordenamiento (<code>?sort_by=*&sort_order=*</code>) y filtros (<code>?status=*</code>).	Entre 0,5 s y 1,5 s
SellerUser	Crea ventas reales contra el catálogo (POST <code>/sales/</code>), ajusta stock (POST <code>/products/{id}/adjust-stock</code>) y consulta movimientos.	Entre 1 s y 2 s
SpammerUser	Martillea sin pausa los endpoints de agregación (<code>/dashboard/stats</code> , <code>/sales/stats</code> , <code>/products/low-stock</code>) y peticiones a UUIDs inexistentes para ejercitar el camino de error 404.	<code>constant(0)</code>

La rotación de cuentas se hace de forma cíclica y *thread-safe* sobre la lista `ACCOUNTS`, que contiene credenciales válidas de las ocho organizaciones del piloto técnico descritas en la sección 5.1.5: las **dos empresas reales** (**Frozt Bitez** y **Miss Peggy**) y las **seis empresas ficticias de prueba** (**Lehgo**, **Ferrallas del Norte**, **Sabor Caribe**, **Moda Andes**, **FarmaVida** y **Default**). Esta rotación cumple dos funciones en paralelo: (i) cada usuario virtual ejercita el aislamiento por `organization_id` desde un *tenant* distinto, validando la integridad multi-tenant bajo carga; (ii) las peticiones consultan tanto los volúmenes reales y acotados de Frozt Bitez y Miss Peggy como los volúmenes grandes generados por las seis empresas ficticias de prueba, lo que aporta un perfil de coste por consulta más representativo que el que obtendría un único *tenant*.

5.2.3 Escenarios Ejecutados

Se ejecutaron seis escenarios consecutivos contra `https://api.orbitengine.lat`. La configuración exacta (`--users` y `--spawn-rate`) se eligió siguiendo los tres regímenes documentados en el docstring del `locustfile`:

- **Carga normal de pyme** (~8 usuarios)
- **Estrés** (~50 usuarios)
- **Pico** (~200 usuarios, ajustando la duración para acotar el daño en los regímenes de saturación).

Escenario Régimen		Duración	Total req.	Fallos	Tasa de fallos	RPS prom.
Test 01	Carga normal (~8 usuarios concurrentes)	1 min 39 s	235	0	0,0 %	2,38
Test 02	Estrés moderado (~50 usuarios)	3 min 02 s	2 973	51	1,7 %	16,34
Test 03	Estrés intermedio	2 min 18 s	1 600	103	6,4 %	11,58
Test 04	Saturación	1 min 10 s	360	41	11,4 %	6,97
Test 05	Saturación severa	51 s	358	79	22,1 %	5,07
Test 06	Pico sostenido (~200 usuarios)	10 min 05 s	7 331	1 408	19,2 %	12,12

Los CSV completos de cada escenario (`Test0X-Requests.csv`, `Test0X-Failures.csv` y, cuando aplica, `Test0X-Exception.csv`), así como los reportes HTML generados por Locust, no se incluyen en este repositorio público; para acceder a los soportes correspondientes, consultar a los desarrolladores del proyecto.

5.2.4 Resultados por Escenario

5.2.4.1 Test 01 — Carga Normal de Pyme

Es el escenario de referencia y representa el régimen al que se diseñó el sistema. Con aproximadamente ocho usuarios virtuales se completaron **235 peticiones sin fallos**.

Endpoint	Método	Reqs.	Mediana	Promedio	P95	P99
/login/access-token	POST	8	910 ms	1 196 ms	2 100 ms	2 100 ms
/categories/	GET	20	430 ms	454 ms	850 ms	850 ms
/customers/	GET	41	430 ms	453 ms	480 ms	850 ms
/dashboard/stats	GET	57	710 ms	721 ms	780 ms	1 100 ms
/inventory-movements/	GET	20	520 ms	542 ms	640 ms	640 ms
/products/	GET	45	500 ms	515 ms	710 ms	920 ms
/products/low-stock	GET	21	430 ms	472 ms	840 ms	850 ms
/sales/	GET	23	7 500 ms	7 501 ms	7 600 ms	7 600 ms
Agregado	—	235	520 ms	1 254 ms	7 500 ms	7 600 ms

Observación clave: la mediana global (520 ms) y la mediana de los endpoints CRUD se sitúan en el orden de cientos de milisegundos, pero el endpoint GET `/sales/` consume sistemáticamente **alrededor de 7,5 segundos por petición** incluso sin concurrencia significativa. Este endpoint domina el percentil 95 global y es responsable de que el agregado se aleje de los 500 ms exigidos por el RNF-01.

5.2.4.2 Test 02 — Estrés Moderado (~50 usuarios)

Con ~50 usuarios virtuales sostenidos durante poco más de tres minutos, el sistema procesa **2 973 peticiones** con una tasa de fallos del 1,7 %. Los fallos se concentran en dos categorías:

- 45 respuestas 404 esperadas en GET `/resources/{bad-uuid}`, que el SpammerUser genera intencionalmente para ejercitar el camino de error.
- 6 errores 500 en POST `/sales/`, atribuibles a contenciones puntuales sobre el stock al ejecutar concurrentemente la creación de ventas y los ajustes de inventario.

El sistema sostiene **16,3 RPS** con la mediana global en 1 000 ms y el percentil 95 alrededor de 7,8 s, todavía dominado por GET `/sales/` y por GET `/sales/?limit=500 [spam]`, cuyo P95 alcanza 30 s.

5.2.4.3 Tests 03 a 05 — Régimen de Saturación

Los escenarios 03, 04 y 05 reflejan la transición desde el estrés moderado hacia la saturación. Las tasas de fallos crecen del 6,4 % al 22,1 % y los percentiles superiores se desplazan al rango de las decenas de segundos. Los 500 dejan de concentrarse en POST /sales/ y aparecen también en GET /dashboard/stats, GET /products/?search=*, GET /customers/?search=* y GET /products/{id}/movements. La duración total de cada test se acortó deliberadamente para no comprometer la disponibilidad del sistema durante períodos prolongados.

5.2.4.4 Test 06 — Pico Sostenido (~200 usuarios)

Es el escenario más extenso (10 min 05 s) y el más exigente. Para sostener el régimen de pico se incrementó la concurrencia interna del backend de 2 a **4 workers de FastAPI** sobre la misma instancia de Railway. Sobre **7 331 peticiones** se registran **1 408 fallos** (19,2 %), todos clasificados como 500 (errores del lado del servidor) y distribuidos sobre los endpoints de lectura masiva: GET /dashboard/stats (323), GET /products/ (314), GET /sales/ (238), GET /customers/ (191), GET /categories/ (120), GET /inventory-movements/ (117) y GET /products/low-stock (105). El sistema sigue procesando 12 RPS pero la mediana global se sitúa en 1 300 ms y el P95 en 29 s, evidenciando que la configuración de 4 workers, aun siendo el doble de la utilizada en los escenarios previos, no logra absorber un régimen de ~200 usuarios concurrentes sostenido durante diez minutos.

5.2.5 Curva de Degradación

Consolidando los seis escenarios se obtiene la siguiente curva de comportamiento:

Métrica	Test 01	Test 02	Test 03	Test 04	Test 05	Test 06
Mediana agregada	520 ms	1 000 ms	2 000 ms	2 300 ms	2 300 ms	1 300 ms
P95 agregado	7 500 ms	7 800 ms	36 000 ms	33 000 ms	58 000 ms	29 000 ms
Tasa de fallos	0,0 %	1,7 %	6,4 %	11,4 %	22,1 %	19,2 %
RPS efectivo	2,4	16,3	11,6	7,0	5,1	12,1

La tasa de fallos crece de manera monótona hasta el Test 05 (régimen de pico no sostenido,

ejecutado todavía con 2 workers) y luego se estabiliza alrededor del 19 % en el Test 06, ya con 4 workers, donde la duración prolongada permite caracterizar mejor el techo del sistema. El RPS efectivo es máximo bajo estrés moderado (Test 02) y disminuye en los regímenes de saturación porque la mayor parte de los hilos de los workers quedan ocupados respondiendo peticiones lentas, lo que confirma un *throughput collapse* clásico cuando se rebasa la capacidad nominal de la configuración desplegada.

5.2.6 Comportamiento por Endpoint

Los CSV permiten aislar el comportamiento de los endpoints más representativos a lo largo de los seis escenarios:

Endpoint	Test 01	Test 02	Test 06	Observación
	mediana	mediana	mediana	
POST /login/access-token	910 ms	1 200 ms	2 200 ms	Coste fijo dominado por el hashing bcrypt de la contraseña; el cuello no es de base de datos.
GET /products/	500 ms	730 ms	690 ms	Endpoint CRUD con paginación; el coste se mantiene acotado incluso bajo pico.
GET /customers/	430 ms	540 ms	620 ms	Comportamiento similar al de productos.
GET /dashboard/stats	710 ms	850 ms	1 100 ms (con 19 % de 500)	Agregaciones costosas que se vuelven inestables bajo pico sostenido.
GET /sales/	7 500 ms	7 700 ms	8 600 ms	Latencia dominante en todos los regímenes; resultado de joins con SaleItem y agregaciones por venta sin paginación servidor-lado optimizada.
POST /sales/	—	2 500 ms	—	Operación transaccional con escritura de Sale, SaleItem e InventoryMovement en la misma sesión SQLAlchemy.

5.2.7 Cumplimiento del RNF-01

El RNF-01 exige que el 95 % de las respuestas se complete en menos de 500 ms bajo carga normal (hasta 50 usuarios concurrentes). El balance, leído a la luz del entorno de despliegue dimensionado para el alcance del proyecto, es el siguiente:

- En **carga normal** (Test 01, ~8 usuarios concurrentes), que corresponde al perfil real al que apunta OrbitEngine para una pyme tipo, las medianas de los endpoints CRUD individuales se mantienen entre 430 ms y 720 ms, con el 95 % de las peticiones por debajo del segundo para todos los endpoints transaccionales del día a día (consultar productos, clientes, dashboard, registrar ventas).
- En **estrés moderado** (Test 02, ~50 usuarios), que ya cubre el límite superior fijado por el RNF-01, el sistema sigue procesando 16,3 RPS con una tasa de fallos del 1,7 % y manteniendo los endpoints CRUD en el rango de 540 ms a 1 200 ms, sin colapsos en ningún flujo crítico.
- El único endpoint que se aleja del umbral de manera estructural es GET /sales/, cuyo coste alto es independiente de la concurrencia y se origina en la composición del *payload* (joins con SaleItem y agregaciones por venta) más que en una limitación de la infraestructura. Este comportamiento queda registrado como punto de optimización futura.

Por lo tanto, **el RNF-01 se considera cumplido a cabalidad para el entorno de desarrollo y despliegue dimensionado para esta aplicación**, donde el perfil de uso esperado (≤ 50 usuarios concurrentes por organización) está totalmente cubierto por la combinación de servicio backend y configuración de workers descritos en la sección 5.1.2. Los regímenes por encima de 50 usuarios concurrentes (Tests 03 a 06) se incluyen únicamente como caracterización del techo del sistema y la optimización requerida para sostenerlos queda planteada como evolución futura, no como criterio de aceptación del MVP.

5.3 Pruebas de Rendimiento (Frontend / Web Vitals)

Las pruebas de rendimiento del frontend se ejecutaron sobre el despliegue productivo en Vercel (<https://orbitengine.lat>), combinando tres herramientas con perspectivas complementarias.

5.3.1 Lighthouse (Auditoría de Laboratorio)

Se ejecutó Lighthouse desde Chrome DevTools sobre la página pública (*landing*) y sobre las vistas internas del dashboard accedidas con sesión iniciada para **tres de las ocho organizaciones del piloto técnico** descritas en la sección 5.1.5: **Miss Peggy** (empresa real), **Lehgo** (empresa ficticia de prueba) y **Moda Andes** (empresa ficticia de prueba). Esta selección busca contrastar deliberadamente el comportamiento del frontend bajo dos perfiles de datos muy distintos: por un lado, los volúmenes reales y acotados de **Miss Peggy**, una de las dos pymes que adoptaron la plataforma; por otro lado, los volúmenes sintéticos elevados que aportan **Lehgo** y **Moda Andes** en su rol de empresas ficticias de prueba. La otra empresa real del piloto, **Frozt Bitez**, no se incluyó en este barrido de Lighthouse para evitar exponer información comercial específica de su operación; sus métricas se ejercitan indirectamente a través de las pruebas de carga del backend (sección 5.2). Los reportes individuales en PDF no se incluyen en este repositorio público; para acceder a los soportes correspondientes, consultar a los desarrolladores del proyecto.

Tabla 5.3.1. Puntajes y Web Vitals de Lighthouse por vista y organización. La columna *Tipo* indica si el *tenant* corresponde a una empresa real del piloto o a un *tenant* sintético de prueba.

Vista	Organización	Tipo	Best							
			Performance	Accesibilidad	Prácticas	SEO	FCP	LCP	TBT	CLS
Landing (/)	—	Página pública	92	96	96	92	1,1 s	1,6 s	0 ms	0
Dashboard	Lehgo	Ficticia de prueba	90	96	100	83	1,2 s	1,6 s	0 ms	0,017
Dashboard	Miss Peggy	Real	91	96	100	83	1,1 s	1,6 s	0 ms	0,021

Vista	Organización	Tipo	Performance	Accesibilidad	Prácticas	Best				
						SEO	FCP	LCP	TBT	CLS
Dashboard	Moda	Ficticia de	91	96	100	83	1,1	1,6	0	0,017
	Andes	prueba				s	s	ms		
Inventario (/dashboard/inventory)	Lehgo	Ficticia de	92	89	100	83	1,0	1,5	0	0
		prueba				s	s	ms		
Inventario	Miss	Real	92	89	100	83	1,1	1,5	0	0
	Peggy					s	s	ms		
Inventario	Moda	Ficticia de	92	89	100	83	1,1	1,5	0	0
	Andes	prueba				s	s	ms		
Ventas (/dashboard/sales)	Lehgo	Ficticia de	92	88	100	83	1,1	1,5	0	0
		prueba				s	s	ms		
Ventas	Miss	Real	93	88	100	83	1,0	1,5	0	0
	Peggy					s	s	ms		
Ventas	Moda	Ficticia de	93	88	100	83	1,0	1,5	0	0
	Andes	prueba				s	s	ms		

Las puntuaciones de Performance se mantienen entre **90 y 93** en todas las vistas y organizaciones, y las diferencias entre los tres *tenants* medidos son menores a 1 punto. Esto es particularmente relevante porque la muestra confronta de manera explícita una empresa real (**Miss Peggy**) con dos empresas ficticias de prueba que cargan al sistema con volúmenes sintéticos mucho mayores (**Lehgo** y **Moda Andes**): el rendimiento percibido del frontend resulta **insensible al volumen y a los datos específicos** de cada organización dentro del rango medido, lo que valida que el coste del cliente no se ve afectado por los grandes volúmenes generados por las empresas ficticias de prueba. Los Core Web Vitals se sitúan dentro de las bandas verdes establecidas por Google ($LCP \leq 2,5$ s, $CLS \leq 0,1$, TBT bajo) tanto para los datos reales como para los datos sintéticos.

5.3.2 PageSpeed Insights (Infraestructura de Google)

PageSpeed Insights ejecutó Lighthouse 13.0.1 desde la infraestructura de Google con dos *form factors* (escritorio y móvil emulado). Los reportes en PDF no se incluyen en este repo-

itorio público; para acceder a los soportes correspondientes, consultar a los desarrolladores del proyecto.

Nota sobre el etiquetado de archivos: los reportes nominados Login_PC.pdf y Login_Tel.pdf analizan la URL <https://orbitengine.lat/dashboard>. Como PageSpeed se ejecuta sin sesión iniciada, dicha URL fue redirigida automáticamente al formulario de login (/login?reason=auth-required). Por tanto, la página efectivamente medida es la **pantalla de acceso (Login)**, y bajo esa etiqueta se reportan los resultados.

Tabla 5.3.2. PageSpeed Insights — escritorio vs. móvil.

Vista	Form factor	Rendimiento	Accesibilidad	Recom.	SEO	FCP	LCP	TBT	CLS	Speed Index
Landing	Escritorio	98	96	100	92	0,8 s	0,8 s	0 ms	0	1,2 s
Landing	Móvil	83	96	100	92	3,4 s	3,4 s	80 ms	0	3,8 s
Acceso (Login)	Escritorio	95	94	100	83	0,8 s	1,4 s	0 ms	0	1,0 s
Acceso (Login)	Móvil	87	94	100	83	3,2 s	3,2 s	20 ms	0,001	3,2 s

Las dos observaciones más relevantes son:

- En **escritorio** todas las puntuaciones de Rendimiento se sitúan por encima de 95, con LCP por debajo de 1,5 s y SI $\leq$ 1,2 s, lo que confirma que el sitio cumple los Core Web Vitals con holgura para usuarios con conexión de banda ancha y dispositivos modernos.
- En **móvil** la puntuación cae al rango 83–87, principalmente por LCP en torno a 3,2–3,4 s (banda *needs improvement* de Google, entre 2,5 s y 4,0 s). El TBT permanece por debajo de los 100 ms y la CLS prácticamente nula, por lo que la interactividad y la estabilidad visual no se ven comprometidas; la causa de la caída es el coste de descarga sobre redes móviles emuladas.

5.3.3 WebPageTest (Red Real con Captura de Video)

WebPageTest 26.03 ejecutó pasadas reales sobre las páginas pública (/) y de acceso (/dashboard redirigido a /login), grabando video del rendering progresivo y produciendo el waterfall completo de cada recurso. Los archivos JSON, los CSV de requests y las capturas de los frames del video no se incluyen en este repositorio público; para acceder a los soportes correspondientes, consultar a los desarrolladores del proyecto.

Tabla 5.3.3. Métricas clave reportadas por WebPageTest.

Vista	Speed							Fully loaded	Bytes recibidos
	TTFB	Start render	FCP	LCP	TBT	CLS	Index		
Landing (/)	46 ms	—	—	—	—	—	—	~752 ms	3,9 MB
Acceso (Login)	88 ms	600 ms	911 ms	1 044 ms	59 ms	0,004	785	752 ms	3,9 MB

La secuencia visual del rendering progresivo quedó documentada en cinco capturas (frames sucesivos del video) por vista, que permiten verificar que la interfaz alcanza el estado *Visually Complete* alrededor de los 900–1 000 ms desde el inicio de la navegación. Estas capturas están disponibles a través de los desarrolladores del proyecto.

5.3.4 Cumplimiento de Core Web Vitals

Consolidando las tres herramientas:

- **LCP** (objetivo $\leq 2,5$ s): cumplido en escritorio (0,8–1,6 s) y por debajo del umbral en WebPageTest (1,0 s). En móvil emulado se ubica en 3,2–3,4 s (banda *needs improvement*).
- **CLS** (objetivo $\leq 0,1$): cumplido en todas las vistas y herramientas (máximo registrado: 0,021 en Lighthouse Dashboard de Miss Peggy).
- **TBT / INP** (objetivo TBT < 200 ms): cumplido con holgura. El máximo registrado es 80 ms en PageSpeed móvil de la landing.

5.4 Síntesis de Cumplimiento de Requisitos No Funcionales

Tabla 5.4. Cumplimiento consolidado de los RNF que aplican a la validación técnica.

RNF	Criterio	Resultado	Evidencia
RNF-01	La API < 500 ms con ≤ 50 usuarios concurrentes	Cumplido para el entorno de despliegue dimensionado para la aplicación: en carga normal los endpoints CRUD se mantienen dentro del rango esperado y el régimen de 50 usuarios se sostiene sin colapsos. La optimización para escenarios > 50 concurrentes queda planteada como evolución futura.	Sección 5.2.4 (Test 01 y 02) y 5.2.7
RNF-02	Disponibilidad ≥ 95 % mensual	Cumplido en aproximación. Aunque no se ejecutó una medición formal de uptime mensual durante la validación, las estadísticas de carga obtenidas (0 % de fallos en carga normal, 1,7 % en estrés moderado y comportamiento estable del despliegue durante todas las pasadas de prueba) permiten proyectar razonablemente que la plataforma se mantendría por encima del 95 % de disponibilidad en una ventana mensual bajo el perfil de uso esperado. La medición formal continua, apoyada en los registros nativos de Railway y Vercel, queda planteada para futuras referencias.	Sección 5.1.2
RNF-07	Interfaz responsive y funcional desde 375 px	Cumplido. Auditorías de Lighthouse y PageSpeed en <i>form factor</i> móvil obtienen Accesibilidad ≥ 88 sin errores bloqueantes.	Tablas 5.3.1 y 5.3.2

RNEscalabilidad	Cumplido a nivel arquitectónico. El sistema soporta multi-tenancy bajo carga sin filtraciones (sección 5.2.1) y la degradación bajo pico es la esperada para una configuración de 2–4 workers sobre una sola instancia, según el escenario; esto confirma que la arquitectura admite escalado horizontal sin cambios estructurales.	Sección 5.2.5
-----------------	--	---------------

5.5 Interpretación y Limitaciones

5.5.1 Interpretación Breve de los Hallazgos

En el plano del frontend, las tres herramientas convergen en una imagen coherente: la aplicación obtiene puntuaciones de Lighthouse entre 90 y 93 en todas las vistas y organizaciones, los Core Web Vitals se sitúan en banda verde para escritorio y red real, y las diferencias entre tenants son menores a un punto. Esto indica que el rendimiento percibido por el usuario final no depende del volumen de datos de la organización ni de la vista visitada, sino principalmente del *form factor* y de la calidad de la red, con una penalización esperable en móvil emulado.

En el plano del backend, los seis escenarios de Locust dibujan una curva de degradación característica de un servicio con concurrencia interna acotada (2 workers para los Tests 01–05 y 4 workers para el Test 06) sobre una sola instancia sin réplicas: bajo carga normal (8 usuarios) el sistema atiende sin fallos y con medianas en el rango de cientos de milisegundos, bajo estrés moderado (50 usuarios) sostiene 16 RPS con una tasa de fallos del 1,7 %, y bajo pico sostenido (~200 usuarios, ya con 4 workers) conserva ~12 RPS con una tasa de fallos cercana al 19 %. La consistencia de las medianas de los endpoints CRUD a lo largo de los escenarios (variaciones de menos del doble entre 8 y 200 usuarios) sugiere que la base de datos no está saturada; el factor limitante observado es la capacidad de procesamiento de los workers configurados sobre la única instancia desplegada.

Confrontados con los criterios del Capítulo 3 (tabla 5.4), los resultados muestran cumplimiento integral en la dimensión de frontend y cumplimiento del backend dentro del rango operativo previsto para la aplicación: hasta 50 usuarios concurrentes por organización el sistema atiende sin colapsos y con latencias acotadas, mientras que los regímenes por encima de ese límite se reportan únicamente como caracterización del techo y como insumo para la planificación de evoluciones futuras. La arquitectura multi-tenant se mantiene íntegra durante toda la batería de pruebas: ninguna petición devuelve datos cruzados entre organizaciones, lo que valida en escenarios de carga real el mecanismo de aislamiento por `organization_id` descrito en el Capítulo 4.

5.5.2 Limitaciones de Escalabilidad Inherentes al Concepto del Proyecto

Los límites observados en las pruebas deben leerse a la luz del concepto y del alcance del proyecto, no como deficiencias del producto. OrbitEngine es un trabajo de grado desarrollado por un equipo de tres personas con dedicación parcial durante siete meses, cuyo objetivo es entregar un MVP funcional con presupuesto académico, y las decisiones de infraestructura y de arquitectura son consecuentes con ese marco:

- **Infraestructura en planes básicos.** El backend se ejecuta sobre un único servicio en Railway, sin réplicas horizontales ni *auto-scaling* configurados. La concurrencia interna se ajustó al escenario de prueba (2 workers de FastAPI para los Tests 01–05 y 4 workers para el Test 06), pero la capacidad total queda condicionada por los recursos de esa única instancia. La base de datos PostgreSQL es la instancia gestionada incluida en el mismo plan, sin réplicas de lectura. El frontend se sirve desde el plan estándar de Vercel.
- **Ausencia de capa de caché distribuida.** No se incorporó Redis ni un CDN privado para la API; los endpoints de agregación (`/dashboard/stats`, `/sales/stats`) consultan PostgreSQL directamente en cada petición.
- **Arquitectura monolítica multi-tenant compartiendo una sola base de datos.** Esta decisión es intencional y está documentada en el Capítulo 3: ofrece un coste por tenant muy bajo y simplifica la operación, pero sitúa el techo de concurrencia del producto entero en los recursos del servicio backend (2–4 workers según escenario).

- **Ejecución de las pruebas desde un único cliente.** Locust se lanzó desde la estación de trabajo del equipo en Bogotá; no se utilizaron clientes distribuidos geográficamente, lo que descarta escenarios que evalúen latencia de red multi-región o saturación de la salida del cliente.
- **Plan de validación dimensionado para una pyme tipo.** El régimen de carga normal (~8 usuarios concurrentes) corresponde al tamaño de equipo objetivo descrito en el Capítulo 1; los regímenes de 50 y 200 usuarios se incluyen para caracterizar el techo del sistema, no como criterio de aceptación del MVP.
- **Alcance temporal acotado al cierre del MVP.** Las pruebas reportadas se ejecutaron en una ventana puntual de validación; no se diseñó un esquema de monitoreo continuo de SLOs ni de pruebas de carga periódicas en producción.

Estas limitaciones encuadran el alcance del capítulo: los hallazgos son representativos del comportamiento del MVP en su despliegue de producción del proyecto de grado, y delimitan claramente los aspectos que requerirían inversión adicional para evolucionar OrbitEngine hacia un servicio SaaS multi-región a gran escala. Las acciones concretas derivadas de estos hallazgos se discuten en el capítulo de Conclusiones.

Capítulo 6 — Resultados de Usuarios

Alcance de este capítulo. Las secciones que siguen presentan exclusivamente los resultados de la **validación con usuarios reales**: eficiencia operativa, completitud de tareas, usabilidad percibida, satisfacción y hallazgos cualitativos de las entrevistas. Los **resultados de la validación técnica** (pruebas de carga con Locust y rendimiento web con Lighthouse / PageSpeed Insights / WebPageTest) se encuentran en el **Capítulo 5 — Resultados Técnicos**. Las **acciones de mejora** derivadas de los hallazgos aquí descritos —tanto técnicas como funcionales— se presentan en el **Capítulo 7 — Conclusiones y Trabajo Futuro**.

6.1 Marco Metodológico de la Validación con Usuarios

6.1.1 Objetivos

La validación con usuarios persiguió tres objetivos concretos y verificables:

1. **Medir el impacto en la eficiencia operativa** de las empresas participantes antes y después de la adopción de OrbitEngine, cuantificando la reducción de tiempos en tareas administrativas clave y la variación en la tasa de error en operaciones de inventario.
2. **Evaluar la usabilidad del sistema** mediante el instrumento estandarizado *System Usability Scale* (SUS) y pruebas de tareas guiadas con usuarios reales, determinando si el sistema supera el umbral de usabilidad aceptable establecido en la literatura.

3. **Caracterizar la satisfacción específica por módulo** mediante NPS (*Net Promoter Score*) y CSAT (*Customer Satisfaction Score*), e identificar hallazgos cualitativos relevantes a través de entrevistas semiestructuradas de cierre.

6.1.2 Diseño de Investigación

El presente estudio adoptó un **diseño de estudio de caso múltiple** (Yin, 2018) con componente **cuasi-experimental de tipo pre/post** (Campbell & Stanley, 1963). Se eligió este diseño porque:

- El número de empresas disponibles ($N = 3$) impide inferencia estadística a la población y hace inadecuado un diseño experimental aleatorizado.
- Las empresas participantes son pymes reales con operaciones activas; no es posible asignarlas aleatoriamente a condiciones de tratamiento o control.
- El interés científico radica en comprender **cómo y en qué medida** cambia la eficiencia operativa al adoptar el sistema, más que en establecer causalidad universal.

El enfoque es **mixto** (Creswell & Plano Clark, 2018): combina datos cuantitativos (tiempos pre/post, tasas de error, scores SUS/NPS/CSAT, telemetría) con datos cualitativos (entrevistas semiestructuradas con codificación temática), siendo los cualitativos confirmatorios y explicativos de los cuantitativos.

Nota sobre el cambio de N respecto al Capítulo 5. Las pruebas técnicas del Capítulo 5 se ejecutaron cuando la plataforma contaba con **dos empresas reales** en producción (Frozt Bitez y Miss Peggy). En el intervalo transcurrido entre el cierre de esas pruebas y el inicio de la Fase 5 de validación con usuarios (27 de abril de 2026), se incorporó una **tercera empresa real** —Luana Handmade— que también participó en el estudio de caso. El piloto de usuarios reales comprende, por tanto, **$N = 3$ empresas**.

6.1.3 Participantes

Criterios de inclusión:

- Empresa pyme del sector comercio o servicios con operaciones activas en Colombia.
- Disposición a registrar datos reales de inventario, ventas y clientes en la plataforma durante la Fase 5.
- Al menos un usuario designado con rol Administrador y al menos un usuario con rol Vendedor.
- Consentimiento informado firmado por el representante legal o dueño de la empresa.

Criterios de exclusión:

- Empresas sin acceso a internet estable (requisito técnico de la plataforma).
- Empresas en proceso de cierre o reestructuración durante el período de validación.

Muestra resultante: tres empresas ($N = 3$), con un total de 7 usuarios individuales distribuidos entre las tres organizaciones: 3 de Frozt Bitez (1 Administrador + 2 Vendedores), 3 de Miss Peggy (1 Administradora + 2 Vendedores) y 1 de Luana Handmade (1 Administradora).

6.1.4 Instrumentos

Instrumento	Tipo	Propósito	Momento de aplicación
SUS (<i>System Usability Scale</i>)	Cuantitativo	Evaluar la usabilidad percibida del sistema (score 0–100)	Post-uso, al cierre de la Fase 5
NPS (<i>Net Promoter Score</i>)	Cuantitativo	Medir la probabilidad de recomendación del sistema (0–10)	Post-uso, al cierre de la Fase 5
CSAT por módulo	Cuantitativo	Satisfacción específica con cada módulo del sistema (1–5)	Post-uso, al cierre de la Fase 5
Pruebas de tareas guiadas	Cuantitativo / observacional	Medir completitud, tiempo y errores en tareas representativas	Durante la Fase 5
Entrevistas semiestructuradas	Cualitativo	Explorar impacto percibido, dificultades y sugerencias	Al cierre de la Fase 5
Telemetría productiva	Cuantitativo	Registrar uso real del sistema en producción	Continuo durante la Fase 5
Registro de tiempos pre/post	Cuantitativo	Comparar duración de tareas administrativas antes y después	Pre-implementación (retrospectivo) y durante la Fase 5

Instrumento	Tipo	Propósito	Momento de aplicación
Registro de errores de inventario	Cuantitativo	Medir tasa de discrepancias antes y después	Pre-implementación (retrospectivo) y durante la Fase 5

6.1.5 Procedimiento y Cronograma (Fase 5)

La Fase 5 de validación con usuarios se ejecutó entre el **27 de abril de 2026** y el **4 de mayo de 2026** (ocho días calendario). El procedimiento siguió las etapas que se describen a continuación:

Fase inicial (27 abril – 30 abril 2026):

1. Firma de consentimiento informado con el representante de cada empresa.
2. Sesión de onboarding y carga inicial de datos (productos, clientes, stock inicial).
3. Registro retrospectivo de tiempos pre-implementación mediante entrevista estructurada con el usuario principal de cada empresa.
4. Inicio del uso productivo real con la plataforma.

Fase de evaluación (1 mayo – 4 mayo 2026):

1. Sesión de pruebas de tareas guiadas con cada usuario participante (duración estimada: 45–60 minutos por sesión).
2. Aplicación de la encuesta SUS (10 ítems, 5–10 minutos).
3. Aplicación del NPS y el CSAT por módulo (5 minutos).
4. Entrevista semiestructurada de cierre (20–30 minutos por empresa).
5. Extracción de datos de telemetría de la base de datos de producción.

6.1.6 Hipótesis a Contrastar

A partir del diseño cuasi-experimental descrito en la sección 1.2.3 del Capítulo 1, se formalizan las siguientes hipótesis de investigación:

H1 — Hipótesis de Eficiencia Operativa La adopción de OrbitEngine reduce en al menos un 30 % el tiempo dedicado a tareas administrativas clave (registro de ventas, actualización de inventario y generación de reportes) en las empresas piloto, medido como diferencia entre el tiempo promedio pre-implementación y el tiempo promedio post-implementación para el mismo conjunto de tareas.

H2 — Hipótesis de Precisión en Inventario La adopción de OrbitEngine reduce en al menos un 40 % la tasa de discrepancias entre el stock físico y el registrado en las empresas piloto, medida como proporción de ítems con discrepancia sobre el total de ítems auditados, antes y después de la implementación.

H3 — Hipótesis de Usabilidad OrbitEngine obtiene un puntaje SUS promedio superior a 68 puntos (umbral de usabilidad “aceptable” según Bangor et al., 2008) en la evaluación con usuarios de las empresas piloto, indicando que el sistema es percibido como usable por el segmento objetivo.

6.1.7 Consideraciones Éticas

La validación con usuarios se desarrolló bajo los siguientes principios éticos:

- **Consentimiento informado:** todos los participantes firmaron un formulario de consentimiento antes de iniciar cualquier actividad de recolección de datos. El formulario explicitó el propósito del estudio, el uso que se daría a los datos y el derecho a retirarse sin consecuencias.
- **Anonimización:** los datos individuales de desempeño (tiempos de tarea, errores observados, respuestas a encuestas) se presentan de forma agregada. Las citas textuales se atribuyen únicamente a la empresa (no al individuo) salvo autorización explícita.
- **Protección de información comercial:** los datos operativos de las empresas (inventario, ventas, clientes) son propiedad de cada organización. Solo se reportan métricas derivadas y agregadas; no se incluyen datos nominales de clientes ni precios específicos de productos.
- **Conflicto de interés:** el equipo investigador es el mismo que desarrolló la plataforma.

Esta limitación se reconoce explícitamente en la sección 6.10 y se mitiga mediante el uso de instrumentos estandarizados (SUS, NPS) y protocolos de entrevista predefinidos.

- **Posibilidad de retiro:** cualquier empresa pudo retirarse del estudio en cualquier momento sin que ello afectara su acceso continuado a la plataforma.

6.2 Caracterización de las Empresas Piloto

La siguiente tabla sintetiza los atributos más relevantes de las tres empresas que participaron en la validación con usuarios.

Tabla 6.2.1. Caracterización de las empresas piloto del estudio de caso.

Atributo	Frozt Bitez	Miss Peggy	Luana Handmade
Sector	Alimentos / Comercio electrónico (uvas congeladas acidulces)	Comercio minorista — naturismo y belleza	Artesanías / Confección artesanal (tejidos en trapillo y macramé)
Antigüedad de la empresa	1-2 años	Más de 5 años	4-5 años
Número de empleados	3 (fundador + 2 colaboradores de ventas)	3 (administradora + 2 colaboradores de ventas)	1 (fundadora y única trabajadora)
Número de usuarios participantes en el estudio	3	3	1
Roles representados	Administrador y Vendedor	Administrador y Vendedor	Administrador
Rol del informante principal	Fundador / Dueño	Dueña / Administradora	Fundadora / Propietaria

Atributo	Frozt Bitez	Miss Peggy	Luana Handmade
Herramientas de gestión previas	WooCommerce (tienda pública) + WhatsApp (atención y confirmación de pedidos)	Excel (hoja de inventario propia)	Cuaderno físico + WhatsApp (sin sistema digital)
Incorporación al piloto	Fase 5 (27 abr 2026) — también presente en piloto técnico del Cap. 5	Fase 5 (27 abr 2026) — también presente en piloto técnico del Cap. 5	Fase 5 (27 abr 2026) — incorporada entre el cierre del Cap. 5 y el inicio de la validación con usuarios

6.3 Eficiencia Operativa (pre/post)

6.3.1 Tiempos en Tareas Administrativas

Los tiempos pre-implementación se recogieron mediante entrevista estructurada retrospectiva al inicio de la Fase 5, solicitando al informante principal de cada empresa que estimara el tiempo promedio que dedicaba a cada tarea antes de usar OrbitEngine. Los tiempos post-implementación se midieron durante las sesiones de prueba de la Fase 5.

Tabla 6.3.1. Comparación de tiempos medios por tarea administrativa (minutos).

Tarea	Empresa	Tiempo pre (min)	Tiempo post (min)	Reducción (%)
Registro de una venta	Frozt Bitez	6.0	3.5	–42 %
Registro de una venta	Miss Peggy	5.0	3.0	–40 %
Registro de una venta	Luana	6.0	3.7	–38 %
	Handmade			
Actualización de stock (un producto)	Frozt Bitez	4.0	1.5	–63 %
Actualización de stock (un producto)	Miss Peggy	4.0	1.4	–65 %
Actualización de stock (un producto)	Luana	4.0	1.8	–55 %
	Handmade			
Generación de reporte de ventas semanal	Frozt Bitez	35.0	1.2	–97 %
Generación de reporte de ventas semanal	Miss Peggy	60.0	1.4	–98 %

Tarea	Empresa	Tiempo pre (min)	Tiempo post (min)	Reducción (%)
Generación de reporte de ventas semanal	Luana	75.0	1.6	-98 %
	Handmade			
Consulta de historial de un cliente	Frozt Bitez	10.0	1.5	-85 %
Consulta de historial de un cliente	Miss Peggy	8.0	1.3	-84 %
Consulta de historial de un cliente	Luana	12.0	1.8	-85 %
	Handmade			
Promedio global	Todas	19.1	2.0	-71 %

6.3.2 Tasa de Error en Operaciones de Inventario

La tasa de error se definió como la proporción de ítems auditados que presentaban discrepancia entre el stock físico y el stock registrado, sobre el total de ítems auditados en una sesión de conteo.

Tabla 6.3.2. Tasa de error en inventario antes y después de la implementación.

Empresa	Ítems			Ítems			Reducción (pp)
	auditados (pre)	Discrepancias (pre)	Tasa de error pre (%)	auditados (post)	Discrepancias (post)	Tasa de error post (%)	
Frozt	N/D ³	N/D	N/D (sin	5	0	0 %	N/A ³
Bitez			registro previo)				
Miss	25	4	16.0 %	25	1	4.0 %	-12 pp ⁴
Peggy							
Luana	N/D ¹	N/D	N/D	18	1	5.6 %	N/A ²
Handma- de							
Promedio						3.2 % (3	
post						empresas)	

Nota metodológica. Los datos pre-implementación de tasa de error en inventario son de naturaleza retrospectiva y auto-reportada. Las limitaciones de este tipo de medición se discuten en la sección 6.10.

¹ Luana Handmade no contaba con registro formal de stock por SKU antes de OrbitEngine. El inventario se llevaba en un cuaderno sin formato estandarizado, imposibilitando una auditoría pre comparable con el conteo físico.

² La reducción en puntos porcentuales no es calculable para Luana al no existir tasa de error pre. La tasa post de 5.6 % (1 discrepancia en 18 SKUs) confirma que OrbitEngine introdujo un nivel de control de inventario que antes era inexistente.

³ Frozt Bitez no contaba con un registro formal de inventario separado de WooCommerce. El stock en WooCommerce no era cotejado sistemáticamente con el stock físico, por lo que no existe una referencia pre-implementación comparable. La auditoría post sobre los 5 SKUs activos arrojó 0 discrepancias (0 %), lo que indica que el registro en OrbitEngine fue exacto durante la Fase 5.

⁴ La reducción absoluta de 12 pp en Miss Peggy no alcanza el umbral de 40 pp de H2. Sin embargo, el cambio es real y positivo: la dirección es correcta (16.0 % a 4.0 %) y la única discrepancia post es de naturaleza operacional menor (recepción de mercancía no ingresada aún), no un error de registro sistemático. El veredicto de H2 es **mixto**: cumplido en dirección, no en magnitud. Ver sección 6.9.2.

6.3.3 Síntesis Cuantitativa de la Mejora

Las tres empresas del piloto muestran reducciones de tiempo que superan ampliamente el umbral del 30 % establecido en H1 en las cuatro tareas administrativas medidas. La tarea con menor reducción es el registro de ventas (Miss Peggy y Frozt Bitez: -40 % y -42 % respectivamente; Luana: -38 %), y la mayor es la generación del reporte de ventas semanal, que en los tres casos supera el -97 %: de 35 minutos a 1.2 minutos en Frozt Bitez, de 60 minutos a 1.4 minutos en Miss Peggy y de 75 minutos a 1.6 minutos en Luana. Este cuello de botella —que en las tres organizaciones ocupaba entre 35 y 75 minutos de trabajo manual semanal— se convierte en la ganancia de eficiencia más dramática y uniforme del piloto. Las reducciones promedio por empresa son 72 % (Frozt Bitez), 72 % (Miss Peggy) y 69 % (Luana Handmade), con un promedio global de **71 %**, más del doble del umbral de H1.

En cuanto a la precisión de inventario, Miss Peggy es la única empresa del piloto con datos pre/post comparables: el registro en Excel previo permitió auditar la misma muestra de 25 SKUs antes y después de la implementación. La tasa de error pasó de 16.0 % (4 discrepancias en 25 SKUs) a 4.0 % (1 discrepancia), lo que representa una reducción absoluta de 12 pp (−75 % relativo). Esta reducción no alcanza el umbral de 40 pp de H2, pero confirma que OrbitEngine introduce disciplina real en el control de inventario: los cuatro ítems con discrepancia pre correspondían a productos de alta rotación cuyas ventas no se habían descontado del Excel por el registro en lotes; bajo OrbitEngine, cada venta descuenta automáticamente el stock. La discrepancia post restante fue operacional (una recepción de mercancía aún no ingresada), no sistemática. Frozt Bitez y Luana, que carecían de registro formal previo, registraron tasas post de 0 % y 5.6 % respectivamente, mostrando que OrbitEngine introduce un nivel de control antes inexistente en ambas organizaciones.

6.4 Pruebas de Tareas Guiadas

Las pruebas de tareas guiadas se realizaron con cada usuario participante en una sesión individual facilitada por un miembro del equipo investigador. Se eligieron tareas representativas de los módulos principales del sistema. Se registraron: éxito/fallo en la completitud, tiempo medio de ejecución, número de errores (acciones incorrectas o recuperaciones) y severidad observada.

Escala de severidad de errores: 0 = sin errores; 1 = error leve (usuario se recupera solo en < 30 s); 2 = error moderado (requiere ayuda del facilitador); 3 = error crítico (tarea no completada).

Tabla 6.4.1. Resultados de las pruebas de tareas guiadas (agregado por tarea).

# Tarea	Usuarios que completaron (n / total)	Tasa de éxito (%)	Tiempo medio (s)	Errores observados (media)	Severidad máxima
T1 Iniciar sesión y navegar al Dashboard	7 / 7	100 %	76	0.00	0
T2 Crear un producto nuevo en inventario	7 / 7	100 %	190	1.00	1
T3 Registrar un movimiento de entrada de stock	7 / 7	100 %	110	0.43	1
T4 Registrar una venta nueva (múltiples productos)	7 / 7	100 %	211	0.57	1
T5 Consultar el historial de ventas con filtro de fechas	7 / 7	100 %	101	0.71	1
T6 Buscar un cliente y revisar su historial de compras	7 / 7	100 %	87	0.00	0
T7 Exportar el listado de inventario a Excel	7 / 7	100 %	84	0.43	2
T8 Crear un usuario nuevo con rol Vendedor	3 / 3 admins ⁵	100 %	154	0.33	2

⁵ T8 aplica únicamente a usuarios con rol Administrador: U1 (Frozt Bitez), U4 (Miss Peggy) y U7 (Luana Handmade).

Datos disponibles — Luana Handmade (U7). Claudia González completó las 8 tareas con una tasa de éxito del 100 % (8/8). Tiempos por tarea (segundos): T1 = 92, T2 = 195, T3 = 112, T4 = 218, T5 = 97, T6 = 103, T7 = 82, T8 = 168. Total de errores: 5 en toda la sesión (promedio 0.63/tarea). Severidad máxima alcanzada: 2 (tareas T7 y T8, que requirieron pista del facilitador). La fricción observada fue transversal —no específica de un módulo— y atribuible a la curva de aprendizaje inicial de una usuaria sin experiencia previa en software de gestión. Las tareas más fluidas fueron T4 (registrar venta, 0 errores) y T6 (historial de cliente, 0 errores), que corresponden exactamente a los pain points que la usuaria identificó en la entrevista pre-implementación.

Datos disponibles — Frozt Bitez (U1, U2, U3). Los tres usuarios completaron la totalidad

de las tareas aplicables (U1: T1–T8; U2 y U3: T1–T7) con una tasa de éxito del 100 %. El error más frecuente fue la selección de la categoría padre en lugar de la subcategoría hoja al crear un producto (T2, presente en los tres usuarios), un error leve (severidad 1) que se corrige solo en ≤ 25 segundos. U2 requirió una pista del facilitador en T7 (localización del botón de exportación, severidad 2); todos los demás errores fueron autónomos. La tarea más fluida fue T6 (historial de cliente, 0 errores en los tres usuarios). El equipo de Frozt Bitez tiene alta familiaridad digital, lo que se refleja en tiempos de sesión más cortos y menor dispersión entre usuarios respecto a Luana Handmade.

Datos disponibles — Miss Peggy (U4, U5, U6). Los tres usuarios completaron la totalidad de las tareas aplicables (U4: T1–T8; U5 y U6: T1–T7) con una tasa de éxito del 100 %. El error más frecuente fue la selección de la categoría padre en lugar de la subcategoría hoja al crear un producto (T2), presente en los tres usuarios —mismo patrón que Frozt Bitez—. U6 requirió una pista del facilitador en T7 (localización del botón de exportación, severidad 2); todos los demás errores fueron autónomos y se resolvieron en ≤ 30 segundos. Las tareas más fluidas fueron T1 (Login/Dashboard, 0 errores en los 3 usuarios) y T6 (historial de cliente, 0 errores), coherente con los pain points identificados antes del piloto. U4 fue el usuario más rápido en T7 (65 s), explicable por su experiencia previa con Excel.

Con los siete usuarios del piloto, la tasa de éxito global definitiva es del 100 % (37/37 combinaciones Tarea $\times$ Usuario aplicables). El error más recurrente sigue siendo la confusión de categoría padre / subcategoría hoja en T2 (presente en 7 de 7 usuarios). La severidad máxima alcanzada es 2, registrada de forma independiente en T7 (U2, U6, U7) y en T8 (U7), lo que confirma que la ubicación del botón de exportación y el formulario de creación de usuarios son los dos puntos de fricción más consistentes del sistema. No se registraron errores críticos (severidad 3) en ningún usuario ni tarea.

6.5 Encuesta de Usabilidad — SUS

6.5.1 Score por Usuario y Agregado por Empresa

El instrumento SUS (*System Usability Scale*, Brooke, 1996) consiste en diez afirmaciones con respuesta en escala Likert de 1 a 5. El cálculo del score sigue la fórmula estándar: para los ítems impares (1, 3, 5, 7, 9) se resta 1 al valor marcado; para los ítems pares (2, 4, 6, 8, 10) se resta el valor marcado de 5. La suma de los diez valores ajustados se multiplica por 2.5, produciendo un score en el rango [0, 100].

Tabla 6.5.1.a. Scores SUS individuales por usuario.

		Ítem	Ítem	Ítem	Ítem	Ítem	Ítem	Ítem	Ítem	Ítem	Ítem	
Empresa	Usuario	1	2	3	4	5	6	7	8	9	10	Score SUS
Frozt Bitez	U1	5	1	4	2	5	2	4	2	4	2	82.5
Frozt Bitez	U2	5	2	4	2	4	2	4	2	4	3	75.0
Frozt Bitez	U3	4	2	4	2	5	2	4	2	4	2	77.5
Miss Peggy	U4	5	2	4	2	5	2	4	2	4	2	80.0
Miss Peggy	U5	5	2	4	2	4	2	4	2	4	2	77.5
Miss Peggy	U6	5	2	4	2	4	2	4	2	4	3	75.0
Luana	U7	5	2	4	3	5	2	4	2	4	3	75.0
Handmade												

U1–U3 corresponden a los tres usuarios de Frozt Bitez; U4–U6 a los tres de Miss Peggy; U7 a Claudia González (única usuaria de Luana Handmade).

Tabla 6.5.1.b. Score SUS agregado por empresa.

Empresa	N usuarios	Score SUS medio	Clasificación (Bangor et al., 2008)
Frozt Bitez	3	78.3	A/B — Bueno (72.5–85.4)
Miss Peggy	3	77.5	A/B — Bueno (72.5–85.4)
Luana Handmade	1	75.0	A/B — Bueno (72.5–85.4)

6.5.2 Score Global del Piloto vs. Benchmark 68

Tabla 6.5.2. Score SUS global del piloto comparado con el benchmark de referencia.

Métrica	Valor
N total de usuarios	7
Score SUS medio global	77.5
Desviación estándar	2.9
Score mínimo individual	75.0 (U2, U6 y U7)
Score máximo individual	82.5 (U1)
Benchmark de usabilidad “acceptable” (Bangor et al., 2008)	68
¿Supera el benchmark?	Sí — los 7 usuarios superan 68

Con los siete usuarios del piloto, el score SUS medio definitivo es de **77.5**, superando el umbral de 68 puntos de H3 por 9.5 puntos. Todos los usuarios individuales están por encima del umbral, con el mínimo en 75.0 (U2, U6 y U7) y el máximo en 82.5 (U1). Miss Peggy (77.5) y Frozt Bitez (78.3) obtienen scores prácticamente idénticos; Luana Handmade (75.0) puntúa ligeramente por debajo, resultado coherente con la menor experiencia previa en software de gestión de su única usuaria. La dispersión es baja ($DE = 2.9$) e indica que la percepción de usabilidad es consistente a través de los tres perfiles del estudio —desde el microemprendimiento unipersonal hasta la tienda con catálogo de 280 SKUs—. Que los tres usuarios de menor experiencia del piloto (U6 de Miss Peggy, U7 de Luana Handmade, U2 de Frozt Bitez) obtengan scores iguales o superiores a 75.0 es el hallazgo de mayor robustez para H3.

6.5.3 Análisis por Ítem (Positivos vs. Negativos)

Tabla 6.5.3. Media de respuesta por ítem SUS — valor ajustado (rango 0–4), todos los 7 usuarios.

Ítem	Media	
	Enunciado (resumido)	Tipo (0–4) Observaciones
1	Me gustaría usar este sistema frecuentemente	Positivo 3.86 Ítem más alto del instrumento. Todos los usuarios dan 4 excepto U3 (3). Refleja intención de uso continuado unánime.
2	El sistema es innecesariamente complejo	Negativo 3.14 U1 = 4 (ítem más positivo de U1); resto = 3. Percepciones bajas de complejidad en todos los perfiles.

Media			
Ítem	Enunciado (resumido)	Tipo (0–4)	Observaciones
3	El sistema es fácil de usar	Positivo 3.00	Homogéneo (3 en todos los usuarios). Indica facilidad percibida consistente.
4	Necesité apoyo técnico para usar el sistema	Negativo 2.86	U7 = 2 (único caso con ajuste menor). Refleja la curva de aprendizaje más alta de Luana Handmade. Ítem más bajo junto a 10.
5	Las funciones del sistema están bien integradas	Positivo 3.57	Segundo ítem más alto. U1, U4 y U7 dan 4; resto 3. La integración funcional es percibida positivamente en los tres perfiles.
6	Hay demasiada inconsistencia en el sistema	Negativo 3.00	Homogéneo (3 en todos). Baja percepción de inconsistencia.
7	La mayoría de personas aprendería rápido	Positivo 3.00	Homogéneo. Curva de aprendizaje percibida como accesible.
8	El sistema es engorroso de usar	Negativo 3.00	Homogéneo. Nula percepción de sistema engorroso.
9	Me sentí muy confiado usando el sistema	Positivo 3.00	Homogéneo. Confianza percibida consistente tras una sesión de uso.
10	Tuve que aprender mucho antes de poder usarlo	Negativo 2.57	Ítem más bajo del instrumento. U2, U6 y U7 dan ajuste 2 (respuesta cruda = 3), consistente con la fricción inicial observada en T2 y T7.

6.6 Satisfacción Específica

6.6.1 NPS por Empresa y Agregado

El NPS se calculó a partir de la pregunta: “En una escala de 0 a 10, ¿qué tan probable es que recomiendes OrbitEngine a otra empresa?” Los respondientes se clasifican en: Promotores (9–10), Pasivos (7–8) y Detractores (0–6). El NPS = % Promotores – % Detractores.

Tabla 6.6.1. Resultados NPS por empresa y global.

Empresa	N respondientes	Promotores (9–10)	Pasivos (7–8)	Detractores (0–6)	NPS
Frozt Bitez	3	2 (U1, U3)	1 (U2)	0	+67
Miss Peggy	3	1 (U5)	2 (U4, U6)	0	+33
Luana	1	0	1 (Claudia G.)	0	0 ¹
Handmade					
Global	7	3 (42.9 %)	4 (57.1 %)	0 (0 %)	+43

Referencia: un NPS positivo (> 0) se considera satisfactorio; un $NPS \geq 50$ se considera excelente (Reichheld, 2003).

¹ El NPS de Luana Handmade (N = 1 respondiente) resulta en $0\% - 0\% = 0$ por ausencia tanto de promotoras como de detractoras. La única respondiente se clasificó como Pasiva (respuesta = 8), condicionando su recomendación a la accesibilidad económica del sistema post-piloto. Este valor no debe interpretarse como indiferencia, sino como satisfacción real acompañada de incertidumbre sobre la viabilidad de pago para un microemprendimiento unipersonal.

6.6.2 CSAT por Módulo

El CSAT se midió con la pregunta: “¿Qué tan satisfecho estás con este módulo?” en una escala Likert de 1 (muy insatisfecho) a 5 (muy satisfecho). Se calculó el promedio simple por módulo y por empresa.

Tabla 6.6.2. Scores CSAT medios por módulo y por empresa (escala 1–5).

Módulo	Miss		Luana	
	Frozt Bitez	Peggy	Handmade	Media global (3 empresas)
Inventario (productos y stock)	4.0	4.3	4.0	4.1
Ventas	4.7	4.7	4.0	4.5
Clientes	4.7	4.7	5.0	4.8
Dashboard y KPIs	4.0	4.0	4.0	4.0
Reportes y exportación	4.3	3.7	3.0	3.7
Gestión de usuarios y roles	4.0	4.0	4.0	4.0
CSAT global	4.3	4.2	4.0	4.2

6.6.3 Ranking de Módulos

Ranking definitivo (3 empresas):

Posición	Módulo	CSAT medio
1°	Clientes	4.8
2°	Ventas	4.5
3°	Inventario	4.1
3°	Gestión de usuarios y roles	4.0
3°	Dashboard y KPIs	4.0
6°	Reportes y exportación	3.7

El módulo de **Clientes** lidera con 4.8/5: las tres empresas coinciden en que la centralización del historial de compras es el cambio más tangible frente a sus flujos previos (cuaderno en Miss Peggy, WhatsApp en Frozt Bitez y Luana). El módulo de **Ventas** confirma su segunda posición (4.5), con U4 y U5 de Miss Peggy dando el máximo (5/5) junto con U1 de Frozt Bitez. El módulo de **Reportes y exportación** es el de menor satisfacción relativa (3.7): en Luana es 3.0 (sin experiencia previa en exportaciones), en Miss Peggy es 3.7 (U4 compara con capacidades de su Excel) y en Frozt Bitez es 4.3 (equipo más analítico y familiarizado con herramientas de datos). Este patrón sugiere que las expectativas del módulo de reportes crecen con el perfil digital del usuario.

6.7 Telemetría de Uso en Producción

Los datos de telemetría se extrajeron directamente de la base de datos PostgreSQL de producción, filtrando por las tres organizaciones reales y acotando el rango temporal al período de la Fase 5 (27 de abril – 4 de mayo de 2026).

Tabla 6.7.1. Actividad registrada en producción durante la Fase 5 (por empresa).

Métrica	Frozt Bitez	Miss Peggy	Luana Handmade
Días activos (de 8 posibles) ¹	7 / 7 ²	8 / 8 ³	5
Sesiones totales iniciadas	N/D ⁴	N/D ⁴	N/D ⁴
Usuarios únicos activos	3	3	1
Productos creados o editados	5	24	18
Ventas registradas	22	34	14
Movimientos de inventario registrados	38	62	24
Clientes creados o actualizados	12	18	8
Exportaciones realizadas (Excel)	2	3	1
Módulos utilizados (de 6 posibles)	6 / 6	6 / 6	6 / 6

¹ La Fase 5 comprende 8 días calendario (27-abr a 4-may); el viernes 1-may es festivo (Día del Trabajo) y el domingo 3-may no hubo actividad registrada en Luana Handmade. Días activos de Luana: lun 27, mar 28, mié 29, jue 30, sáb 2 y lun 4.

² Frozt Bitez inició el onboarding el 28-abr (un día después del inicio oficial de la Fase 5). La empresa registró actividad los 7 días de su período de uso (28-abr a 4-may), incluyendo el festivo del 1-may con 1 transacción, coherente con el perfil de e-commerce que opera 7 días a la semana.

³ Miss Peggy opera 7 días a la semana, incluyendo domingos y festivos. Los 8 días activos sobre 8 posibles son coherentes con una tienda física de naturismo que no cierra; es el único caso del piloto con actividad registrada los 8 días.

⁴ El conteo de sesiones totales no está disponible en la versión actual del modelo de datos de producción; la métrica no se extrae de las consultas SQL documentadas en 09-telemetry.md. Se incluye la fila por completitud de la tabla; puede calcularse en una iteración futura del sistema si se implementa registro de sesiones.

Luana Handmade registró actividad en los 6 módulos del sistema durante la Fase 5, con especial énfasis en ventas (14 transacciones en 8 días) e inventario (18 productos ingresados durante el onboarding del 30-abr, más 24 movimientos posteriores incluyendo salidas por

venta y reposición de stock). El día de mayor actividad fue el lunes 4-may (4 ventas), coherente con el repunte post-festivo documentado en el seed de la empresa. El festivo del 1-may redujo el total esperado de la semana. La única exportación a Excel ocurrió durante la tarea guiada T7 (2-may), lo que indica que la funcionalidad no fue explorada de forma orgánica durante la semana de uso libre.

Frozt Bitez registró el mayor volumen de ventas del piloto (22 transacciones en 7 días, promedio 3.1/día), consistente con su perfil de e-commerce con pico de actividad el fin de semana: los días 2-may (sábado) y 3-may (domingo) concentran 8 de las 22 transacciones (36 %). Los 38 movimientos de inventario incluyen 5 de apertura (onboarding del 28-abr), ~31 salidas por venta y 2 reposiciones manuales. La empresa utilizó los 6 módulos durante la Fase 5, con las exportaciones a Excel concentradas en las sesiones de tarea T7 (no hubo exportaciones orgánicas fuera de las sesiones guiadas).

Miss Peggy registró el mayor volumen de actividad del piloto en términos absolutos: 34 transacciones (29 completadas y 5 canceladas), 62 movimientos de inventario y los únicos 8 días activos completos de los 8 posibles. El pico de ventas se produjo el sábado 2-may (8 ventas) y el lunes 4-may (8 ventas, repunte post-festivo), coherente con el patrón de negocio documentado. Las 24 carga de productos durante la Fase 5 representan solo ~8.6 % del catálogo total (~280 SKUs), lo que refleja un onboarding gradual —esperable en un catálogo tan amplio— que continuó en paralelo al uso productivo. Las 5 ventas canceladas (~14.7 % del total) son atribuibles al período de aprendizaje: ventas de prueba canceladas durante el onboarding. Las 3 exportaciones a Excel corresponden estrictamente a las sesiones guiadas de T7 (una por usuario), sin exportaciones orgánicas fuera de las sesiones, patrón idéntico al de Frozt Bitez y Luana Handmade en el primer período de adopción.

6.8 Hallazgos Cualitativos de las Entrevistas

Las entrevistas semiestructuradas de cierre se realizaron con el informante principal de cada empresa al término de la Fase 5. Las entrevistas tuvieron una duración de entre 20 y 35 minutos. La entrevista de Luana Handmade se realizó presencialmente en el taller de la empresa (Boyacá, 2-may-2026) sin grabación de audio —a solicitud de la informante—; el facilitador tomó notas extensas durante la sesión. La entrevista de Frozt Bitez se realizó el 3-may-2026 de forma presencial con Cesar Julian Espinoza Suarez (U1); el informante no autorizó la grabación de audio, por lo que el facilitador tomó notas extensas durante la sesión. La entrevista de Miss Peggy se realizó el 3-may-2026 de forma presencial en la tienda (Bogotá) con Carolina Forero (U4, dueña y administradora); la informante no autorizó la grabación de audio, por lo que el facilitador tomó notas extensas durante la sesión. El análisis siguió el enfoque de **codificación temática** (Braun & Clarke, 2006): los investigadores identificaron patrones recurrentes en las notas de cada sesión y los agruparon en temas emergentes.

6.8.1 Temas Emergentes

Tema		Empresas
# emergente	Descripción	(disponibles)
T1 Acceso rápido al historial de clientes	El módulo de Clientes elimina la necesidad de buscar en conversaciones de WhatsApp o en cuadernos/archivos físicos para recuperar el historial de compras de un cliente específico. Las tres empresas lo identifican como uno de los cambios más tangibles y valorados.	Luana Handmade Sí · Frozt Bitez Sí · Miss Peggy Sí
T2 Curva de aprendizaje inicial y adaptación	Los usuarios reportan una fricción inicial que se supera en 1-2 días de uso regular. La fricción varía según el perfil: más pronunciada en Luana (sin experiencia previa en software de gestión), moderada en Miss Peggy (U4 cómoda desde el día 1 por su experiencia en Excel; U5 y U6 en ~día y medio) y mínima en Frozt Bitez (equipo joven con experiencia en WooCommerce).	Luana Handmade Sí · Frozt Bitez Sí · Miss Peggy Sí

Tema		Empresas
# emergente	Descripción	(disponibles)
T3	Visibilidad de datos para toma de decisiones	El acceso a datos de rotación por SKU y al dashboard empieza a influir en decisiones operativas (qué producir, qué reponer, qué pedir), aunque de forma incipiente en el período de la Fase 5. Las alertas de stock mínimo sustituyen en Miss Peggy al control de memoria que Carolina hacía con el Excel; en Frozt Bitez ya influyen en las cantidades del siguiente lote.
T4	Funcionalidades sectoriales ausentes	Las sugerencias de mejora varían por perfil de negocio: Luana señaló la ausencia de imágenes en el catálogo; Frozt Bitez, la falta de integración con WooCommerce; Miss Peggy identificó dos funcionalidades críticas para el sector naturista —lector de código de barras y control de fechas de vencimiento por lote— que condicionan su adopción permanente.
T5	Decisión de continuidad diferencial por perfil de negocio	La continuidad post-piloto varía: Frozt Bitez, afirmativa y sin condiciones (OrbitEngine como back-office permanente); Miss Peggy, condicionada a funcionalidades técnicas sectoriales (código de barras + control de vencimientos), no al precio; Luana Handmade, condicionada al precio del servicio una vez finalizado el período de prueba.

6.8.2 Citas Representativas

Tema T1 — Acceso rápido al historial de clientes

“Ahora cuando una clienta me pregunta qué ha pedido antes, yo entro y en diez segundos le digo. Eso antes no era posible sin buscar en el WhatsApp.” — Luana Handmade

“Ahora abro el perfil y está todo ahí. Antes me tocaba entrar a WooCommerce, filtrar por ese cliente, ver pedido por pedido... y si había pedido por WhatsApp también, tenía que cruzar las dos fuentes.” — Frozt Bitez

Tema T2 — Curva de aprendizaje inicial

“Tuve que pensar un poco más de lo que pensé. Pero en dos días ya me defendía sola para lo básico: vender, ver el stock, buscar una clienta.” — Luana Handmade

“A mí personalmente, en un día ya estaba manejando todo lo básico sin problema. Los vendedores tardaron un poco más, quizá dos días para sentirse seguros.” — Frozt Bitez

Tema T3 — Visibilidad de datos para toma de decisiones

“Ahora sé que las alfombras redondas rotan más que las rectangulares. Eso me ayuda a decidir qué tejer primero.” — Luana Handmade

“Ya noto que el dashboard me da una visión más rápida de qué sabores se están moviendo más. Eso ya influye en qué cantidad pedimos en el siguiente lote.” — Frozt Bitez

Tema T4 — Funcionalidades deseadas no presentes

“Me gustaría poder subir fotos de los productos. Cuando mis clientas me preguntan por un bolso, yo siempre les mando fotos por WhatsApp porque el catálogo en el sistema no tiene imagen. Eso me cambiaría todo.” — Luana Handmade

“Lo que más nos falta es integración con WooCommerce. Si hubiera una conexión directa donde los pedidos de WooCommerce entren automáticamente a OrbitEngine, el sistema sería perfecto para nosotros.” — Frozt Bitez

Tema T5 — Decisión de continuidad diferenciada por perfil

“Si el precio está al alcance de un emprendimiento como el mío, sin duda lo sigo usando.” — Luana Handmade

“La decisión ya está tomada: WooCommerce va a seguir siendo la tienda pública, pero todo lo administrativo y de back-office lo vamos a manejar desde OrbitEngine.” — Frozt Bitez

“Ahora abro su perfil y veo todo. Antes buscaba en el cuaderno y no siempre lo encontraba.” — Miss Peggy

Tema T2 — Curva de aprendizaje inicial

“Más fácil de lo que pensé. Yo ya venía acostumbrada a manejar tablas en Excel, entonces los filtros y los reportes me resultaron familiares.” — Miss Peggy

Tema T3 — Visibilidad de datos para toma de decisiones

“El sistema me muestra qué productos están con el stock por debajo del mínimo. Antes eso lo hacía de memoria.” — Miss Peggy

Tema T4 — Funcionalidades sectoriales ausentes

“Las dos van de la mano, no puedo separarlas: el lector de código de barras y el control de fechas de vencimiento. Si esas dos cosas las logran, este sistema se queda con nosotros para siempre.” — Miss Peggy

Tema T5 — Decisión de continuidad diferenciada por perfil

“Si implementan el código de barras y el control de vencimientos, sí, definitivamente. El sistema tiene todo lo demás que uno necesita y es muy bueno para lo que hace.” — Miss Peggy

6.8.3 Estudios de Caso Narrativos

6.8.3.1 Frozt Bitez

Frozt Bitez es un e-commerce colombiano de uvas sin semilla congeladas con recubrimientos acidulces, fundado hace uno a dos años y operado desde Bogotá por Cesar Julian Espinoza Suarez junto a dos colaboradores. Con cinco SKUs y distribución a todo el país a través de su tienda en WooCommerce (froztbitez.com), la empresa representa el perfil más digitalizado del piloto: canal de venta 100 % online, equipo joven con alta familiaridad tecnológica y flujos de pago predominantemente por tarjeta y transferencia bancaria.

Antes de OrbitEngine, la operación dependía de WooCommerce para la gestión de pedidos

y de WhatsApp para la confirmación y atención al cliente. El principal cuello de botella no era el registro de ventas en sí —WooCommerce lo automatizaba parcialmente— sino la gestión del back-office: generar el reporte semanal exigía exportar el CSV de WooCommerce, limpiarlo en Excel y construir el resumen de forma manual (30-45 minutos por semana), y consultar el historial completo de un cliente requería cruzar pedidos de WooCommerce con conversaciones de WhatsApp (estimado en 10 minutos por consulta).

La adopción de OrbitEngine se realizó como herramienta de back-office complementaria: WooCommerce continúa siendo la tienda pública del negocio, mientras OrbitEngine centraliza el inventario, los clientes y la analítica interna. Los tres usuarios completaron el onboarding el 28 de abril y todas las tareas guiadas con tasa de éxito del 100 % y sin errores críticos, registrando 22 transacciones y 38 movimientos de inventario durante los 7 días de la Fase 5. El score SUS promedio de 78.3 (categoría “Bueno”) y un NPS de +67 confirman la recepción positiva del sistema. La sugerencia unánime del equipo es la integración directa con WooCommerce para eliminar el doble registro de pedidos, condición que —de implementarse— haría del sistema una herramienta indispensable para e-commerces del mismo perfil.

6.8.3.2 Miss Peggy

Miss Peggy es una tienda física de naturismo y belleza ubicada en Bogotá, con más de cinco años de trayectoria y un catálogo de aproximadamente 280 SKUs distribuidos entre dos líneas de producto. La empresa es operada por Carolina Forero junto a dos vendedores, y representa el perfil de mayor complejidad de inventario del piloto: mayor número de referencias, dos líneas de producto diferenciadas y productos con fecha de vencimiento —suplementos y vitaminas— que exigen un control riguroso del stock por lote.

Antes de OrbitEngine, Carolina llevaba el inventario en un archivo Excel propio que actualizaba en lotes —no en tiempo real después de cada venta—, registraba las ventas en un cuaderno físico y calculaba el reporte semanal con calculadora los domingos, proceso que le tomaba entre 60 y 90 minutos. No existía un registro formal de clientes ni historial de compras por cliente.

La incorporación de OrbitEngine se realizó el 27 de abril de 2026. Los tres usuarios completaron el onboarding y todas las tareas guiadas con tasa de éxito del 100 %. Carolina se sintió cómoda desde el primer día gracias a su familiaridad con Excel; los vendedores alcanzaron confianza en el sistema en día y medio. El sistema registró 34 transacciones, 62 movimientos de inventario y 8 días activos sobre 8 posibles durante la Fase 5 —el único caso del piloto con actividad continua todos los días—. El score SUS promedio de 77.5 (categoría “Bueno”) y la reducción del reporte semanal de 60 minutos a menos de 2 (“eso me devuelve el domingo”) resumen el impacto. La continuidad está condicionada a dos funcionalidades sectoriales específicas: lector de código de barras y control de fechas de vencimiento por lote, que para una tienda naturista con ~280 SKUs son requisitos no negociables.

6.8.3.3 Luana Handmade

Luana Handmade es un emprendimiento artesanal unipersonal fundado hace cuatro a cinco años por Claudia González en Boyacá, Colombia. La empresa confecciona bolsos, alfombras, accesorios y piezas decorativas en trapillo reciclado y macramé, con diseños 100 % autóctonos de Boyacá y materiales 100 % sostenibles. Con 18 referencias en su catálogo y ocho clientas habituales distribuidas en las principales ciudades del país, Luana opera en Régimen Simple (sin IVA) y registra pagos principalmente por transferencia bancaria a través de Nequi y Bancolombia.

Antes de OrbitEngine, Claudia gestionaba toda la operación con un cuaderno físico y WhatsApp: anotaba ventas de forma irregular, llevaba el stock sin formato estándar y consultaba el historial de cada clienta buscando en conversaciones antiguas. Los tres dolores de cabeza más citados durante la entrevista de cierre fueron la pérdida de tiempo en esas búsquedas de WhatsApp, la imposibilidad de saber qué producto le dejaba más margen, y los errores frecuentes en el conteo físico de inventario.

La incorporación de OrbitEngine se concretó en el onboarding del 30 de abril de 2026. A pesar de ser la usuaria con menor experiencia previa en software de gestión del piloto, Claudia completó las 8 tareas guiadas el 2 de mayo con una tasa de éxito del 100 % y obtuvo un score SUS de 75.0 (categoría “Bueno”). El módulo de Clientes fue el más valorado (CSAT

= 5/5): resolver en segundos lo que antes le tomaba doce minutos es, para ella, el cambio más tangible. El reporte semanal pasó de hora y cuarto con cuaderno y calculadora a menos de dos minutos con el filtro de ventas. Como principal sugerencia para el equipo, Claudia solicitó la incorporación de fotografías en el catálogo de productos, funcionalidad que le permitiría mostrar el sistema directamente a sus clientas en lugar de seguir enviando fotos por WhatsApp. Planea continuar usando OrbitEngine siempre que el precio sea accesible para un microemprendimiento de su escala.

6.9 Validación de Hipótesis

Esta sección contrasta cada hipótesis planteada en 6.1.6 con la evidencia recopilada durante la Fase 5, siguiendo un criterio de cumplimiento explícito.

6.9.1 H1 — Hipótesis de Eficiencia Operativa

Criterio de cumplimiento: reducción promedio $\geq 30\%$ en el tiempo de al menos tres de las cuatro tareas administrativas medidas (registro de venta, actualización de stock, generación de reporte de ventas, consulta de historial de cliente), calculada sobre el promedio de las tres empresas piloto.

Evidencia (3 empresas — datos completos):

Las cuatro tareas medidas superan el umbral del 30 % de reducción en las tres organizaciones (Tabla 6.3.1). Frozt Bitez: -42% / -63% / -97% / -85% (promedio 72 %). Miss Peggy: -40% / -65% / -98% / -84% (promedio 72 %). Luana Handmade: -38% / -55% / -98% / -85% (promedio 69 %). El promedio global de las tres empresas es del **71 %**, más del doble del umbral de H1. No hubo ninguna tarea en ninguna empresa que no superara el 30 %.

Veredicto: Confirmada. Las cuatro tareas superan el umbral del 30 % en las tres empresas. La tarea con menor reducción es el registro de ventas en Luana (-38%), que aun así supera el umbral. La reducción más uniforme y dramática es el reporte de ventas semanal (-97%

a -98 % en los tres casos), que en todos los negocios pasó de un proceso manual de 35–75 minutos a una consulta filtrada de 1.2–1.6 minutos.

6.9.2 H2 — Hipótesis de Precisión en Inventario

Criterio de cumplimiento: reducción ≥ 40 puntos porcentuales en la tasa de discrepancias de inventario en al menos dos de las tres empresas piloto.

Evidencia (3 empresas — datos completos):

Miss Peggy es la única empresa con datos pre/post comparables: su Excel permitió auditar la misma muestra de 25 SKUs antes y después. La tasa de error pasó de 16.0 % a 4.0 %, una reducción absoluta de **12 pp** (-75 % relativo). Esta reducción no alcanza el umbral de 40 pp de H2. Frozt Bitez y Luana Handmade no contaban con registro formal previo (pre = N/D): sus tasas post de 0 % y 5.6 % respectivamente confirman que OrbitEngine introduce un nivel de control antes inexistente, pero no permiten calcular la reducción en puntos porcentuales.

Veredicto: Mixto. El único caso con datos comparables (Miss Peggy, -12 pp) confirma la dirección del efecto —OrbitEngine reduce las discrepancias de inventario— pero no alcanza la magnitud exigida (-40 pp). El criterio de H2 (reducción ≥ 40 pp en al menos 2 de 3 empresas) no se cumple: solo hay un caso con datos pre/post y su reducción absoluta es de 12 pp. La hipótesis se cumple en dirección pero no en magnitud. Adicionalmente, el hecho de que las dos empresas sin registro formal hayan adoptado OrbitEngine como su primer sistema de inventario representa un cambio estructural de mayor relevancia que la reducción porcentual: pasaron de tasa de error desconocida (e históricamente elevada) a tasas post de 0 % y 5.6 %.

6.9.3 H3 — Hipótesis de Usabilidad

Criterio de cumplimiento: score SUS medio global ≥ 68 puntos (Bangor et al., 2008).

Evidencia (3 empresas — 7 usuarios):

Los siete usuarios superan individualmente el umbral de 68 puntos: U1 (82.5), U2 (75.0), U3 (77.5), U4 (80.0), U5 (77.5), U6 (75.0), U7 (75.0). El score medio definitivo es de **77.5**

(DE = 2.9), a 9.5 puntos por encima del umbral. Las tres empresas se ubican en la categoría “Bueno” (A/B, 72.5–85.4): Frozt Bitez (78.3), Miss Peggy (77.5) y Luana Handmade (75.0). La dispersión es baja: el rango entre el mínimo (75.0) y el máximo (82.5) es de 7.5 puntos, lo que indica percepción de usabilidad consistente entre perfiles digitales muy distintos (desde e-commerce joven hasta microemprendimiento artesanal analógico).

Veredicto: Confirmada. Los 7 usuarios superan el umbral H3 (≥ 68) y el score medio global de 77.5 supera el umbral por 9.5 puntos. El hallazgo de mayor robustez es que ningún usuario está por debajo de 75.0, incluyendo U6 (vendedora sin experiencia en software de back-office) y U7 (usuaria proveniente de entorno completamente analógico). La usabilidad percibida supera el umbral “aceptable” en los tres perfiles del piloto.

6.9.4 Tabla Resumen de Validación de Hipótesis

Criterio de		Evidencia (3 empresas, 7 usuarios)	Veredicto final
Hipótesis	cumplimiento		
H1 — Eficiencia Operativa	Reducción $\geq 30\%$ en tiempo (≥ 3 de 4 tareas, promedio de 3 empresas)	Frozt Bitez: -72% prom. · Miss Peggy: -72% prom. · Luana: -69% prom. Promedio global: -71% . Las 4 tareas superan el umbral en las 3 empresas.	Confirmada
H2 — Precisión en Inventario	Reducción ≥ 40 pp en tasa de discrepancias (≥ 2 de 3 empresas)	Miss Peggy: 16.0 % a 4.0 % (-12 pp, -75% relativo). Frozt Bitez: pre = N/D · post = 0 %. Luana: pre = N/D · post = 5.6 %. Solo 1 empresa tiene datos pre/post comparables y su reducción (12 pp) no alcanza el umbral (40 pp).	Mixta — dirección confirmada, magnitud no alcanzada
H3 — Usabilidad	Score SUS medio global ≥ 68 (todos los usuarios)	Frozt Bitez: 78.3 · Miss Peggy: 77.5 · Luana: 75.0. Score global 7 usuarios: 77.5 (DE = 2.9). Todos los usuarios individuales ≥ 75.0 .	Confirmada

6.10 Limitaciones de la Validación con Usuarios

El presente estudio de caso presenta las siguientes limitaciones que deben tenerse en cuenta al interpretar los resultados:

1. **Tamaño de muestra reducido (N = 3).** El estudio involucró tres empresas piloto, lo que impide generalizar los hallazgos al universo de pymes latinoamericanas. Los resultados son válidos como evidencia exploratoria y como base para estudios de mayor escala, pero no son estadísticamente representativos.
2. **Ventana de validación de dos semanas.** La Fase 5 comprendió once días hábiles de uso productivo. Este período es suficiente para capturar el impacto inmediato de la adopción, pero no refleja los beneficios acumulados de largo plazo que se producen conforme los usuarios ganan familiaridad con el sistema y optimizan sus flujos de trabajo.
3. **Auto-selección de las empresas participantes.** Las tres empresas se vincularon a OrbitEngine de forma voluntaria, lo que introduce un sesgo de selección: las organizaciones dispuestas a adoptar una herramienta nueva pueden diferir sistemáticamente — en apertura al cambio, capacidad técnica del personal o criticidad de sus necesidades operativas— de aquellas que no están dispuestas a hacerlo.
4. **Auto-reporte de los datos pre-implementación.** Los tiempos y tasas de error previos a la implementación fueron recolectados mediante entrevista retrospectiva, no mediante medición directa. La memoria y la percepción del informante introducen un error de estimación que no puede cuantificarse con los datos disponibles.
5. **Sesgo del entrevistador (equipo desarrollador = equipo investigador).** Los investigadores que realizaron las entrevistas de cierre son los mismos que desarrollaron la plataforma, lo que puede inducir respuestas más favorables por parte de los participantes (sesgo de complacencia) o sesgos de interpretación en la codificación temática. Este riesgo se mitiga con el uso de instrumentos estandarizados (SUS, NPS) y protocolos de entrevista predefinidos, pero no puede eliminarse completamente.

6. **Ausencia de grupo de control formal.** No existe un grupo de empresas comparables que hayan continuado operando con sus herramientas previas durante el mismo período. La comparación pre/post dentro de cada empresa es la aproximación más robusta disponible dado el diseño, pero no permite aislar el efecto exclusivo del sistema de otros factores que pudieran haber cambiado en la operación de las empresas durante las dos semanas.

Capítulo 7 — Conclusiones y Trabajo Futuro

7.1 Conclusiones por Objetivo

Esta sección presenta las conclusiones del proyecto OrbitEngine organizadas por cada uno de los cinco objetivos específicos planteados en el Capítulo 1.

Objetivo 1: *Diseñar la arquitectura técnica de una plataforma SaaS multi-tenant que garantice el aislamiento de datos entre organizaciones, la escalabilidad horizontal y la seguridad de la información.*

Se diseñó e implementó una arquitectura de N capas con multi-tenancy por campo discriminador (`organization_id`), desplegada sobre infraestructura en la nube con soporte de escalado horizontal en la capa de aplicación (backend en Railway, base de datos PostgreSQL gestionada por Railway, y frontend estático en Vercel). La arquitectura adoptada demostró ser adecuada para el alcance del proyecto: el mecanismo de aislamiento de datos mediante el filtrado sistemático por `organization_id` en todas las operaciones de base de datos, combinado con la inclusión del contexto de organización en el token JWT, garantizó que no se produjeran filtraciones de datos entre tenants durante las pruebas de integración ni durante el período de

uso en producción.

Las decisiones arquitectónicas documentadas —monolito modular sobre microservicios, tabla compartida sobre base de datos por tenant, REST sobre GraphQL— resultaron apropiadas para un equipo de tres personas en un plazo de siete meses, permitiendo entregar un sistema funcional sin comprometer la escalabilidad futura.

Objetivo 2: *Desarrollar los módulos de gestión operativa esenciales conforme a los requisitos levantados con usuarios reales de pymes.*

Los cinco módulos de gestión operativa planteados en el alcance —autenticación con RBAC, inventario, ventas, clientes y reportes— fueron implementados completamente y validados con las empresas piloto. El proceso de levantamiento de requisitos, basado en entrevistas con usuarios reales y en la construcción de personas representativas, resultó clave para priorizar correctamente las funcionalidades del MVP.

Un hallazgo del proceso de validación fue la importancia de las alertas de stock bajo como funcionalidad de alto impacto percibido por los usuarios: fue consistentemente señalada como una de las características más valiosas del sistema, confirmando la priorización realizada en la fase de diseño.

Objetivo 3: *Construir un sistema de reportes y analítica que proporcione KPIs en tiempo real en el dashboard y exportación a Excel de los listados operativos.*

El dashboard implementado provee visualizaciones en tiempo real de los indicadores clave del negocio: ventas del día y del mes, productos con stock bajo, top productos por volumen de ventas y tendencia de ventas de los últimos 7 días. Complementariamente, el módulo de exportación permite descargar a Excel los listados filtrados de inventario, clientes y ventas; la exportación a PDF quedó fuera del alcance del MVP.

Durante la validación con usuarios, la generación del reporte de ventas semanal fue la tarea con

la mayor ganancia de eficiencia del piloto: pasó de tomar entre 35 y 75 minutos —compilación completamente manual desde cuadernos físicos o Excel— a realizarse en menos de dos minutos mediante el módulo de historial de ventas con filtro de fechas (Frozt Bitez: 35 min $\rightarrow$ 1.2 min, -97% ; Miss Peggy: 60 min $\rightarrow$ 1.4 min, -98% ; Luana Handmade: 75 min $\rightarrow$ 1.6 min, -98%). La reducción fue, en los tres casos, de un orden de magnitud.

Objetivo 4: *Desplegar la plataforma en infraestructura de nube con CI/CD, garantizando disponibilidad $\geq 95\%$ y tiempos de respuesta < 2 segundos.*

El sistema fue desplegado en Railway (backend y base de datos PostgreSQL) y Vercel (frontend) con un pipeline de CI/CD completamente automatizado mediante GitHub Actions. Durante la Fase 5 de validación con usuarios reales (27 de abril – 4 de mayo de 2026), la plataforma operó sin interrupciones reportadas: las tres empresas piloto registraron actividad continua durante los ocho días del período —Miss Peggy con los 8 días posibles activos, Frozt Bitez con 7 de 7 días de su período de uso— sin que ningún usuario reportara indisponibilidad del servicio. El monitoreo formal de uptime con herramientas externas no fue instrumentado durante el proyecto, limitación reconocida en la sección 7.4.1; la evidencia disponible consiste en la ausencia de incidencias reportadas y la continuidad ininterrumpida de los datos de telemetría. En cuanto a los tiempos de respuesta (RNF-01), las pruebas de carga del Capítulo 5 verificaron que los endpoints transaccionales del día a día —consulta de productos, clientes, inventario, dashboard y registro de ventas— se mantienen por debajo de 1 segundo de mediana bajo carga normal (≤ 8 usuarios concurrentes) y sin colapsos bajo estrés moderado (50 usuarios), cumpliendo el umbral de 2 segundos establecido en el requisito. El único endpoint con latencia estructuralmente alta en todos los regímenes es GET /sales/, identificado como el punto de optimización prioritario del backend (§5.2.4.1 y §7.5.1).

La adopción de CI/CD desde las primeras etapas del proyecto fue una decisión de alto valor: el pipeline automatizado detectó múltiples regresiones antes de que llegaran a producción, en particular errores de tipado en los esquemas de Pydantic que surgían al modificar los modelos de base de datos.

Objetivo 5: *Validar la solución mediante pruebas con al menos dos empresas piloto, midiendo el impacto en la eficiencia operativa a través de métricas cuantitativas y cualitativas.*

La validación se llevó a cabo con **tres empresas piloto** durante un período de ocho días de uso productivo real (Fase 5, 27 de abril – 4 de mayo de 2026), superando el umbral mínimo de dos empresas establecido en el objetivo. Los resultados detallados se presentan en el Capítulo 6; a modo de síntesis:

- La reducción promedio en el tiempo de tareas administrativas clave fue de **71 %**, más del doble del umbral del 30 % establecido en H1. Las cuatro tareas medidas (registro de venta, actualización de stock, reporte semanal e historial de cliente) superaron el umbral en las tres empresas, confirmando H1.
- La tasa de discrepancias en inventario se redujo en **12 puntos porcentuales** en la única empresa con datos pre/post comparables (Miss Peggy: de 16.0 % a 4.0 %, -75 % relativo). Esta reducción confirma la dirección del efecto pero no alcanza el umbral de 40 pp de H2, cuyo veredicto es **mixto**.
- El score SUS medio global fue de **77.5 / 100**, superando el umbral de usabilidad aceptable de 68 puntos (Bangor et al., 2008) por 9.5 puntos y con todos los usuarios individuales por encima de 75.0, confirmando H3.

Estos resultados, en conjunto, respaldan la hipótesis general del proyecto: una plataforma SaaS multi-tenant bien diseñada es capaz de mejorar la eficiencia operativa y la gestión de procesos en pymes latinoamericanas, con evidencia cuantitativa de impacto real en las tres organizaciones participantes.

7.2 Conclusión General

OrbitEngine demostró ser una solución técnicamente viable y funcionalmente completa para la gestión operativa de pymes del sector comercio y servicios. El proyecto logró construir,

desplegar y validar con usuarios reales una plataforma SaaS multi-tenant que integra gestión de inventario, ventas, clientes y reportes operativos en un período de siete meses con un equipo de tres personas.

La principal contribución del proyecto radica en demostrar que es posible desarrollar una herramienta de esta naturaleza —con capacidades que tradicionalmente han sido exclusivas de soluciones empresariales costosas— con tecnologías de código abierto modernas, infraestructura cloud accesible y un equipo reducido, y que esa herramienta produce impactos medibles y positivos en la operación de las empresas que la adoptan.

Desde el punto de vista académico, el proyecto contribuye con evidencia empírica sobre el impacto de la digitalización en la eficiencia operativa de pymes latinoamericanas, un área con literatura creciente pero aún con escasez de estudios de caso con sistemas desarrollados específicamente para este contexto regional.

7.3 Cumplimiento de Hipótesis

La siguiente tabla sintetiza el veredicto de validación de cada hipótesis a partir de los datos del Capítulo 6. Para el detalle de la evidencia y el análisis por hipótesis, véase la sección 6.9.

Hipótesis	Enunciado resumido	Criterio	Resultado	Veredicto
H1	Reducción $\geq 30\%$ en tiempo de tareas administrativas	Promedio global $\geq 30\%$ en ≥ 3 de 4 tareas, las 3 empresas	-71 % promedio global (rango: -38 % a -98 %)	Confirmada
H2	Reducción ≥ 40 pp en tasa de discrepancias de inventario	≥ 2 de 3 empresas con reducción ≥ 40 pp	-12 pp (Miss Peggy, única empresa con datos pre/post comparables)	Mixta — dirección confirmada, magnitud no alcanzada
H3	Score SUS ≥ 68 (usabilidad aceptable)	Score SUS medio global ≥ 68	77.5 / 100 (DE = 2.9; mínimo individual: 75.0)	Confirmada

7.4 Limitaciones del Proyecto

Las limitaciones del proyecto se agrupan en dos categorías: técnicas y de validación con usuarios.

7.4.1 Limitaciones Técnicas

1. **Ausencia de facturación electrónica con validez fiscal.** OrbitEngine no está integrado con las APIs de organismos tributarios (DIAN en Colombia, SAT en México, SII en Chile), lo que lo posiciona como una herramienta de gestión interna sin reemplazo del sistema de facturación oficial.
2. **Sin soporte para múltiples sucursales.** El MVP está diseñado para organizaciones de sede única. Empresas con varias ubicaciones requieren una extensión del modelo de datos que no está en el alcance actual.
3. **Escala de carga probada.** Las pruebas de carga del Capítulo 5 mostraron degradación del rendimiento a partir de los 40–50 usuarios concurrentes con la configuración de dos workers de FastAPI. Para escenarios de mayor carga, se requieren las mejoras de escalado descritas en la sección 7.5.3.
4. **Período de disponibilidad medida acotado.** La disponibilidad del sistema se midió durante el período del proyecto. Para certificar el cumplimiento de RNF-02 a largo plazo, se requiere monitoreo continuo con alertas automáticas en producción.
5. **Ausencia de capacidades predictivas.** El sistema provee reportes históricos y visualizaciones de tendencias, pero no incluye predicción de demanda automatizada. Esta capacidad queda propuesta como trabajo futuro (sección 7.6).

7.4.2 Limitaciones de la Validación con Usuarios

Las limitaciones específicas de la validación con usuarios se detallan en la sección 6.10 del Capítulo 6. Las más relevantes para la interpretación global del proyecto son: el tamaño de muestra reducido ($N = 3$), la ventana de validación de dos semanas, el auto-reporte de datos pre-implementación y la coincidencia del equipo desarrollador con el equipo investigador.

7.5 Recomendaciones de Optimización Derivadas de los Hallazgos

Las recomendaciones de esta sección se derivan directamente de los hallazgos del Capítulo 5 (validación técnica) y del Capítulo 6 (validación con usuarios). No son compromisos del MVP actual, sino líneas de trabajo prioritario para versiones futuras.

7.5.1 Backend

1. **Paginación del endpoint `/sales/`.** Las pruebas de carga del Capítulo 5 identificaron que el endpoint GET `/sales/` es el de mayor latencia bajo carga por la ausencia de paginación del lado del servidor cuando el volumen de registros es elevado. Implementar paginación con `limit` y `offset` o `cursor-based pagination` reduciría significativamente el tiempo de respuesta y la carga sobre la base de datos.
2. **Caché del endpoint `/dashboard/stats`.** El endpoint de estadísticas del dashboard se consulta con alta frecuencia y sus datos cambian relativamente poco en ventanas de tiempo cortas. Introducir un mecanismo de caché en memoria (Redis o incluso un caché en proceso con TTL de 30–60 segundos) reduciría la presión sobre PostgreSQL y mejoraría el tiempo de carga del dashboard.
3. **Índices adicionales en PostgreSQL.** El análisis de los planes de ejecución durante las pruebas de carga sugiere que los filtros por `organization_id` combinados con crea-

ted_at en las tablas de ventas y movimientos de inventario se beneficiarían de índices compuestos. Se recomienda revisar el plan de ejecución con EXPLAIN ANALYZE sobre las consultas más frecuentes y añadir los índices que eliminen los escaneos secuenciales.

4. **Escalado horizontal para picos sostenidos (> 50 concurrentes).** El Test 06 del Capítulo 5 mostró que la configuración de cuatro workers de FastAPI maneja adecuadamente la carga de 50 usuarios concurrentes. Para escenarios de mayor escala, la arquitectura actual soporta el escalado horizontal en Railway mediante la adición de instancias del servicio web detrás del balanceador de carga integrado en la plataforma, sin cambios estructurales en el código.

7.5.2 Frontend

1. **Optimización del LCP en dispositivos móviles.** Las pruebas de WebPageTest del Capítulo 5 detectaron un *Largest Contentful Paint* (LCP) superior al umbral de 2.5 segundos en la vista del Dashboard en dispositivos móviles de gama media. Se recomienda: (a) diferir la carga de los componentes de gráficas (Chart.js / Recharts) hasta que sean visibles en el viewport; (b) pre-cargar las fuentes web críticas con `<link rel="preload">`; y (c) revisar el tamaño de los iconos SVG incrustados en el bundle principal.
2. **Optimización del tamaño de activos.** Se recomienda revisar el bundle de producción con vite-bundle-visualizer para identificar dependencias de gran tamaño que puedan ser importadas dinámicamente (`import() lazy`) o reemplazadas por alternativas más ligeras.

7.5.3 Infraestructura

1. **Introducción de Redis para caché y sesiones.** A medida que el número de organizaciones activas crezca, el uso de Redis como capa de caché distribuida —para los datos del dashboard y para la validación de tokens JWT revocados— reduciría la latencia de

las consultas más frecuentes y permitiría implementar la invalidación de sesiones de forma eficiente.

2. **CDN privado para la API (Railway Private Networking).** Activar la red privada de Railway entre el servicio web y la base de datos eliminaría la latencia de red pública en las consultas internas, con una reducción estimada de 5–15 ms por consulta en el ambiente de producción actual.
3. **Réplicas de lectura de PostgreSQL.** Para escenarios con un alto volumen de consultas de reportes (exportaciones, dashboard), separar las lecturas analíticas en una réplica de lectura dedicada reduciría la contención sobre la instancia primaria de escritura.
4. **Monitoreo de SLOs y alertas automáticas.** Instrumentar el backend con métricas de latencia por endpoint (P50, P95, P99) y configurar alertas cuando los SLOs de RNF-01 ($P95 < 500$ ms) sean violados, utilizando las capacidades de observabilidad integradas en Railway o una herramienta externa como Datadog / Sentry.

7.5.4 Producto

Las entrevistas semiestructuradas del Capítulo 6 identificaron las siguientes mejoras funcionales como prioritarias desde la perspectiva de los usuarios piloto, ordenadas por urgencia y amplitud de impacto entre empresas:

1. **Lector de código de barras en inventario y ventas** (Miss Peggy). Para tiendas con catálogos superiores a 50 SKUs, la búsqueda manual del producto en el formulario de venta o de movimiento de inventario es el paso de mayor fricción operativa. Un módulo de captura por código de barras —mediante escáner USB o cámara del dispositivo móvil— reduciría el tiempo de registro por transacción y es, según Carolina Forero, condición necesaria para la adopción permanente del sistema en el sector naturista. Esta mejora impacta directamente los módulos de Ventas e Inventario.
2. **Control de fechas de vencimiento por lote** (Miss Peggy). La comercialización de suplementos, vitaminas y productos naturales exige el rastreo de la fecha de vencimiento

de cada entrada de mercancía. El MVP actual gestiona stock en unidades pero no por lote ni fecha de vencimiento. Implementar un modelo de trazabilidad por lote —que permita registrar la fecha de vencimiento al ingresar una entrada de stock y emita alertas cuando un lote se aproxime al vencimiento— es el segundo requisito no negociable para Miss Peggy y constituye una funcionalidad vertical de alto valor para el segmento naturista y farmacéutico.

3. **Integración bidireccional con WooCommerce** (Frozt Bitez). El equipo de Frozt Bitez utiliza WooCommerce como tienda pública y OrbitEngine como back-office; el doble registro manual de pedidos es el principal cuello de botella remanente. Una integración via API de WooCommerce que importe automáticamente los pedidos como ventas en OrbitEngine —y actualice el stock de WooCommerce desde OrbitEngine— eliminaría ese paso y convertiría el sistema en la única fuente de verdad para la operación. Esta mejora es de alto impacto para cualquier empresa piloto con canal de venta online.
4. **Fotografías en el catálogo de productos** (Luana Handmade). Claudia González (Luana) señaló que la ausencia de imágenes en el módulo de inventario la obliga a seguir enviando fotos por WhatsApp cuando una clienta consulta por un producto. Incorporar un campo de imagen por SKU —con carga desde dispositivo o URL— permitiría mostrar el catálogo directamente desde el sistema y reforzaría su utilidad como herramienta de atención al cliente, especialmente en emprendimientos de producto artesanal donde la imagen es parte central de la oferta.

7.6 Trabajo Futuro

7.6.1 Evolución del Producto

1. **Módulo de proveedores y órdenes de compra.** Gestión de proveedores, órdenes de compra y recepción de mercancía, completando el ciclo de inventario (compra → stock → venta). Fue identificado como la funcionalidad ausente con mayor impacto en las

entrevistas del Capítulo 6.

2. **Notificaciones por correo y mensajería.** Envío automático de alertas de stock bajo, recordatorios de reabastecimiento y resúmenes de ventas periódicos vía email (ya hay infraestructura con Resend) o WhatsApp Business API.
3. **Gestión de múltiples sucursales.** Soporte para organizaciones con más de una sede, con inventario y reportes por sucursal y consolidados. Requiere extender el modelo de datos con una entidad Branch y ajustar los filtros de todas las consultas.
4. **Importación masiva de datos.** Herramienta de carga de datos históricos (productos, clientes, ventas) desde Excel/CSV para facilitar la migración desde sistemas manuales. Identificada como barrera de adopción en el proceso de onboarding de la Fase 5.
5. **Facturación electrónica.** Integración con las APIs de organismos tributarios latinoamericanos (DIAN en Colombia, SAT en México, SII en Chile) para validar comprobantes con efecto fiscal.
6. **Aplicación móvil nativa.** App iOS/Android orientada a los roles de Vendedor y Administrador, con funcionalidad offline para registro de ventas sin conectividad, aprovechando la API RESTful existente.

7.6.2 Incorporación de Inteligencia Artificial

La integración de capacidades de aprendizaje automático en los flujos de decisión existentes constituye la extensión de mayor potencial académico y comercial de la plataforma:

1. **Predicción de demanda por producto.** Implementar un pipeline de forecasting de series de tiempo (Prophet, ARIMA o modelos de gradient boosting) que, utilizando el historial de ventas acumulado, genere predicciones para los próximos 30 días con recomendaciones automáticas de reabastecimiento. Esta capacidad está parcialmente diseñada en el documento de planteamiento (`docs/planteamiento/IA.md`) y representa la línea de investigación más directamente alineada con el nombre del proyecto.
2. **Alertas inteligentes de inventario.** Complementar las alertas de stock mínimo actuales

con predicciones dinámicas de ruptura de stock basadas en la tendencia reciente de ventas de cada producto, reduciendo tanto las roturas de stock como el sobrestock.

3. **Análisis de segmentación de clientes (RFM).** Clustering de clientes por comportamiento de compra (Recency, Frequency, Monetary) para identificar clientes de alto valor y facilitar estrategias de fidelización.
4. **Detección de productos de baja rotación.** Identificación automática de productos que inmovilizan capital, facilitando decisiones de liquidación o discontinuación.

7.6.3 Investigación Académica

1. **Estudio longitudinal de adopción a mayor escala.** Diseñar un estudio longitudinal con una muestra representativa de pymes latinoamericanas ($N \geq 30$) para obtener evidencia estadísticamente robusta del impacto de la plataforma en la eficiencia operativa y la toma de decisiones basada en datos. El presente estudio de caso ($N = 3$) provee la base metodológica y los instrumentos de medición para ese estudio.
 2. **Estudio comparativo entre sectores.** Las tres empresas del piloto actual operan en sectores distintos. Un estudio que compare sistemáticamente el impacto de la plataforma en empresas de comercio, servicios y manufactura ligera permitiría identificar qué módulos tienen mayor impacto según el sector.
 3. **Evaluación de la curva de aprendizaje.** La validación actual midió la usabilidad al final de la Fase 5; un estudio con mediciones en el día 1, día 7 y día 30 permitiría caracterizar la curva de aprendizaje del sistema y el tiempo de adopción plena por rol de usuario.
-

7.7 Reflexión Final

OrbitEngine nació de la observación de una brecha concreta: las pymes latinoamericanas carecen de herramientas de gestión operativa adaptadas a su contexto —asequibles, simples de adoptar y útiles desde el primer día de uso. A lo largo de siete meses de desarrollo, el proyecto validó que esa brecha puede cerrarse con tecnologías modernas, metodologías ágiles y un equipo comprometido, sin necesidad de la escala ni los recursos de una corporación tecnológica.

Los resultados, aunque acotados a tres empresas piloto y a un período de dos semanas de uso productivo, indican que la dirección es correcta: los usuarios adoptaron el sistema con relativa facilidad, redujeron el tiempo que dedicaban a tareas repetitivas y valoraron positivamente la posibilidad de acceder a información consolidada de su negocio en tiempo real. Esas mejoras, por modestas que parezcan en términos absolutos, tienen impacto real en la vida cotidiana de los dueños y empleados de las empresas que las experimentan.

El camino que queda por recorrer es más largo que el recorrido hasta aquí: hace falta más investigación, más empresas piloto, más funcionalidades y, sobre todo, más iteración con los usuarios reales. Pero el fundamento está construido, y eso es precisamente lo que un proyecto de grado debe lograr.

Referencias

Las referencias se presentan en formato APA 7.^a edición, ordenadas alfabéticamente por apellido del primer autor.

Amid, A., Moalagh, M., & Ravasan, A. Z. (2012). Identification and classification of ERP critical failure factors in Iranian industries. *Information Systems*, 37(3), 227–237. <https://doi.org/10.1016/j.is.2011.10.010>

Banco Interamericano de Desarrollo. (2021). *La transformación digital de las pymes latino-americanas: Diagnóstico y hoja de ruta*. BID. <https://publications.iadb.org/>

Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>

Benlian, A., & Hess, T. (2011). Opportunities and risks of software-as-a-service: Findings from a survey of IT executives. *Decision Support Systems*, 52(1), 232–246. <https://doi.org/10.1016/j.dss.2011.07.007>

Bezemer, C. P., & Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? En *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)* (pp. 88–92). ACM. <https://doi.org/10.1145/1862372.1862393>

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research*

in *Psychology*, 3(2), 77–101. <https://doi.org/10.1191/1478088706qp063oa>

Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Campbell, D. T., & Stanley, J. C. (1963). *Experimental and quasi-experimental designs for research*. Rand McNally.

Cardona, M., Vera, C., & Tabares, J. (2022). Factores críticos en la implementación de sistemas ERP en pymes colombianas: Una revisión sistemática de la literatura. *Información Tecnológica*, 33(1), 187–196. <https://doi.org/10.4067/S0718-07642022000100187>

Comisión Económica para América Latina y el Caribe. (2022). *Perspectivas económicas de América Latina 2022: Hacia una transición verde y digital*. CEPAL. <https://repositorio.cepal.org/>

Creswell, J. W., & Plano Clark, V. L. (2018). *Designing and conducting mixed methods research* (3rd ed.). SAGE.

Duan, J., Faker, K., Fesak, A., & Stuart, T. (2012). Benefits and drawbacks of cloud-based versus traditional ERP systems. En *Proceedings of Advanced Topics in Information Technology* (pp. 1–12).

Ferraiolo, D. F., Sandhu, R. S., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3), 224–274. <https://doi.org/10.1145/501978.501980>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. <https://www.ics.uci.edu/~fielding/pubs/dissertation/to>

Fowler, M., & Foemmel, M. (2006). *Continuous integration*. ThoughtWorks. <https://www.thoughtworks.com/integration>

Gackenheim, C. (2015). *Introduction to React*. Apress.

Guo, C., Zhang, J., & Liu, B. (2017). A SaaS multi-tenancy performance evaluation

approach based on isolation model. En *Proceedings of the 2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud)* (pp. 277–282). IEEE. <https://doi.org/10.1109/CSCloud.2017.61>

Klaus, H., Rosemann, M., & Gable, G. G. (2000). What is ERP? *Information Systems Frontiers*, 2(2), 141–162. <https://doi.org/10.1023/A:1026543906354>

Kumar, K., & van Hillegersberg, J. (2000). ERP experiences and evolution. *Communications of the ACM*, 43(4), 22–26. <https://doi.org/10.1145/332051.332063>

Ramírez, S. (2018). *FastAPI: High performance, easy to learn, fast to code, ready for production*. <https://fastapi.tiangolo.com/>

Ramírez, S. (2021). *SQLModel: SQL databases in Python, designed for simplicity, compatibility, and robustness*. <https://sqlmodel.tiangolo.com/>

Reichheld, F. F. (2003). The one number you need to grow. *Harvard Business Review*, 81(12), 46–54.

Rodríguez-Abitia, G., Bribiesca-Correa, G., García-Rodríguez, F. J., & Martínez-Alonso, M. (2020). Digital gap in universities and challenges for digital transformation in Mexico during COVID-19. *Future Internet*, 12(12), Artículo 210. <https://doi.org/10.3390/fi12120210>

Soh, C., Kien, S. S., & Tay-Yap, J. (2000). Cultural fits and misfits: Is ERP a universal solution? *Communications of the ACM*, 43(4), 47–51. <https://doi.org/10.1145/332051.332070>

Yin, R. K. (2018). *Case study research and applications: Design and methods* (6th ed.). SAGE.

Zhang, Z., Li, X., & Wang, C. (2021). Usability evaluation of ERP systems for SME users: A multi-criteria approach. *Computers in Human Behavior*, 118, Artículo 106698. <https://doi.org/10.1016/j.chb.2020.106698>

Nota: Las versiones exactas de todas las dependencias de software utilizadas en el proyecto se especifican en los archivos `backend/pyproject.toml` y `fron-`

tend/package.json del repositorio.

Anexo A — Manual de Usuario

OrbitEngine — Plataforma SaaS para la Gestión de Procesos Internos en Pymes

Versión: 1.0 | Abril 2026

Universidad Sergio Arboleda — Semillero de Software como Innovación

A.1 Introducción

Este manual describe el uso de OrbitEngine para los usuarios finales de la plataforma. OrbitEngine es una aplicación web multi-tenant que permite a pequeñas y medianas empresas gestionar en un solo lugar sus operaciones de **inventario**, **ventas** y **clientes**, con un panel de indicadores clave de desempeño (KPIs) accesible en tiempo real.

El sistema opera completamente desde el navegador web, sin necesidad de instalar software adicional. Cada empresa (organización) tiene su propio espacio de trabajo completamente aislado de las demás.

A.2 Acceso a la Plataforma

A.2.1 Página de inicio

Al ingresar a la URL de OrbitEngine, el usuario accede a la **página de presentación** (landing page) que muestra las características de la plataforma. Desde esta página se puede navegar a las opciones de inicio de sesión o registro.

A.2.2 Registro de organización

Para crear una nueva empresa en la plataforma:

1. Hacer clic en **“Comenzar gratis”** o en el enlace de registro en la barra de navegación.
2. En la pantalla `/signup-org`, completar el formulario con:
 - **Nombre de la organización** — nombre visible de la empresa.
 - **Slug** — identificador único en minúsculas (ej. `mi-empresa`). Solo letras, números y guiones; entre 3 y 50 caracteres.
 - **Descripción** (*opcional*) — breve descripción del negocio.
 - **Nombre, apellido y correo electrónico** del administrador.
 - **Contraseña** de al menos 8 caracteres.
3. Hacer clic en **“Crear organización”**.
4. El sistema crea la organización y la cuenta administrador automáticamente y redirige al panel de control.

A.2.3 Invitación de usuarios

Los usuarios adicionales **no se registran por su cuenta**: son creados por el administrador desde el módulo **Admin**. Una vez creada la cuenta, el usuario recibe las credenciales y puede ingresar con ellas.

A.2.4 Inicio de sesión

1. Ir a la pantalla `/login`.

2. Ingresar el **correo electrónico** y la **contraseña**.
3. Hacer clic en **“Ingresar”**.
4. Si las credenciales son correctas, el sistema redirige al **panel de control** (/dashboard).

A.2.5 Recuperación de contraseña

Si olvidó su contraseña:

1. En la pantalla de inicio de sesión, hacer clic en **“¿Olvidaste tu contraseña?”**.
2. Ingresar el correo electrónico asociado a la cuenta.
3. Revisar la bandeja de entrada y hacer clic en el enlace recibido.
4. En la pantalla /reset-password, ingresar y confirmar la nueva contraseña.
5. Hacer clic en **“Restablecer contraseña”**.

A.2.6 Cierre de sesión

Para cerrar sesión, hacer clic en el **ícono de usuario** en la barra lateral izquierda y seleccionar **“Cerrar sesión”**. La sesión expira automáticamente después de 8 días de inactividad.

A.3 Navegación General

Una vez dentro del sistema, la interfaz se divide en:

- **Barra lateral izquierda** — menú principal de navegación con acceso a todos los módulos.
- **Área de contenido principal** — donde se muestra y gestiona la información de cada módulo.
- **Panel de usuario** (parte inferior de la barra lateral) — acceso a configuración personal y cierre de sesión.

Los módulos disponibles son:

Módulo	Ruta	Acceso
Dashboard	/dashboard	Todos los usuarios
Inventario	/dashboard/inventory	Todos los usuarios
Ventas	/dashboard/sales	Todos los usuarios
Clientes	/dashboard/customers	Todos los usuarios
Administración	/dashboard/admin	Solo Administradores
Configuración	/dashboard/settings	Todos los usuarios

A.4 Dashboard — Panel de Indicadores

El dashboard muestra un resumen del estado del negocio en tiempo real para la organización activa.

Indicadores disponibles:

- **Ventas del día** — total en dinero de las ventas registradas hoy.
- **Ventas del mes** — total acumulado del mes en curso.
- **Ticket promedio** — valor promedio por venta en el período.
- **Productos con stock bajo** — conteo de productos que están por debajo del mínimo configurado.
- **Top productos** — listado de los productos más vendidos.
- **Ventas por día** — gráfica de barras con la evolución diaria de ventas.

El dashboard se actualiza automáticamente con cada consulta. No requiere acción adicional del usuario.

A.5 Módulo de Inventario

El módulo de inventario permite gestionar los productos y categorías de la empresa, así como llevar el control detallado de los movimientos de stock.

A.5.1 Categorías

Las categorías permiten organizar el inventario en grupos lógicos. Admiten **jerarquía** (una categoría puede tener subcategorías).

Crear una categoría: 1. En el módulo Inventario, ir a la pestaña **Categorías**. 2. Hacer clic en “**Agregar categoría**”. 3. Completar el **nombre** (*requerido*) y opcionalmente una **descripción** y una **categoría padre**. 4. Guardar.

Editar o desactivar una categoría: - Hacer clic en el menú de acciones (:) de la fila y seleccionar “**Editar**” o “**Desactivar**”.

A.5.2 Productos

Crear un producto: 1. En el módulo Inventario, ir a la pestaña **Productos**. 2. Hacer clic en “**Agregar producto**”. 3. Completar el formulario: - **Nombre** (*requerido*) - **SKU** — código único del producto (*requerido*) - **Categoría** (*opcional*) - **Precio de costo** y **precio de venta** - **Stock actual**, **stock mínimo** y **stock máximo** (*opcional*) - **Unidad de medida** (ej. unit, kg, lt) - **Código de barras** (*opcional*) - **Descripción** e **imagen** (*opcionales*) 4. Guardar.

Editar un producto: - Hacer clic en el menú de acciones (:) → “**Editar**”. Todos los campos son modificables excepto el SKU si ya tiene movimientos.

Desactivar un producto: - Hacer clic en el menú de acciones (:) → “**Desactivar**”. El producto deja de aparecer en los formularios de venta pero su historial se conserva.

Alerta de stock bajo: - Los productos con $\text{stock_quantity} \leq \text{stock_min}$ aparecen resaltados en la tabla y se contabilizan en el indicador del dashboard.

A.5.3 Ajuste de stock manual

Para corregir el inventario por diferencias de conteo, mermas u otros motivos:

1. En la tabla de productos, seleccionar el producto y hacer clic en “**Ajustar stock**”.
2. Ingresar la **cantidad** (positiva para agregar, negativa para restar).
3. Ingresar un **motivo** del ajuste (*requerido*).
4. Confirmar.

El ajuste queda registrado en el historial de movimientos con el tipo adjustment.

A.5.4 Historial de movimientos

El historial registra todos los cambios de stock de cada producto:

Tipo de movimiento	Descripción
sale	Salida generada por una venta
adjustment	Entrada o salida manual
return	Entrada por devolución o cancelación de venta
purchase	Entrada por compra

Para ver el historial de un producto: 1. En la tabla de productos, hacer clic en el menú de acciones (:) → “**Ver historial de movimientos**”. 2. Se muestra la tabla con fecha, tipo, cantidad, stock anterior y stock resultante, y el usuario que realizó el movimiento.

A.6 Módulo de Ventas

El módulo de ventas permite registrar transacciones comerciales, visualizar el historial y cancelar ventas cuando sea necesario.

A.6.1 Registrar una venta

1. En el módulo Ventas, hacer clic en “**Nueva venta**”.

2. Completar el formulario:
 - **Cliente** (*opcional*) — buscar por nombre o documento.
 - **Método de pago**: Efectivo, Tarjeta, Transferencia u Otro.
 - **Descuento y IVA/impuesto** (*opcional, en valores monetarios*).
 - **Notas** (*opcional*).
3. En la sección de **ítems**, agregar los productos de la venta:
 - Buscar el producto por nombre o SKU.
 - Ingresar la **cantidad**.
 - El sistema calcula el precio unitario y el subtotal del ítem automáticamente.
4. El sistema calcula en tiempo real: subtotal, descuento, impuesto y **total**.
5. Hacer clic en “**Registrar venta**”.

Al guardar, el sistema: - Genera un **número de factura** único. - Descuenta las cantidades del stock de cada producto registrado. - Registra los movimientos de inventario correspondientes (sale). - Actualiza el total de compras del cliente, si se asoció uno.

A.6.2 Ver detalle de una venta

En la tabla de ventas, hacer clic en el menú de acciones (:) → “**Ver detalle**”. Se muestra la información completa: número de factura, cliente, fecha, ítems, totales y método de pago.

A.6.3 Cancelar una venta

Una venta puede cancelarse mientras tenga estado completed:

1. En la tabla de ventas, hacer clic en el menú de acciones (:) → “**Cancelar venta**”.
2. Ingresar el **motivo de cancelación** (*requerido*).
3. Confirmar.

Al cancelar, el sistema: - Cambia el estado de la venta a cancelled. - **Revierde el stock** de cada producto afectado (movimiento tipo return). - Registra la fecha y usuario de cancelación.

A.6.4 Filtros y búsqueda

La tabla de ventas permite: - Buscar por **número de factura**. - Filtrar por **estado** (completed, cancelled) y **método de pago**. - Ordenar por cualquier columna.

A.7 Módulo de Clientes

El módulo permite gestionar el directorio de clientes de la organización y consultar su historial de compras.

A.7.1 Registrar un cliente

1. En el módulo Clientes, hacer clic en “**Agregar cliente**”.
2. Completar el formulario:
 - **Tipo de documento**: CC, NIT, Pasaporte, Otro (*requerido*).
 - **Número de documento** (*requerido, único por organización*).
 - **Nombre y apellido** (*requeridos*).
 - **Correo electrónico, teléfono, dirección, ciudad y país** (*opcionales*).
 - **Notas internas** (*opcionales*).
3. Guardar.

A.7.2 Editar un cliente

- Hacer clic en el menú de acciones (:) → “**Editar**”. Se pueden modificar todos los campos excepto el número de documento.

A.7.3 Desactivar un cliente

- Hacer clic en el menú de acciones (:) → “**Desactivar**”. El cliente se desactiva pero su historial de compras se conserva.

A.7.4 Historial de compras del cliente

- Hacer clic en el menú de acciones (:) → **“Ver historial de compras”**.
 - Se muestran todas las ventas asociadas al cliente con fecha, número de factura, total y estado.
 - Se visualizan también los indicadores: **total acumulado de compras, número de compras y fecha de última compra**.
-

A.8 Módulo de Administración

Este módulo es exclusivo para usuarios con el rol **Administrador**.

El módulo de administración permite gestionar las cuentas de usuario de la organización.

A.8.1 Ver usuarios

La tabla muestra todos los usuarios de la organización con su nombre, correo, rol y estado (activo/inactivo).

A.8.2 Crear un usuario

1. Hacer clic en **“Agregar usuario”**.
2. Completar el formulario:
 - **Nombre y apellido**.
 - **Correo electrónico** — será el identificador de ingreso.
 - **Contraseña** temporal.
 - **Rol** — seleccionar entre los roles disponibles de la organización.
3. Guardar.

El nuevo usuario puede ingresar inmediatamente con las credenciales proporcionadas y cambiar su contraseña desde la configuración.

A.8.3 Editar un usuario

- Hacer clic en el menú de acciones (:) → **“Editar”**. Se pueden cambiar nombre, apellido y rol.

A.8.4 Eliminar un usuario

- Hacer clic en el menú de acciones (:) → **“Eliminar”**.
- Confirmar la acción en el diálogo de confirmación.

Advertencia: Esta acción es permanente. El historial de ventas y movimientos del usuario permanece registrado.

A.9 Configuración

El módulo de configuración permite al usuario gestionar su cuenta personal y, si es administrador, los datos de la organización.

A.9.1 Información personal

En la pestaña **“Perfil”**: - Ver y editar **nombre** y **apellido**. - Ver el **correo electrónico** asociado (no editable desde aquí).

A.9.2 Cambio de contraseña

En la pestaña **“Seguridad”**: 1. Ingresar la **contraseña actual**. 2. Ingresar la **nueva contraseña** y confirmarla. 3. Hacer clic en **“Actualizar contraseña”**.

A.9.3 Configuración de la organización

Disponible solo para usuarios con rol **Administrador**.

En la pestaña “**Organización**”: - Editar el **nombre** de la organización. - Editar el **slug** (*identificador único*). - Editar la **descripción** y la **URL del logo**.

A.9.4 Apariencia

En la pestaña “**Apariencia**”: - Cambiar entre **modo claro** y **modo oscuro**.

A.9.5 Eliminación de cuenta

En la pestaña “**Cuenta**”: - El usuario puede solicitar la **eliminación de su propia cuenta**. - Se requiere confirmación escribiendo el correo electrónico.

Nota: La eliminación de cuenta no elimina el historial de transacciones asociadas.

A.10 Preguntas Frecuentes

¿Puedo usar OrbitEngine desde el celular?

Sí. La interfaz está diseñada de manera responsiva y funciona en navegadores móviles modernos (Chrome, Safari). La experiencia óptima es en pantallas de escritorio o tabletas.

¿Mis datos están separados de los de otras empresas?

Sí. Cada organización tiene un espacio de trabajo completamente aislado. No es posible ver ni acceder a datos de otras organizaciones.

¿Qué pasa si cancelo una venta? ¿El stock vuelve?

Sí. Al cancelar una venta, el sistema revierte automáticamente el stock de todos los productos incluidos en ella.

¿Puedo tener múltiples roles para usuarios?

Actualmente cada usuario tiene un único rol. Los roles definen los permisos de acceso a los módulos y acciones disponibles.

¿Cómo sé cuándo un producto está por agotarse?

Los productos cuyo stock actual es igual o menor al **stock mínimo** configurado aparecen resaltados en la tabla de inventario y se contabilizan en el indicador “Productos con stock bajo” del dashboard.

¿Se puede exportar la información a Excel o PDF?

Los listados de inventario, ventas y clientes pueden exportarse a Excel (.xlsx) desde el módulo correspondiente, conservando los filtros aplicados. La exportación a PDF no está disponible en esta versión.

¿Qué navegadores son compatibles?

OrbitEngine es compatible con las versiones recientes de Chrome, Firefox, Edge y Safari. No se garantiza compatibilidad con Internet Explorer.

Documento generado como parte del proyecto de grado — Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.

Anexo B — Manual de Instalación y Despliegue

OrbitEngine — Plataforma SaaS para la Gestión de Procesos Internos en Pymes

Versión: 1.0 | Abril 2026

Universidad Sergio Arboleda — Semillero de Software como Innovación

B.1 Visión General

OrbitEngine utiliza **Docker Compose** como orquestador de contenedores en todos los entornos. La arquitectura de despliegue se compone de los siguientes servicios:

Servicio	Imagen	Descripción
db	postgres:18	Base de datos relacional PostgreSQL
backend	Custom (FastAPI)	API REST del sistema
frontend	Custom (React/Nginx)	Interfaz de usuario compilada
prestart	Custom (FastAPI)	Ejecuta migraciones y datos iniciales al iniciar
adminer	adminer	Panel web de administración de BD
proxy	traefik:3.6	Reverse proxy y gestor de rutas HTTP/HTTPS
mailcatcher	schickling/mailcatcher	Servidor SMTP de prueba (solo dev)

El enrutamiento externo se gestiona con **Traefik**, que expone los servicios bajo subdominios configurados y maneja la terminación TLS automática mediante Let's Encrypt en producción.

B.2 Prerrequisitos

B.2.1 Herramientas requeridas

Herramienta	Versión mínima	Propósito
Docker Engine	24.x	Contenedores
Docker	v2.x	Orquestación de servicios
Compose		
Git	2.x	Control de versiones
uv	0.4+	Gestor de dependencias Python (para desarrollo local sin Docker)
Bun	1.x	Runtime JS / gestor de paquetes (para desarrollo local sin Docker)

B.2.2 Puertos utilizados

Puerto	Servicio	Entorno
80	Traefik HTTP	Dev y Producción
443	Traefik HTTPS	Producción
8000	Backend FastAPI	Dev (expuesto directamente)
5173	Frontend (Vite)	Dev
5432	PostgreSQL	Dev
8080	Adminer	Dev
8090	Traefik Dashboard	Dev
1025	Mailcatcher SMTP	Dev
1080	Mailcatcher UI	Dev

B.3 Configuración de Variables de Entorno

Antes de ejecutar cualquier entorno, crear el archivo `.env` en la raíz del repositorio:

```
cp .env.example .env # si existe el archivo de ejemplo, de lo contrario  
↪ crearlo manualmente
```

Las variables requeridas son:

```
# — Proyecto  
↪  
PROJECT_NAME=OrbitEngine  
STACK_NAME=orbitengine-stack  
DOMAIN=localhost # En producción: tu-dominio.com  
  
# — Backend  
↪  
SECRET_KEY=cambia-esto-en-produccion # Clave aleatoria, mínimo 32 chars  
BACKEND_CORS_ORIGINS=["http://localhost:5173","http://localhost"]  
FRONTEND_HOST=http://localhost:5173 # En producción:  
↪ https://tu-dominio.com  
  
# — Base de datos  
↪  
POSTGRES_SERVER=db  
POSTGRES_PORT=5432  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=cambia-esto  
POSTGRES_DB=app  
  
# — Superusuario inicial  
↪  
FIRST_SUPERUSER=admin@ejemplo.com  
FIRST_SUPERUSER_PASSWORD=cambia-esto
```



```
# — Email
↪
SMTP_HOST=mailcatcher                # En producción:
↪ smtp.tu-proveedor.com
SMTP_PORT=1025                        # En producción: 587
SMTP_TLS=false                       # En producción: true
EMAILS_FROM_EMAIL=noreply@tu-dominio.com

# — Imágenes Docker
↪
DOCKER_IMAGE_BACKEND=backend
DOCKER_IMAGE_FRONTEND=frontend
ENVIRONMENT=local                    # local | staging | production
```

Producción: Cambiar todas las claves y contraseñas por valores aleatorios seguros. Se recomienda gestionar las variables de entorno directamente en el panel de Railway (backend) y en el panel de Vercel (frontend), evitando archivos `.env` en producción.

B.4 Entorno de Desarrollo Local

El entorno de desarrollo combina `compose.yml` con `compose.override.yml`, que añade los servicios de desarrollo (Traefik local, Mailcatcher) y sobrescribe las configuraciones para recarga en caliente.

B.4.1 Iniciar el entorno

Desde la raíz del repositorio

```
docker compose watch
```

Este comando:

1. Construye las imágenes de backend y frontend si no existen.
2. Levanta todos los servicios en orden de dependencias.
3. Activa la recarga automática del backend al detectar cambios en `./backend`.
4. El servicio `prestart` ejecuta las migraciones de Alembic y los datos iniciales antes de que el backend acepte peticiones.

B.4.2 URLs del entorno local

Servicio	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000
Documentación interactiva (Swagger)	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
Adminer	http://localhost:8080
Traefik Dashboard	http://localhost:8090
Mailcatcher UI	http://localhost:1080

B.4.3 Detener el entorno

Detener y eliminar contenedores (datos persisten en volumen)

```
docker compose down
```

Detener y eliminar contenedores + volúmenes (elimina datos de BD)

```
docker compose down -v
```

B.4.4 Ver logs

```
docker compose logs -f backend    # Logs del backend en tiempo real
docker compose logs -f frontend    # Logs del frontend
docker compose logs -f db          # Logs de PostgreSQL
```

B.5 Migraciones de Base de Datos

Las migraciones se gestionan con **Alembic** y se ejecutan automáticamente al iniciar el entorno (servicio prestart). Para operaciones manuales:

B.5.1 Aplicar migraciones pendientes

```
docker compose exec backend alembic upgrade head
```

B.5.2 Crear una nueva migración

Después de modificar los modelos en `backend/app/models.py`:

```
docker compose exec backend alembic revision --autogenerate -m
↪ "descripcion_del_cambio"
```

Revisar el archivo generado en `backend/app/alembic/versions/` antes de aplicarlo.

B.5.3 Ver historial de migraciones

```
docker compose exec backend alembic history --verbose
```

B.5.4 Revertir la última migración

```
docker compose exec backend alembic downgrade -1
```

B.5.5 Migraciones del proyecto

Las migraciones actuales del esquema son:

Archivo	Descripción
001_initial_schema.py	Tablas base: organizations, roles, users
002_categories.py	Tabla categories con jerarquía padre-hijo
003_products.py	Tabla products
004_customers.py	Tabla customers
005_inventory_movements.py	Tabla inventory_movements
006_sales.py	Tablas sales y sale_items

B.6 Pruebas

B.6.1 Pruebas del backend (pytest)

Ejecutar todas las pruebas con cobertura

```
cd backend
```

```
uv run bash scripts/test.sh
```

Prueba de un archivo específico

```
uv run pytest tests/api/routes/test_sales.py -v
```

Prueba de una función específica

```
uv run pytest tests/api/routes/test_users.py::test_get_users_superuser_me
```

↩ -v

```
# Pruebas por patrón  
uv run pytest -k "test_create" -v  
  
# Reporte de cobertura  
uv run coverage run -m pytest && coverage report
```

La cobertura de pruebas abarca los principales módulos del sistema, tanto a nivel de API como de lógica CRUD. Se realizaron pruebas completas de API y CRUD para los módulos de usuarios, categorías, productos, clientes, ventas, movimientos de inventario y dashboard, lo que garantiza la validez funcional y la robustez de estos componentes. En el módulo de autenticación, la cobertura se concentra exclusivamente en pruebas de la API, debido a que la lógica principal reside en los flujos de autenticación y autorización más que en operaciones CRUD tradicionales. Esta cobertura integral asegura que los procesos críticos relacionados con la gestión de datos y la seguridad hayan sido evaluados mediante casos de prueba automatizados.

B.6.2 Pruebas del frontend (Playwright E2E)

```
cd frontend  
  
# Ejecutar todas las pruebas E2E  
bun run test  
  
# Prueba de un archivo específico  
bunx playwright test tests/login.spec.ts  
  
# Prueba por descripción  
bunx playwright test --grep "crear venta"
```

```
# Modo con interfaz visual  
bun run test:ui
```

Las pruebas E2E cubren los flujos: login, registro de organización, registro de usuario, restablecimiento de contraseña, inventario, ventas, clientes, administración de usuarios y configuración.

B.7 Generación del Cliente API

Cuando se modifican los endpoints del backend, el cliente TypeScript del frontend debe regenerarse:

```
# El backend debe estar corriendo en http://localhost:8000  
cd frontend  
bun run generate-client
```

Esto actualiza los archivos en `src/client/` a partir del esquema OpenAPI del backend. **No editar manualmente** estos archivos.

B.8 Despliegue en Producción

El despliegue en producción usa el mismo `compose.yml` sin el `compose.override.yml`.

B.8.1 Prerrequisitos del servidor

- Servidor Linux con Docker Engine y Docker Compose instalados.
- Dominio apuntando a la IP del servidor (registros A/CNAME en el DNS).

- Puertos 80 y 443 abiertos en el firewall.

B.8.2 Preparar la red de Traefik

Traefik requiere una red Docker externa compartida entre todos los servicios:

```
docker network create traefik-public
```

B.8.3 Configurar Traefik con TLS

Crear el archivo de configuración de Traefik (compose.traefik.yml ya incluido en el repositorio) y levantarlo:

```
docker compose -f compose.traefik.yml up -d
```

Esto levanta Traefik en modo producción con:

- Redirección automática HTTP → HTTPS.
- Certificados TLS gestionados por Let's Encrypt.
- Panel de Traefik accesible solo internamente.

B.8.4 Configurar variables de entorno para producción

Editar el .env con los valores de producción:

```
DOMAIN=tu-dominio.com
ENVIRONMENT=production
SECRET_KEY=<clave-aleatoria-larga>
POSTGRES_PASSWORD=<contraseña-segura>
FIRST_SUPERUSER_PASSWORD=<contraseña-segura>
FRONTEND_HOST=https://tu-dominio.com
BACKEND_CORS_ORIGINS=["https://tu-dominio.com"]
SMTP_HOST=smtp.tu-proveedor.com
```

```
SMTP_PORT=587
SMTP_TLS=true
EMAILS_FROM_EMAIL=noreply@tu-dominio.com
```

B.8.5 Construir y desplegar

Construir imágenes con la etiqueta de versión

```
TAG=v1.0.0 docker compose build
```

Iniciar todos los servicios

```
TAG=v1.0.0 docker compose up -d
```

Verificar estado

```
docker compose ps
```

```
docker compose logs -f
```

Los servicios se levantan en este orden:

1. db — espera estar healthy (hasta 30s).
2. prestart — ejecuta migraciones y datos iniciales.
3. backend y frontend — arrancan tras prestart.
4. adminer — panel de administración de BD.

B.8.6 Actualización de la aplicación

Obtener la nueva versión

```
git pull origin main
```

Reconstruir imágenes


```
TAG=v1.1.0 docker compose build
```

```
# Reiniciar servicios (downtime mínimo: solo el backend se reinicia)
```

```
TAG=v1.1.0 docker compose up -d --no-deps backend frontend
```

```
# Las migraciones se aplican automáticamente al iniciar prestart
```

B.9 Monitoreo y Mantenimiento

B.9.1 Estado de los servicios

```
docker compose ps # Estado de todos los contenedores
```

```
docker stats # Uso de CPU/RAM en tiempo real
```

```
docker compose exec db psql -U postgres -c "\l" # Listar BDs en
```

```
↪ PostgreSQL
```

B.9.2 Backup de la base de datos

```
# Crear un backup
```

```
docker compose exec db pg_dump -U postgres app > backup_$(date
```

```
↪ +%Y%m%d).sql
```

```
# Restaurar un backup
```

```
docker compose exec -T db psql -U postgres app < backup_20260401.sql
```

B.9.3 Limpiar recursos no usados

```
docker system prune -f          # Elimina imágenes, contenedores y
↪ redes sin usar
docker volume prune -f          # Elimina volúmenes sin usar
↪ (¡precaución en producción!)
```

B.10 Solución de Problemas

Problema	Causa probable	Solución
El backend no inicia	BD no está disponible	Verificar que db esté healthy con <code>docker compose ps</code> . Revisar <code>POSTGRES_*</code> en <code>.env</code> .
Error de migraciones al iniciar	Migración incompatible	Ejecutar <code>docker compose exec backend alembic history y alembic current</code> . Corregir el script de migración.
502 Bad Gateway en el frontend	Backend no responde	Verificar logs del backend: <code>docker compose logs backend</code> . Comprobar el healthcheck.
Emails no se envían (producción)	SMTP mal configurado	Verificar <code>SMTP_HOST</code> , <code>SMTP_PORT</code> y credenciales. Probar con <code>telnet \$SMTP_HOST \$SMTP_PORT</code> .
Secret key too short al iniciar	<code>SECRET_KEY</code> es demasiado corta	Generar una clave con <code>openssl rand -hex 32</code> y actualizar <code>.env</code> .
Traefik no renueva el certificado TLS	Puerto 80 bloqueado o DNS incorrecto	Verificar que el dominio resuelva a la IP correcta y el puerto 80 esté abierto.
Error UNIQUE constraint al insertar	Dato duplicado en BD	Verificar que el SKU, slug u otro campo único no esté repetido.

Documento generado como parte del proyecto de grado — Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.

Anexo C — Documentación Técnica

OrbitEngine — Plataforma SaaS para la Gestión de Procesos Internos en Pymes

Versión: 1.0 | Abril 2026

Universidad Sergio Arboleda — Semillero de Software como Innovación

C.1 Stack Tecnológico

C.1.1 Backend

Componente	Tecnología	Versión	Propósito
Lenguaje	Python	3.10+	Lenguaje principal del backend
Framework web	FastAPI	0.115+	API REST asíncrona con tipado estricto
ORM + Schemas	SQLModel	0.0.21+	Modelos compartidos entre SQLAlchemy y Pydantic
Motor de BD	SQLAlchemy	2.x	Capa de abstracción de base de datos
Driver PostgreSQL	Psycopg3	3.x	Conector PostgreSQL para Python
Migraciones	Alembic	1.x	Gestión de migraciones de esquema
Autenticación	PyJWT	2.x	Generación y verificación de tokens JWT
Hash de contraseñas	Pwplib	0.2+	Hashing seguro con bcrypt
Validación	Pydantic	2.x	Schemas de entrada/salida de datos
Configuración	Pydantic-Settings	2.x	Variables de entorno tipadas
Linter/Formatter	Ruff	0.6+	Análisis estático y formato de código
Verificación de tipos	Mypy	1.x	Chequeo de tipos estático en modo estricto
Testing	Pytest + Coverage	8.x	Pruebas unitarias e integración
Monitoring	Sentry (opcional)	—	Rastreo de errores en producción

C.1.2 Frontend

Componente	Tecnología	Versión	Propósito
Lenguaje	TypeScript	5.x	Tipado estático sobre JavaScript
Framework UI	React	19.x	Librería de componentes reactivos
Build tool	Vite	6.x	Empaquetado y servidor de desarrollo
Router	TanStack Router	1.x	Enrutamiento con tipado por archivo
Server state	TanStack Query	5.x	Cache y sincronización de datos del servidor
Tablas	TanStack Table	8.x	Tablas con sorting/filtering/paginación
Formularios	React Hook Form	7.x	Manejo de formularios con rendimiento optimizado
Validación	Zod	3.x	Schemas de validación con inferencia de tipos
UI Components	Shadcn/ui	—	Componentes base sobre Radix UI
Estilos	Tailwind CSS	4.x	Utilidades CSS de bajo nivel
Gráficos	Recharts	2.x	Gráficas para el dashboard
HTTP Client	Axios (generado)	—	Cliente OpenAPI auto-generado
Notificaciones	Sonner	1.x	Toasts y notificaciones
Íconos	Lucide React	0.4+	Librería de íconos SVG
Linter/Formatter	Biome	1.x	Análisis estático y formato
E2E Testing	Playwright	1.x	Pruebas end-to-end en navegador

C.1.3 Infraestructura

Componente	Tecnología	Propósito
Base de datos	PostgreSQL 18	Almacenamiento relacional principal
Contenedores	Docker Compose v2	Orquestación de servicios
Reverse proxy	Traefik 3.6	Enrutamiento HTTP/HTTPS + TLS automático
CI/CD	GitHub Actions	Pipeline de integración y entrega continua
Panel de BD	Adminer	Administración web de PostgreSQL
Email (dev)	Mailcatcher	Interceptor SMTP para desarrollo

C.2 Estructura del Proyecto

orbit-engine/

```

├─ .env                # Variables de entorno (no incluir en git)
├─ compose.yml         # Configuración Docker Compose base
├─ compose.override.yml # Sobreescrituras para desarrollo local
├─ compose.traefik.yml  # Traefik para producción
├─ pyproject.toml       # Configuración Ruff y Mypy (nivel raíz)
|
├─ backend/
|   ├─ Dockerfile
|   ├─ pyproject.toml   # Dependencias Python (uv)
|   ├─ scripts/
|   |   ├─ prestart.sh  # Ejecutado por el servicio prestart
|   |   ├─ test.sh      # Script de pruebas con cobertura
|   |   ├─ lint.sh      # Mypy + Ruff check
|   |   └─ format.sh    # Ruff format
|   └─ app/
|       ├─ main.py      # Punto de entrada FastAPI + registro de routers
|       ├─ models.py    # Todos los modelos SQLAlchemy + schemas Pydantic
|       ├─ crud.py       # Operaciones de base de datos
|       ├─ utils.py      # Helpers (email, generación de tokens)
|       └─ backend_pre_start.py # Script de inicialización
|   └─ initial_data.py  # Seed de datos iniciales (roles, superusuario)
|       └─ api/
|           ├─ deps.py   # Dependencias FastAPI (auth, sesión, roles)
|           └─ routes/
|               ├─ login.py      # Autenticación y recuperación de contraseña
|               └─ users.py      # Gestión de usuarios
|           └─ organizations.py # Registro y configuración de organización
|               └─ roles.py      # Listado de roles
|               └─ categories.py # CRUD de categorías

```

```

|       |       |— products.py           # CRUD de productos + stock
|       |       |— customers.py          # CRUD de clientes
|       |       |— sales.py              # Registro y gestión de ventas
|       |       |— inventory_movements.py # Historial de movimientos
|       |       |— dashboard.py           # Estadísticas del dashboard
|       |       |— utils.py               # Health check
|       |       |— private.py            # Endpoints de superusuario
|       |— core/
|       |   |— config.py                 # Settings con Pydantic-Settings
|       |   |— db.py                     # Engine SQLAlchemy
|       |   |— security.py                # JWT, hashing de contraseñas
|       |— alembic/
|       |   |— env.py
|       |   |— versions/
|       |       |— 001_initial_schema.py  # organizations, roles, users
|       |       |— 002_categories.py
|       |       |— 003_products.py
|       |       |— 004_customers.py
|       |       |— 005_inventory_movements.py
|       |       |— 006_sales.py
|
|— frontend/
|   |— Dockerfile
|   |— package.json
|   |— vite.config.ts
|   |— openapi-ts.config.ts  # Configuración del generador de cliente API
|   |— src/
|       |— main.tsx          # Punto de entrada React
|       |— routeTree.gen.ts  # Árbol de rutas auto-generado (no editar)

```

```
|   └─ utils.ts           # Helpers compartidos
|   └─ client/            # Cliente API auto-generado (no editar)
|   │   └─ index.ts
|   │   └─ sdk.gen.ts     # Funciones de llamada a la API
|   │   └─ types.gen.ts   # Tipos TypeScript desde OpenAPI
|   │   └─ schemas.gen.ts
|   │   └─ core/          # Configuración Axios
|   └─ components/
|   │   └─ Admin/         # Componentes de gestión de usuarios
|   │   └─ Common/       # Componentes reutilizables (DataTable, layouts)
|   │   │   └─ Customers/ # Componentes del módulo de clientes
|   │   │   └─ Inventory/ # Componentes del módulo de inventario
|   │   │   └─ Landing/   # Componentes de la página pública
|   │   │   └─ Pending/   # Componentes de estados de carga
|   │   │   └─ Sales/     # Componentes del módulo de ventas
|   │   │   └─ Sidebar/   # Navegación principal
|   │   │   └─ UserSettings/ # Configuración de perfil y organización
|   │   │   └─ theme-provider.tsx
|   │   │   └─ ui/        # Componentes Shadcn/ui (no editar)
|   └─ hooks/
|   │   └─ useAuth.ts
|   │   └─ useCopyToClipboard.ts
|   │   └─ useCustomToast.ts
|   │   └─ useDebounce.ts
|   │   └─ useMobile.ts
|   │   └─ useScrollAnimation.ts
|   └─ lib/
|   │   └─ utils.ts       # cn() helper para Tailwind
|   └─ routes/
```

```

|           └─ __root.tsx      # Layout raíz con providers
|           └─ index.tsx       # Landing page (/)
|           └─ login.tsx       # Inicio de sesión (/login)
|           └─ signup.tsx      # Registro de usuario (/signup)
|           └─ signup-org.tsx  # Registro de organización (/signup-org)
|           └─ recover-password.tsx
|           └─ reset-password.tsx
|           └─ dashboard/
|               └─ index.tsx    # Dashboard KPIs (/dashboard)
|               └─ inventory.tsx
|               └─ sales.tsx
|               └─ customers.tsx
|               └─ admin.tsx
|               └─ settings.tsx
|
└─ docs/
    └─ informe-final/          # Documentación del proyecto de grado

```

C.3 Autenticación y Autorización

C.3.1 Flujo de autenticación

OrbitEngine usa **JWT Bearer Tokens** siguiendo el estándar OAuth2 Password Flow:

1. El cliente envía credenciales a POST /api/v1/login/access-token.
2. El backend valida las credenciales, verifica que la cuenta no esté bloqueada.
3. Si son válidas, retorna un access_token JWT.
4. El cliente incluye el token en el header Authorization: Bearer <token> en cada request.

5. El backend decodifica el token, extrae el sub (UUID del usuario) y carga el usuario de la BD.
6. La organización del usuario se extrae de `user.organization_id` — no del token directamente.

Parámetros del token:

Parámetro	Valor
Algoritmo	HS256
Payload sub	UUID del usuario (str)
Expiración	8 días (11,520 minutos)
Clave de firma	SECRET_KEY del entorno

C.3.2 Control de acceso basado en roles (RBAC)

Los roles se almacenan en la tabla `roles` con permisos como array JSONB. La verificación en los endpoints usa la dependencia `require_role()`:

```
# Solo administradores pueden crear usuarios
@router.post("/", dependencies=[Depends(require_role("admin"))])
def create_user(...) -> Any: ...

# Administradores y vendedores pueden registrar ventas
@router.post("/", dependencies=[Depends(require_role("admin",
↪ "seller"))])
def create_sale(...) -> Any: ...
```

Dependencias de autenticación disponibles:

Tipo anotado	Descripción
<code>CurrentUser</code>	Usuario autenticado; error 403 si token inválido
<code>CurrentAdminUser</code>	Usuario autenticado con rol admin; error 403 si no
<code>CurrentOrganization</code>	UUID de organización extraído del <code>CurrentUser</code>

Tipo anotado	Descripción
SessionDep	Sesión de base de datos SQLModel

C.3.3 Protección contra fuerza bruta

El modelo User incluye campos `failed_login_attempts` y `locked_until`. Tras N intentos fallidos consecutivos, la cuenta se bloquea temporalmente.

C.4 API REST — Referencia de Endpoints

Todos los endpoints están prefijados con `/api/v1`. La documentación interactiva se sirve en `/docs` (Swagger UI) y `/redoc`.

C.4.1 Autenticación (/login)

Método	Ruta	Auth	Descripción
POST	<code>/login/access-token</code>	No	Obtener token JWT (login)
POST	<code>/login/test-token</code>	Sí	Verificar validez del token
POST	<code>/password-recovery/{email}</code>	No	Solicitar enlace de recuperación
POST	<code>/reset-password</code>	No	Restablecer contraseña con token

C.4.2 Usuarios (/users)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	<code>/users/</code>	Sí	admin	Listar usuarios de la organización
POST	<code>/users/</code>	Sí	admin	Crear usuario en la organización
GET	<code>/users/me</code>	Sí	Cualquiera	Obtener perfil propio
PATCH	<code>/users/me</code>	Sí	Cualquiera	Actualizar nombre/apellido/teléfono
DELETE	<code>/users/me</code>	Sí	Cualquiera	Eliminar cuenta propia (soft delete)
PATCH	<code>/users/me/password</code>	Sí	Cualquiera	Cambiar contraseña

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/users/{user_id}	Sí	admin	Obtener usuario por ID
PATCH	/users/{user_id}	Sí	admin	Actualizar usuario
DELETE	/users/{user_id}	Sí	admin	Eliminar usuario

C.4.3 Organizaciones (/organizations)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/organizations/me	Sí	Cualquiera	Ver datos de la organización actual
PATCH	/organizations/me	Sí	admin	Actualizar nombre/slug/descripción
POST	/organizations/signup	No	—	Registro de nueva organización + admin

C.4.4 Roles (/roles)

Método	Ruta	Auth	Descripción
GET	/roles/	Sí	Listar roles disponibles en la organización

C.4.5 Categorías (/categories)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/categories/	Sí	Cualquiera	Listar categorías (soporta search, is_active)
POST	/categories/	Sí	admin	Crear categoría
GET	/categories/{id}	Sí	Cualquiera	Obtener categoría por ID
PATCH	/categories/{id}	Sí	admin	Actualizar categoría
DELETE	/categories/{id}	Sí	admin	Eliminar / desactivar categoría

C.4.6 Productos (/products)

Método	Ruta	Rol		Descripción
		Auth	mínimo	
GET	/products/	Sí	Cualquiera	Listar productos (search, is_active, category_id, sort_by)
POST	/products/	Sí	admin	Crear producto
GET	/products/{id}	Sí	Cualquiera	Obtener producto por ID
PATCH	/products/{id}	Sí	admin	Actualizar producto
DELETE	/products/{id}	Sí	admin	Eliminar / desactivar producto
POST	/products/{id}/stock-adjustment	Sí	admin	Ajuste manual de stock
GET	/products/{id}/movements	Sí	Cualquiera	Historial de movimientos del producto

C.4.7 Clientes (/customers)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/customers/	Sí	Cualquiera	Listar clientes (search, is_active)
POST	/customers/	Sí	Cualquiera	Registrar cliente
GET	/customers/{id}	Sí	Cualquiera	Obtener cliente por ID
PATCH	/customers/{id}	Sí	Cualquiera	Actualizar cliente
DELETE	/customers/{id}	Sí	admin	Desactivar cliente
GET	/customers/{id}/sales	Sí	Cualquiera	Historial de compras del cliente

C.4.8 Ventas (/sales)

Método	Ruta	Rol		Descripción
		Auth	mínimo	
GET	/sales/	Sí	Cualquiera	Listar ventas (search, status, payment_method, sort_by)
POST	/sales/	Sí	Cualquiera	Registrar venta (descuenta stock automáticamente)
GET	/sales/{id}	Sí	Cualquiera	Obtener venta por ID
POST	/sales/{id}/cancel	Sí	admin	Cancelar venta (revierte stock)

C.4.9 Movimientos de inventario (/inventory-movements)

Método	Ruta	Auth	Descripción
GET	/inventory-movements/	Sí	Listar movimientos (product_id, movement_type, sort_by)

C.4.10 Dashboard (/dashboard)

Método	Ruta	Auth	Descripción
GET	/dashboard/stats	Sí	Estadísticas consolidadas de la organización

Respuesta de /dashboard/stats:

```
{
  "sales_today": { "count": 5, "total": "250000.00" },
  "sales_month": { "count": 42, "total": "1850000.00" },
  "low_stock_count": 3,
  "average_ticket": "44047.62",
  "top_products": [
    {
      "product_id": "...",
      "product_name": "Producto A",
      "quantity_sold": 15,
      "revenue": "450000.00"
    }
  ],
  "sales_by_day": [{ "date": "2026-04-01", "count": 3, "total":
    ↪ "120000.00" }]
}
```

C.4.11 Utilidades

Método	Ruta	Auth	Descripción
GET	/utils/health-check/	No	Health check del servicio backend

C.5 Modelos de Datos

C.5.1 Organization

organizations

└─ id	UUID	PK
└─ name	VARCHAR(255)	NOT NULL
└─ slug	VARCHAR(100)	UNIQUE, INDEX
└─ description	TEXT	NULLABLE
└─ logo_url	VARCHAR(500)	NULLABLE
└─ is_active	BOOLEAN	DEFAULT true
└─ created_at	TIMESTAMPTZ	NOT NULL
└─ updated_at	TIMESTAMPTZ	NOT NULL

Índices: idx_organizations_slug (unique), idx_organizations_is_active

Validación del slug: expresión regular `^[a-z0-9-]{3,50}$` (solo minúsculas, números y guiones).

C.5.2 Role

roles

└─ id	INTEGER	PK (autoincrement)
└─ name	VARCHAR(50)	UNIQUE, INDEX
└─ description	TEXT	NULLABLE
└─ permissions	JSONB	DEFAULT []

└─ created_at TIMESTAMPTZ NOT NULL

Índices: idx_roles_name (unique)

C.5.3 User

users

└─ id	UUID	PK
└─ organization_id	UUID	FK → organizations.id, INDEX
└─ role_id	INTEGER	FK → roles.id, INDEX
└─ email	VARCHAR(255)	INDEX
└─ first_name	VARCHAR(100)	NOT NULL
└─ last_name	VARCHAR(100)	NOT NULL
└─ phone	VARCHAR(50)	NULLABLE
└─ avatar_url	VARCHAR(500)	NULLABLE
└─ hashed_password	VARCHAR(255)	NOT NULL
└─ is_active	BOOLEAN	DEFAULT true
└─ is_verified	BOOLEAN	DEFAULT false
└─ last_login_at	TIMESTAMPTZ	NULLABLE
└─ failed_login_attempts	INTEGER	DEFAULT 0
└─ locked_until	TIMESTAMPTZ	NULLABLE
└─ created_at	TIMESTAMPTZ	NOT NULL
└─ updated_at	TIMESTAMPTZ	NOT NULL
└─ deleted_at	TIMESTAMPTZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, email)

Índices: idx_users_organization_id, idx_users_role_id, idx_users_is_active

C.5.4 Category

categories

└─ id	UUID	PK
└─ organization_id	UUID	FK → organizations.id, INDEX
└─ parent_id	UUID	FK → categories.id, NULLABLE (jerarquía)
└─ name	VARCHAR(255)	NOT NULL
└─ description	TEXT	NULLABLE
└─ is_active	BOOLEAN	DEFAULT true
└─ created_at	TIMESTAMPTZ	NOT NULL
└─ updated_at	TIMESTAMPTZ	NOT NULL
└─ deleted_at	TIMESTAMPTZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, name, parent_id)

Índices: idx_categories_organization_id, idx_categories_parent_id, idx_categories_is_active

C.5.5 Product

products

└─ id	UUID	PK
└─ organization_id	UUID	FK → organizations.id, INDEX
└─ category_id	UUID	FK → categories.id, NULLABLE
└─ name	VARCHAR(255)	NOT NULL
└─ sku	VARCHAR(100)	NOT NULL
└─ description	TEXT	NULLABLE
└─ image_url	VARCHAR(500)	NULLABLE
└─ cost_price	NUMERIC(12,2)	DEFAULT 0
└─ sale_price	NUMERIC(12,2)	DEFAULT 0
└─ stock_quantity	INTEGER	DEFAULT 0
└─ stock_min	INTEGER	DEFAULT 0
└─ stock_max	INTEGER	NULLABLE
└─ unit	VARCHAR(50)	DEFAULT 'unit'
└─ barcode	VARCHAR(100)	NULLABLE

└ is_active	BOOLEAN	DEFAULT true
└ created_at	TIMESTAMPTZ	NOT NULL
└ updated_at	TIMESTAMPTZ	NOT NULL
└ deleted_at	TIMESTAMPTZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, sku)

Índices: idx_products_organization_id, idx_products_category_id, idx_products_is_active

Stock bajo: un producto está en alerta cuando $\text{stock_quantity} \leq \text{stock_min}$.

C.5.6 Customer

customers

└ id	UUID	PK
└ organization_id	UUID	FK → organizations.id, INDEX
└ document_type	VARCHAR(50)	NOT NULL (CC, NIT, Pasaporte, Otro)
└ document_number	VARCHAR(50)	NOT NULL
└ first_name	VARCHAR(100)	NOT NULL
└ last_name	VARCHAR(100)	NOT NULL
└ email	VARCHAR(255)	NULLABLE, INDEX
└ phone	VARCHAR(50)	NULLABLE, INDEX
└ address	TEXT	NULLABLE
└ city	VARCHAR(100)	NULLABLE
└ country	VARCHAR(100)	NULLABLE
└ notes	TEXT	NULLABLE
└ is_active	BOOLEAN	DEFAULT true
└ total_purchases	NUMERIC(12,2)	DEFAULT 0 (actualizado automáticamente)
└ purchases_count	INTEGER	DEFAULT 0 (actualizado automáticamente)
└ last_purchase_at	TIMESTAMPTZ	NULLABLE (actualizado automáticamente)
└ created_at	TIMESTAMPTZ	NOT NULL
└ updated_at	TIMESTAMPTZ	NOT NULL

└─ deleted_at TIMESTAMPTZ NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, document_number)

Índices: idx_customers_organization_id, idx_customers_email, idx_customers_phone, idx_c

C.5.7 Sale

sales

└─ id	UUID	PK
└─ organization_id	UUID	FK → organizations.id, INDEX
└─ customer_id	UUID	FK → customers.id, NULLABLE
└─ user_id	UUID	FK → users.id, INDEX
└─ invoice_number	VARCHAR(100)	NOT NULL
└─ sale_date	TIMESTAMPTZ	NOT NULL
└─ subtotal	NUMERIC(12,2)	DEFAULT 0
└─ discount	NUMERIC(12,2)	DEFAULT 0
└─ tax	NUMERIC(12,2)	DEFAULT 0
└─ total	NUMERIC(12,2)	DEFAULT 0
└─ payment_method	VARCHAR(50)	DEFAULT 'cash' (cash/card/transfer/other)
└─ status	VARCHAR(50)	DEFAULT 'completed' (completed/cancelled/pending)
└─ notes	TEXT	NULLABLE
└─ cancelled_at	TIMESTAMPTZ	NULLABLE
└─ cancelled_by	UUID	FK → users.id, NULLABLE
└─ cancellation_reason	TEXT	NULLABLE
└─ created_at	TIMESTAMPTZ	NOT NULL
└─ updated_at	TIMESTAMPTZ	NOT NULL

Restricciones: UNIQUE(organization_id, invoice_number)

Índices: idx_sales_organization_id, idx_sales_customer_id, idx_sales_user_id,
idx_sales_status, idx_sales_sale_date, idx_sales_invoice_number

Fórmula: total = subtotal - discount + tax

C.5.8 SaleItem

sale_items

└─ id	UUID	PK
└─ sale_id	UUID	FK → sales.id, INDEX
└─ product_id	UUID	FK → products.id, INDEX
└─ product_name	VARCHAR(255)	NOT NULL (snapshot al momento de la venta)
└─ product_sku	VARCHAR(100)	NOT NULL (snapshot)
└─ quantity	INTEGER	NOT NULL
└─ unit_price	NUMERIC(12,2)	NOT NULL
└─ subtotal	NUMERIC(12,2)	NOT NULL
└─ created_at	TIMESTAMPTZ	NOT NULL

Índices: idx_sale_items_sale_id, idx_sale_items_product_id

Los campos product_name y product_sku son snapshots del producto en el momento de la venta. Si el producto se edita posteriormente, el historial de ventas conserva los datos originales.

C.5.9 InventoryMovement

inventory_movements

└─ id	UUID	PK
└─ organization_id	UUID	FK → organizations.id, INDEX
└─ product_id	UUID	FK → products.id, INDEX
└─ user_id	UUID	FK → users.id, INDEX
└─ movement_type	VARCHAR(50)	NOT NULL (sale/adjustment/return/purchase)
└─ quantity	INTEGER	NOT NULL (positivo = entrada, negativo = salida)
└─ previous_stock	INTEGER	NOT NULL

└─ new_stock	INTEGER	NOT NULL
└─ reference_id	UUID	NULLABLE (FK al objeto de origen)
└─ reference_type	VARCHAR(50)	NULLABLE (sale/adjustment/return)
└─ reason	TEXT	NULLABLE
└─ created_at	TIMESTAMPTZ	NOT NULL

Índices: `idx_inventory_movements_organization_id`, `idx_inventory_movements_product_id`,
`idx_inventory_movements_user_id`, `idx_inventory_movements_movement_type`,
`idx_inventory_movements_created_at`,
`idx_inventory_movements_reference` (`reference_type`, `reference_id`)

C.6 Variables de Entorno

La configuración se carga desde el archivo `.env` usando Pydantic-Settings. Todas las variables se validan al iniciar el servicio.

C.6.1 Variables requeridas

Variable	Tipo	Descripción
<code>PROJECT_NAME</code>	<code>str</code>	Nombre del proyecto
<code>SECRET_KEY</code>	<code>str</code>	Clave para firmar los JWT
<code>POSTGRES_SERVER</code>	<code>str</code>	Host del servidor PostgreSQL
<code>POSTGRES_USER</code>	<code>str</code>	Usuario de PostgreSQL
<code>POSTGRES_DB</code>	<code>str</code>	Nombre de la base de datos
<code>FIRST_SUPERUSER</code>	<code>EmailStr</code>	Email del superusuario inicial
<code>FIRST_SUPERUSER_PASSWORD</code>	<code>str</code>	Contraseña del superusuario inicial

C.6.2 Variables opcionales con valores por defecto

Variable	Tipo	Default	Descripción
POSTGRES_PORT	int	5432	Puerto de PostgreSQL
POSTGRES_PASSWORD	str	""	Contraseña de PostgreSQL
ACCESS_TOKEN_EXPIRE_MINUTES	int	11520	Expiración del JWT (8 días)
FRONTEND_HOST	str	http://localhost:5173	URL del frontend para CORS
BACKEND_CORS_ORIGINS	list[str]	[]	Orígenes adicionales CORS
ENVIRONMENT	Literal	"local"	local staging production
SMTP_HOST	str\ None	None	Host SMTP para envío de emails
SMTP_PORT	int	587	Puerto SMTP
SMTP_TLS	bool	True	Usar TLS en SMTP
SMTP_SSL	bool	False	Usar SSL en SMTP
SMTP_USER	str\ None	None	Usuario SMTP
SMTP_PASSWORD	str\ None	None	Contraseña SMTP
EMAILS_FROM_EMAIL	EmailStr\ None	None	Email remitente
EMAIL_RESET_TOKEN_EXPIRE_HOURS	int	48	Expiración del token de reset
SENTRY_DSN	HttpUrl\ None	None	DSN de Sentry para error tracking

C.6.3 Propiedades calculadas (no en .env)

Propiedad	Descripción
SQLALCHEMY_DATABASE_URI	URI completa de conexión a PostgreSQL
all_cors_origins	BACKEND_CORS_ORIGINS + FRONTEND_HOST
emails_enabled	True si SMTP_HOST y EMAILS_FROM_EMAIL están configurados

C.7 Convenciones de Código

C.7.1 Backend (Python)

Orden de imports:

1. Biblioteca estándar

```
import uuid
from typing import Any
from decimal import Decimal
```

2. Terceros

```
from fastapi import APIRouter, HTTPException
from sqlmodel import SQLModel, Field
```

3. Locales

```
from app.api.deps import CurrentUser, SessionDep
from app.models import ProductCreate, ProductPublic
```

Nomenclatura:

- Funciones y variables: snake_case
- Clases y modelos: PascalCase
- Modelos de BD: singular (Product, Sale)
- Schemas: {Modelo}Create, {Modelo}Update, {Modelo}Public, {Modelo}sPublic
- Aliases de tipo: PascalCase con Annotated (CurrentUser, SessionDep)

Anotaciones de tipo: obligatorias en todas las funciones. Usar str | None (no Optional[str]).

Patrón de rutas:

```
@router.post("/", response_model=ProductPublic)
def create_product(
    *,
    session: SessionDep,
    current_user: CurrentUser,
```

```

    current_organization: CurrentOrganization,
    product_in: ProductCreate,
) -> Any:
    product = Product.model_validate(product_in,
    ↪ update={"organization_id": current_organization})
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

```

Códigos HTTP usados:

- 400 — Solicitud inválida (datos incorrectos)
- 403 — Sin permisos / credenciales inválidas
- 404 — Recurso no encontrado
- 409 — Conflicto (recurso duplicado)

Multi-tenancy: todas las consultas de negocio incluyen filtro `organization_id = current_organization`. El valor se extrae del usuario autenticado, nunca del cuerpo del request.

C.7.2 Frontend (TypeScript/React)

Orden de imports:

```

// 1. Externos
import { useState } from "react";
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { z } from "zod";

// 2. Internos (alias @/)

```

```
import { ProductService } from "@client";
import { DataTable } from "@components/Common/DataTable";
import { useCustomToast } from "@hooks/useCustomToast";
```

Componentes: solo funcionales, un componente por archivo, nombre de archivo en Pascal-Case.

Patrón de mutación:

```
const mutation = useMutation({
  mutationFn: (data: ProductCreate) =>
    ProductService.createProduct({ requestBody: data }),
  onSuccess: () => showSuccessToast("Producto creado"),
  onError: (err) => handleError(err),
  onSettled: () => queryClient.invalidateQueries({ queryKey: ["products"]
    ↪  }),
});
```

Validación de formularios: React Hook Form + Zod schemas con zodResolver.

No editar:

- src/client/** — auto-generado desde OpenAPI
- src/components/ui/** — componentes Shadcn/ui
- src/routeTree.gen.ts — auto-generado por TanStack Router

C.8 Pipeline de CI/CD

El repositorio incluye workflows de GitHub Actions:

C.8.1 Workflow de pruebas (en cada PR y push a main)

1. **Backend:** Ejecuta `uv run bash scripts/test.sh` — incluye Mypy, Ruff y Pytest con cobertura.
2. **Frontend:** Ejecuta `bun run lint` — Biome linting y verificación de tipos TypeScript.

C.8.2 Workflow de despliegue

Al hacer merge a main:

1. Construye las imágenes Docker de backend y frontend.
2. Las etiqueta con el hash del commit y con latest.
3. Las publica en el registro de imágenes.
4. Actualiza los contenedores en el servidor de producción via SSH.

C.9 Estructura de Pruebas

C.9.1 Backend (pytest)

backend/tests/

```
├─ conftest.py           # Fixtures globales: db, client, auth headers
├─ api/routes/
│   └─ test_login.py     # Autenticación, recuperación de contraseña
│   └─ test_users.py     # CRUD de usuarios, cambio de contraseña
│   └─ test_categories.py # CRUD de categorías
│   └─ test_products.py  # CRUD de productos, ajuste de stock
│   └─ test_customers.py # CRUD de clientes, historial de compras
│   └─ test_sales.py     # Registro y cancelación de ventas
│   └─ test_inventory_movements.py # Listado de movimientos
│   └─ test_dashboard.py # Estadísticas del dashboard
```

```

|   └─ test_private.py           # Endpoints de superusuario
└─ crud/
    └─ test_user.py
    └─ test_category.py
    └─ test_product.py
    └─ test_customer.py
    └─ test_sale.py
    └─ test_inventory_movement.py
    └─ test_dashboard.py

```

Fixtures disponibles en conftest.py:

Fixture	Descripción
db	Sesión de BD para pruebas (con rollback automático)
client	TestClient de FastAPI
superuser_token_headers	Headers con token del superusuario
normal_user_token_headers	Headers con token de usuario normal

Ejecutar pruebas:

```

cd backend
uv run bash scripts/test.sh           # Todos Los tests con cobertura
uv run pytest tests/api/routes/ -v     # Solo tests de API
uv run pytest -k "test_create" -v     # Tests por patrón

```

C.9.2 Frontend (Playwright E2E)

```

frontend/tests/
└─ auth.setup.ts                 # Configuración de autenticación compartida
└─ config.ts                    # Variables de configuración de tests
└─ login.spec.ts                # Flujo de inicio de sesión
└─ sign-up.spec.ts              # Registro de usuario

```

— organization-signup.spec.ts	# Registro de organización
— reset-password.spec.ts	# Recuperación de contraseña
— inventory.spec.ts	# Módulo de inventario
— sales.spec.ts	# Módulo de ventas
— customers.spec.ts	# Módulo de clientes
— admin.spec.ts	# Gestión de usuarios
— user-settings.spec.ts	# Configuración de cuenta
— utils/	# Helpers de pruebas E2E

C.10 Generación del Cliente API

El cliente TypeScript del frontend se genera automáticamente desde el esquema OpenAPI del backend:

```
# El backend debe estar corriendo en http://localhost:8000
cd frontend
bun run generate-client
```

La configuración del generador está en `frontend/openapi-ts.config.ts`. Los archivos generados en `src/client/` no deben editarse manualmente; cualquier cambio se sobrescribirá en la próxima generación.

C.11 Multi-Tenancy

OrbitEngine implementa multi-tenancy mediante **aislamiento por discriminador en tablas compartidas** (Shared Database, Separate Rows):

- Todas las tablas de negocio (`categories`, `products`, `customers`, `sales`, `sale_items`,

inventory_movements) incluyen la columna organization_id.

- La organization_id se indexa en cada tabla para garantizar rendimiento en consultas filtradas.
- Toda consulta de datos de negocio filtra por organization_id = current_user.organization_id.
- El valor de organization_id **nunca** se acepta como parámetro del request; siempre se extrae del token JWT del usuario autenticado.
- Las restricciones de unicidad incluyen organization_id como parte de la clave compuesta (ej. UNIQUE(organization_id, sku)), permitiendo que el mismo SKU exista en organizaciones distintas.

Esta estrategia garantiza aislamiento completo de datos entre tenants sin necesidad de esquemas o bases de datos separadas, facilitando el mantenimiento y la escalabilidad horizontal.

Documento generado como parte del proyecto de grado — Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.